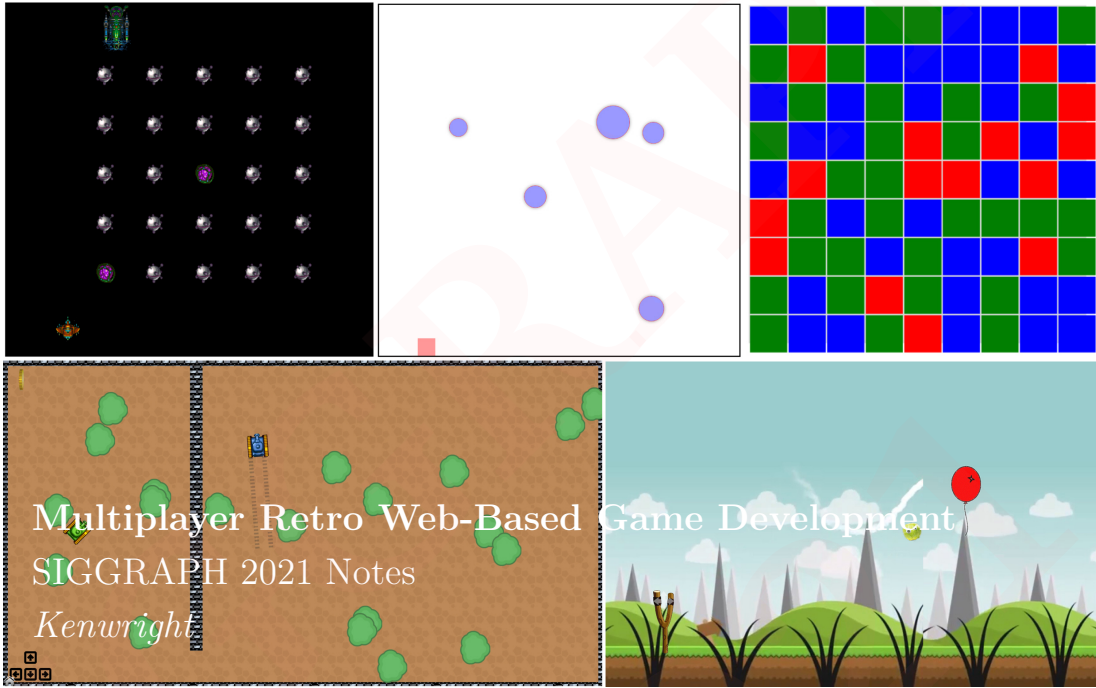




Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the Owner/Author. Copyright is held by the owner/author(s). SIGGRAPH '21 Educator's Forum, August 09-13, 2021, Virtual Event, USA. ACM 978-1-4503-8363-9/21/08.
10.1145/3450549.3464407

COURSE TITLE: Multiplayer Retro Web-Based Game Development
DOI: 10.1145/3450549.3464407

Revision: 017052021
Author: Kenwright



Brief Contents

1	Introduction and Overview	3
2	Web-Based Technologies	9
3	Game Mechanics	15
4	CSS and HTML	19
5	Javascript Basics	31
6	Javascript Advanced	55
7	Collisions and Intersections	59
8	Simulations and Physics	63
9	NodeJS	69
10	Node Packages/Modules	83
11	Building a Simple Website	89
12	Multi-Client Drawing	95
13	Chat-Client	107
14	Tic-Tac-Toe	113
15	Tips, Hints and Warnings	123
	Index	129

Contents



1	Introduction and Overview	3
1.1	Course Structure (Outline)	3
1.2	Multiplayer Online Games (vs Single-Player)	4
1.3	Who is this Course for?	5
1.4	Why Learn Multiplayer Game Programming?	6
1.5	Course Notes Convention	6
2	Web-Based Technologies	9
2.1	Internet	9
2.2	Web Browsers (simple web pages to complex interactive <i>video game</i> solutions)	9
2.3	Browser Compatibility	10
2.4	2D Games (HTML/CSS)	10
2.5	Document Object Model (DOM)	11
2.5.1	Recusing the Hierarchy	12
2.6	No Canvas	12
2.7	No Engines (Native/Vanilla Javascript)	13
2.8	Libraries (jQuery and Ajax)	13
2.8.1	Example, Waiting for the DOM to be loaded	13
3	Game Mechanics	15
3.1	What is a game?	15
3.2	Win or lose	15
3.2.1	Keeping players coming back for more	16
3.2.2	Fun Factor	16
3.3	Types of games (mazes, scrolling, ..)	16
3.4	Essential Concepts	17
4	CSS and HTML	19
4.1	Visual aspects	19
4.2	HTML is the content	20
4.3	Classes and IDs	20
4.3.1	ID or Class	20
4.4	CSS Animations	21
4.4.1	CSS @ Symbol	21

4.4.2	@keyframes Rule	21
4.5	Position (Relative, Fixed, Absolute)	22
4.5.1	CSS Position and Helper Properties	22
4.5.2	Static	23
4.5.3	Relative	23
4.5.4	Absolute	24
4.5.5	Fixed	25
4.5.6	Sticky	25
4.6	Transforms	26
4.7	Position (Translation) and Size	26
4.8	Rotation	27
4.8.1	Rotation around (predefined point)	27
4.8.2	Direction and Rotation	27
4.9	Depth (Z-Index)	27
4.10	Sprite Sheets	28
4.10.1	Showing Part of an Image	28
4.10.2	Animating the Sprite Sheet	28
4.11	CSS or Javascript	29
5	Javascript Basics	31
5.1	Creating CSS elements on the fly (createElement)	31
5.2	Generate HTML for the page (document.write)	32
5.3	Setting/Getting CSS Style information in Javascript	32
5.4	Variables let, var (or const)	32
5.5	Types of Variables	34
5.5.1	null	35
5.5.2	undefined	35
5.5.3	boolean	35
5.5.4	number	35
5.5.5	string	36
5.5.6	symbol	36
5.5.7	objects	36
5.6	typeof	36
5.7	Mutable or Immutable Data Types	37
5.7.1	Copy Array (or Object)	38
5.7.2	Pass-by-Value or Pass-by-Reference	38
5.8	Conditional logic	38
5.9	Loops	39
5.9.1	For loop	40
5.9.2	While loop	40
5.9.3	Do while loop	40
5.10	Functions	40
5.10.1	Modularize - functions to contain task	41
5.11	Classes and Objects	41
5.11.1	Four ways to define a Javascript object (class)	41
5.11.2	1. Function definition	42
5.11.3	2. Object literals	43
5.11.4	3. Singleton using a function	43
5.11.5	4. Classes (class keyword)	44

5.12	<code>use strict;</code>	44
5.13	debug assert (console)	45
5.13.1	Console.log isn't available?	45
5.14	Visually Debugging	46
5.15	Creating Elements	46
5.16	Debug Draw	46
5.17	Debug Output	46
5.18	Drawing Lines without Canvas	47
5.19	User input	48
5.19.1	Mouse input	49
5.19.2	Keyboard Input	49
5.19.3	Reacting to Key Events	49
5.20	Timer Events (Timing Information)	50
5.21	Events and Elements	50
5.22	Game Loop	51
5.22.1	requestAnimationFrame vs setInterval (or setTimeout)	51
5.22.2	Document Object Model (Ready?)	51
5.22.3	Timing Problems	51
5.23	Rotating Elements to Face a Particular Direction	52
6	Javascript Advanced	55
6.1	Data (JSON)	55
6.1.1	Javascript JSON helper functions	55
6.2	Pausing and Continuing Animations (Overriding CSS)	56
6.3	Rotating and Animating (with CSS)	56
6.4	Rotating Programmatically	57
6.5	Memory Management	57
7	Collisions and Intersections	59
7.1	Collision vs Intersection	59
7.2	Elements On Screen (Screen-Box-Area)	59
7.3	Mouse-Element Check (Point in Box)	60
7.4	Axis-Aligned Bounding Box (AABB)	60
7.5	Sphere-Sphere	60
7.6	Shape Hierarchies/Decomposition	61
7.7	Problems	62
8	Simulations and Physics	63
8.1	Newton's Laws (Basics)	64
8.2	Relationship between position, velocity and acceleration	64
8.3	Forces to Control Movement	65
8.4	Vectors (2-dimensional movement)	65
8.5	Time Step (Fixed)	67



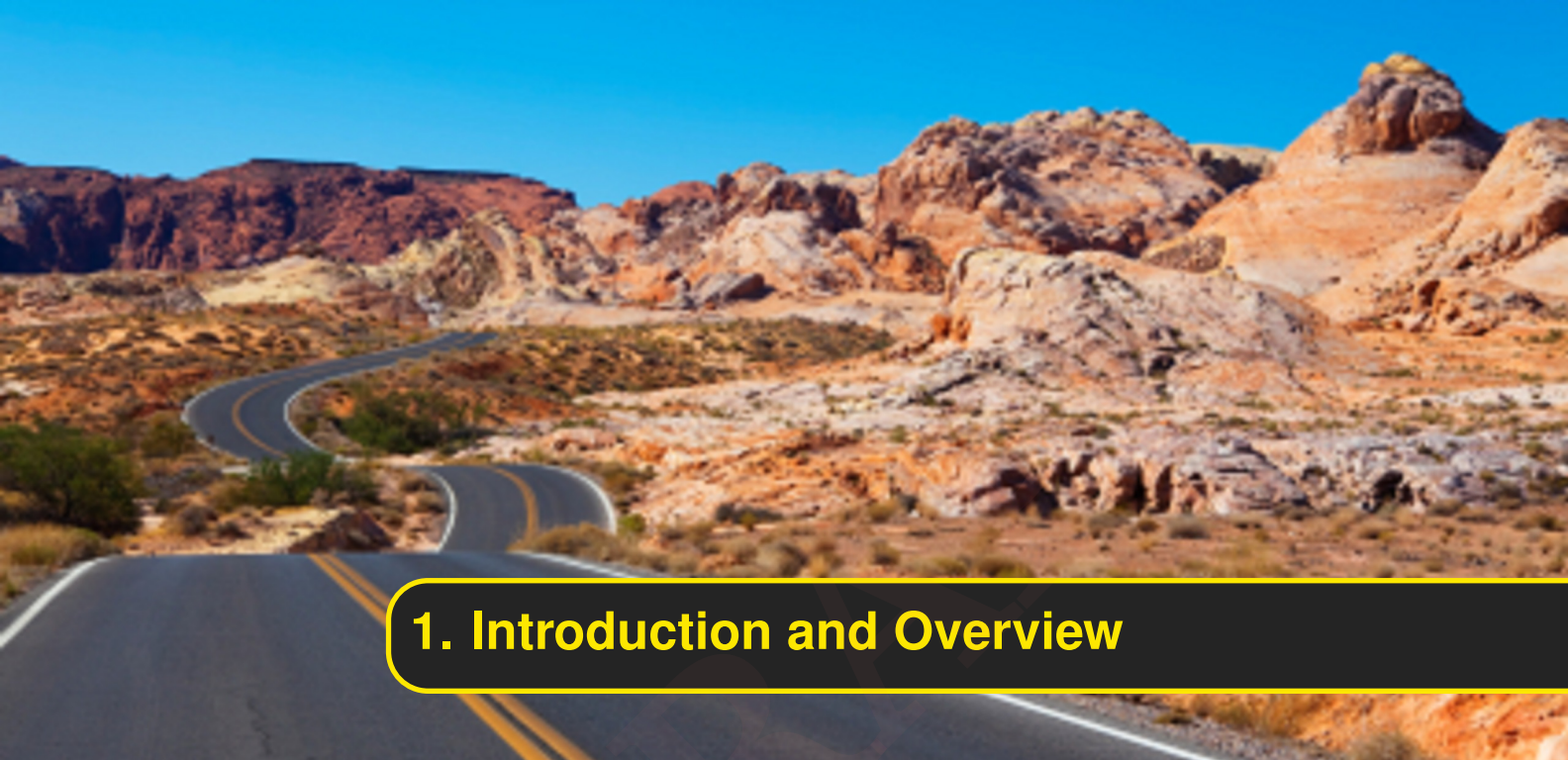
9	NodeJS	69
9.1	What is NodeJS?	69
9.2	Why Learn and Use NodeJS?	69
9.3	Who's using NodeJS?	71
9.4	NodeJS Architecture (Single-Threaded)	72
9.4.1	Events	72
9.5	Getting Started with NodeJS?	72
9.6	Installing and Testing NodeJS	72
9.6.1	Locally	72
9.6.1.1	Where to find NodeJS?	72
9.6.1.2	Hello NodeJS	74
9.6.1.3	Text Editors and 'js' Files	74
9.6.2	Online Tools	75
9.6.2.1	Simple Online Test	76
9.6.2.2	Online Editor	76
9.6.3	Alternatives to NodeJS (e.g., NodeJS vs PHP)	77
9.7	Summary	78
9.8	Questions	78
10	Node Packages/Modules	83
10.1	What is a Module?	83
10.1.1	Built-in Modules	83
10.2	Difference between a package vs module?	84
10.2.1	What is the file 'package.json'?	84
10.3	What is NPM (Node Package Manager)?	84
10.4	Initializing your project	85
10.4.1	Installing modules	86
10.4.2	Using Modules	87
10.4.3	Standard Modules	87
10.4.4	Creating your own custom modules	87
10.5	Questions	88
11	Building a Simple Website	89
11.1	Creating the web-server	89
11.1.1	Calling the HTTP module	89
11.1.2	Create the server	90
11.1.3	Sending responses	90
11.1.4	Run your NodeJS site!	91
11.1.5	Open your NodeJS site in a browser	91
11.2	Paths	91
11.2.1	Acting based on the browser path	92
11.2.2	Files	92
11.3	Summary	93
11.4	Questions	93
12	Multi-Client Drawing	95
12.1	Client-server communication	95
12.2	Web-Sockets	95
12.2.1	How do Web-sockets Work?	96

12.2.2	Web-socket (socket.io)	96
12.2.3	Tick-Tock	96
12.2.3.1	Establishing the Connection	97
12.2.3.2	Sending Data from Server to Client	98
12.2.3.3	Sending Data from Client to Server	98
12.2.3.4	Testing your Program	98
12.2.3.5	Important Parts	99
12.3	Sending Mouse Information	99
12.4	CANVAS (Drawing)	102
12.5	Further Ideas	104
13	Chat-Client	107
13.1	Sending Key-Presses	107
13.2	Chat Demo	108
13.3	Summary	111
13.4	Further Ideas	111
14	Tic-Tac-Toe	113
14.1	Further Ideas	120
15	Tips, Hints and Warnings	123
15.1	When shouldn't you use NodeJS?	123
15.2	Never forget about security!	123
15.3	Using Strict Mode In Coding	124
15.4	Node shell	124
15.5	Global variable make a good Cache	125
15.6	Asynchronous looping	125
15.7	Quick and simple logging	126
15.8	Avoid using asterix * for dependencies	126
		
	Index	129

Preface

Developing online multiplayer games is an exciting and important topic to learn, not just because of entertainment value, but also the ability to manage multi-user interactions online. This course uses step-by-step game-based examples to teach multiplayer web-based gaming concepts - combined with tutorials, explanations and projects. Ultimately the goal of this course is to help you create dynamic online web-based solutions that users can interact with in an engaging, fun and playful way (not just the software but also other users in real-time).

Ultimately, these notes are to help you go beyond basic web-pages, towards more interactive multi-user solutions by opening up the hood and taking a look inside. By understanding how web-pages are able to actively communicate, you can develop innovative web-based solutions (like multi-player video games).



1. Introduction and Overview

Due to the number of areas covered within this course, the focus of the examples/demos will be on retro games (i.e., 2D games). As we believe, it's better to cover the key components of multiplayer games instead of overwhelming the listener with lots of different techniques, libraries and technologies.

Hence, the course builds upon advanced HTML5 (HTML/CSS/JS) and NodeJS to create real-time interactive gaming demos that provide the platform for larger projects. Once the learner is comfortable with the underlying mechanics (e.g., managing data, interactions, user-inputs and client-server concepts), they can continue to integrate advanced features available in most browsers, such as WebGL API, Geolocation API and Speech API.

1.1 Course Structure (Outline)

1. About this course
2. What this course is 'not'
3. Javascript, CSS, HTML (Core Concepts) Document Object Model Tags and Identifiers
4. Common Mistakes/Traps (Javascript)
5. Creating, editing and manipulating elements on the fly (words of warning, good practices)
6. Demos example of what can be done in a browser (e-books, word-processors, games, 3D/2D, ...)
7. Game-based examples to learn from (interactive and dynamic projects to show the potential of how far you can push a browser)

8. Moving objects around (editing/manipulating on-the-fly)
9. Creating a retro classic game from scratch (asteroids/breakout)
10. Giving users more control Creating an editor (add/remove text, images, layout)
11. Introduction to NodeJS (Adding client-server communication)
12. Online Multi-User Drawing Program (share mouse information)
13. Tank Game (step by step from the login screen to placing the tanks in the ‘common’ virtual world and moving them around)
14. Future projects/enhancements (what next)
15. Summary and discussion

1.2 Multiplayer Online Games (vs Single-Player)

Web-based technologies (one of the greatest achievements) has allowed online multiplayer games to flourish. Although the changing tastes of gamers also play a huge role, the number of multiplayer games getting³ released today outweigh single-player ones.

Online multiplayer games are on the rise, especially for new titles, such as Valorant, League of Legends, Fall Guys and Fortnite over Mass Effect, Red Dead Redemption and Witcher 3. The reasons for such a popularity switch are many, ranging from technical developments to exciting gameplay options.

Hint: Shift Towards Online Multiplayer Games

If you have long been thinking of working on a multiplayer online game - then this course is for you. You’ll learn the underlying mechanics and approaches of online development using the latest web-based technologies. This enable you to quickly prototype and run online games that are stable and interactive, while allowing multiple players to simultaneously communicate with one-another in real-time.

Exciting Like it or not, most online multiplayer games are more exciting compared to single-player titles regarding the gameplay modes. It is quite common for most shooters to have a Battle Royale, Capture the Flag, Elimination, Free for All, and other game modes available. In the case of single-player games, gamers are limited to available campaigns that are usually short in gameplay time. For that sole reason, such franchises as Fortnite are becoming more popular over time.

Online Games Are More Flexible

Social and Content Aspects Another convincing reason worth mentioning is that online multiplayer games are usually updated in terms of content. Did you know that World of Warcraft was released back in 2004? This game is still active, attracting millions of people to its new expansions and content. The same goes for virtually any popular multiplayer game that is popular among its audience. While single-player games started innovating their graphics, in-game features, and storytelling, some online multiplayer games focused merely on gameplay. That’s

why new genres appeared, such as MOBAs, aka Multiplayer Online Battle Arenas, with such games as Dota 2 or League of Legends attracting millions of fans. Those games allow players to develop skills and even become esports athletes to compete for multi-million dollar prize pools.

Some online multiplayer games are truly unique regarding the **social component** of gameplay. For instance, some games feature role-playing servers where people can create their alter egos and act accordingly to the world's setting.

Warning: While online games are more exciting and offer unlimited potential - you must also be aware of the software engineering complexities and security considerations.

Multiplayer Games In today's changing world, gamers are prone to accessing online multiplayer games more often than ever before. Even though iconic single-player games are still being released, the overall tendency to switch to multiplayer is noticeable. These games offer more entertainment, but they also carry an essential social aspect that is crucial for most players.

Don't forget that all those changes became possible over time, thanks to the development of new technologies, libraries and tools. Developers should think twice when they're deciding in they should create a single or multiple player game, embracing multiplayer games may also help realize what made a game so astonishing great can be even greater!

1.3 Who is this Course for?

This course is designed for anyone who wants to get started developing multiplayer interactive games (using web-based technologies as a spring-board). You will learn how to design and build responsive and interactive web-based games that are both fun and dynamic (and extensible). The course introduces basic concepts and features, from responsive web design through to the latest Javascript, HTML and CSS technologies.

Examples: Academics The course would provide insightful examples and material to help *teachers, instructors* or anyone involved in *education* and *learning* to develop bespoke *interactive* learning solutions (e.g., game-based projects to *teach students* mathematics, physics or programming principles in a creative and fun way)

Hobbyists The course offers multiple projects to help beginners master the topic of web technologies by implementing and enhancing simple self contained *retro games* (fun factor).

Web-Artists/Designers The course provides information and insights on how to stretch what the capabilities of websites, e.g., *programmatically* alter the content on the fly, interact and explore web content in new and interesting ways and more.

This course will open your mind to new ideas, while giving you the opportunity to acquire new skills and extensive knowledge. The material is practical based enabling you to take a hands-on

approach to creating demos/and working solutions that you can use in the real-world (i.e., not just theory).

1.4 Why Learn Multiplayer Game Programming?

One of the best parts of this retro, is that developing **your own retro games** that can be played with other people online - which is incredibly **fun**. You'll certainly run into the same pitfalls that you face in regular development: times of frustration, not knowing how to proceed, and even feeling like you have to fight against the tools you're using. However, developing retro games with HTML/CSS/Javascript lends itself to an entertaining development process. You get to play and test your game as you go, and there's nothing like the feeling of finally having enough of your game made that you get to share it with others (through a browser).

Hopefully, after this course, you'll be convinced that retro game development with web technologies is for you. If you're wondering where to start, this course is for you. We'll take you through the core concepts and provide practical (simplified) examples for you to work through. It does have a somewhat steep learning curve, but it is incredibly flexible and powerful, and learning problem solve and develop various solutions with it will enable you to create practically any game you'd ever want to.

Prerequisites

While the material in these notes are introductory, it assumes you to have some background programming knowledge (for example the ability to understand basic mathematical concepts and conditional logic). All of the demos and examples are explained, not to mention, available for you to run and test out. So they could be used in conjunction with other learning material (e.g., while you're learning Web-Design, *Javascript*, *HTML* and *CSS*).

1.5 Course Notes Convention

Code samples will be given in blocks (highlighted and easy to identify and copy/type out):

```
1 var a = 2; // assign variable `a` the value 2
2 let b = 3;
3
4 // This is a one line comment
5 /*
6    This is a multi-line comment
7 */
```

Also throughout the notes, there are various tips and *comments* - emphasizing and highlighted as shown below:

Get into the habit of **commenting** your scripts.



2. Web-Based Technologies

This section provides an introduction to a number of core web-based technologies. Not to mention, the history and the advantages of web-based development, both for video games and other interactive applications.

2.1 Internet

The internet has revolutionized your world both with its technical *capabilities* and the ability to connect with billions of other users across the world. While the majority of the text focuses on browser based video game solutions, the internet is so much more. Especially, now that *internet* access is becoming popular on *mobile* phones (mobile apps), not to mention, the *internet-of-things* (IoT), social media and streaming content.

2.2 Web Browsers (simple web pages to complex interactive *video game* solutions)

The *web*, in terms of user experience and functionality has changed enormously over the past decade, thanks to HTML5 and CSS3 and the evolution of web browsers (technologies).

The advantages of browser based solutions, such as, games, offers greater reach, accessibility from multiple device types and benefits from connecting with search, social media, email and affiliates. The shackles of limited features, user experience (UX) and functionality has disappeared. The combination of new web technologies, and extended Javascript resources, means you have the ability to tap into your device features, like location, camera, not to mention, the ability to work offline, so you can deliver interactive games with great user-experience - all via a web browsers.

As a simple example *vibrate* feature on a mobile phone:

```
1 window.navigator.vibrate(200); // vibrate for 200ms
2 window.navigator.vibrate([100, 30, 100, 30, 100, 30, 200, 30, 200, 30, 200, 30, 100,
  ↪ 30, 100, 30, 100]); // Vibrate 'SOS' in Morse.
```

In Javascript, if your device supports the vibrate hardware and is enabled in your browser, then the `Navigator.vibrate(..)` method pulses the vibration hardware on the device. Of course, if the device doesn't support vibration, this method has no effect.

2.3 Browser Compatibility

All browsers are not made equal! (e.g., Chrome, Firefox, 360 Secure Browser, ..).

Always worth testing your retro games on different browsers. *Cross browser testing* important! If your retro game is not tested and debugged on different platforms and browsers, it won't work the same on all of them, causing inconvenience and sadness to your players.

While you might think you'll need to download and install every single browser since the beginning of time - a good place to start testing for cross browser issues, is to lookup which browsers support which features.

For example, if you want to check your current browser's capabilities, there are automated solutions available.

Online tools, such as, <https://html5test.com/>, test which features your browser support and evaluating your **browser's ranking**.

2.4 2D Games (HTML/CSS)

Today people are overloaded with content and there is a trend for ever more realistic interactive 3D graphics, so you might be wondering if *2D games* are out dated? Boring? NO! A 2D game can be just as fun, engaging and exciting as a high-end 3D commercial game.

Previously, web developers relied mostly on Flash to create and deploy games and animations on the web - an expensive undertaking. Now with the latest HTML and CSS web-based games and animations can be both powerful and attractive while being easy to develop. As you'll learn in

this course, it's relatively painless to create stunning interactive games with only HTML, CSS, and JavaScript (in fact it's fun).

2.5 Document Object Model (DOM)

The *Document Object Model (DOM)* is one of the most popular web-based concepts you need to master. The *DOM* is efficient (in terms of memory), and easy to work with, offering amenities like click event handlers and visual elements. It's relatively straightforward to create/remove and access objects in the DOM hierarchy (e.g., both through html and dynamically using Javascript).

For example, this is what a simple HTML web page DOM might look like:

```

1 document
2 |
3 +-root <html>
4 |
5   +- element <head>
6     +- element <title>
7       +- text "my title"
8     ...
9   +- element <body>
10     +- element <h1>
11       +- text "a heading"
12     +- element <a>
13       +- text "link text"
14     ...

```

The equivalent html file for this DOM hierarchy would look like:

```

1 <html>
2   <head>
3     <title>my title</title>
4   </head>
5   <body>
6     <h1>a heading</h1>
7     <a>link text</a>
8   </body>
9 </html>

```

You need to notice certain elements, such as `document` which contains the `root`. You'll always have access to the `document`, which means, you'll always be able to search and access any elements in the hierarchy. You'll give your elements unique `IDs` and `classNames` so that you're able to identify them.

To *create*, *add*, *remove*, and *find* elements in yourfrom the DOM hierarchy on the fly (in Javascript) you have access to the following functions:

- `createElement(...)`
- `appendChild(...)`
- `removeChild(...)`
- `getElementById(...)`

Example (Adding element to the DOM hierarchy)

```
1 var node = document.createElement("div"); // Create<div> node
2 node.id = "myid"; // set the unique ID for the node
3 node.className = "someClass"; // associate the node with a class
4 document.body.appendChild( node ); // add the node element to the DOM hierarchy
```

Example (Removing element DOM hierarchy)

```
1 // any time we want the element, we can find it using its unique ID
2 var mydiv = document.getElementById("myid"); // Get the <div> element
3 mydiv.parentNode.removeChild( mydiv ); // Remove the element if we need to from the
    ↪ hierarchy
```

2.5.1 Recusing the Hierarchy

Each element in the *hierarchy* is connect (i.e., it knows who its parent and how it's children are). Using the **parentNode** and **childNodes** member variables you can at any time crawl or recurse over the Document Object elements.

As you become more familiar with the DOM API, you'll notice there is a **parentNode** and a **parentElement** accessor. What is the difference? The documentation says that **parentElement** is only for listing connected **elements**, however, you might ask, can objects in the DOM hierarchy not be elements? The **parentElement** property returns the **element** parent, while **parentNode** returns **any node** parent. These properties are usually the same: they both get the parent.

With the one exception of `document.documentElement`:

```
1 alert( document.documentElement.parentNode ); // document
2 alert( document.documentElement.parentElement ); // null
```

Similarly there are **childElements** and **childNodes** accessor functions.

2.6 No Canvas

The games and concepts at the start of these notes are dependent on the Document Object Model (DOM) and browser capabilities (leverage CSS/HTML features). You don't need to be using *Canvas* graphical component (yet). The HTML *Canvas* element is for low-level graphical control, such as, pixel-by-pixel drawing. While Canvas offers finer grained control over rendering but comes at the cost of having to manage every detail manually, like hover state animations. The biggest reason to use Canvas is when you start to develop detailed graphical effects (e.g., draw/generate content on the fly) in the browser. However, for the majority of the retro games, you'll not need canvas, and will use graphical components like gifs and jpgs for visual details not to mention style sheet animations/effects (CSS).

You will use Canvas a bit, for example, in later examples when you want to draw the mouse cursor movement (multi-user drawing program) - easier and less work to use Canvas to draw the lines (versus doing them using DOM).

2.7 No Engines (Native/Vanilla Javascript)

The focus of this course is on developing interactive retro games from the ground up using Vanilla Javascript/CSS/HTML, and not external game *engines* or pre-written game-frameworks. You'll learn about the low-level mechanics, collision detection algorithms and optimization techniques - implement them as you need them for your game.

This provides you with a more comprehensive understanding, not to mention more flexibility (and power).

2.8 Libraries (jQuery and Ajax)

AJAX is an acronym standing for **Asynchronous JavaScript and XML** and this technology will help you load data from servers without a browser page refresh (e.g., upload and download high score data on the fly without needing to refresh the page). As you start to master game development with web-technologies, you'll find Ajax an invaluable tool for any online communication (multiplayer or downloading/uploading information). The *AJAX* syntax (when used with *jQuery*) is just Javascript (and is greatly simplified, e.g., an AJAX call can be a single line - with *jQuery*):

```
1 $.ajax({ url: "test.html", success: function(){/*do stuff here*/}});
```

jQuery is a great tool which provides a rich set of methods to develop next generation web application (compact syntax).

While not covered in this course, it's still worth trying out other libraries and frameworks, once you've mastered the basic gaming concepts. You might want to explore integrating some of your game programming concepts with **React** and **Angular**.

The important thing to remember when developing your games, is that you understand the concepts, and can implement them either in vanilla Javascript or another language (i.e., won't be dependent on a specific library like *jQuery*).

2.8.1 Example, Waiting for the DOM to be loaded

The *jQuery* way:

```
1 $(document).ready(() => {  
2     //...  
3 })
```

The Javascript/DOM way:

```
1 document.addEventListener("DOMContentLoaded", () => {  
2     //...  
3 })
```



3. Game Mechanics

3.1 What is a game?

A game is an *activity* - typically this is for *pleasure* (with or without a purpose). You might think, a game always has a **purpose**, however, imagine playing with model trains or dolls (not always about conquest or killing).

While fun is great. A game needs rules. Rules are really useful. Rules that **everyone** knows. Rules shouldn't be a secret. They help make sense of what is happening and why. A common place you'll see rules, is in board games.

- What are types of game rules?
- Can *game rules* be broken?
- Rules that are *laws*

3.2 Win or lose

While winning or losing is important, you always want to keep the player coming back for more. For example, if the player, plays the game for hours and is killed and has to start again from the beginning, he'll be very annoyed, he might actually not play again. Instead, you should always offer the player some incentive, to keep playing, new storylines, save their progress, freedom to

explore, and more.

3.2.1 Keeping players coming back for more

A few ideas, include:

- High scores (peer recognition)
- New levels
- A challenge (stretch the players abilities)

3.2.2 Fun Factor

While the game should have some reward/punish element, remember, ultimately you need to have a *fun factor*. Fun, FUN **FUN**. Which is more than simply exploring and collecting points. You need to stimulate and challenge the player (but also tease and nurture their curiosity - keep them interested and wanting more).

The player needs to feel fulfilled when playing your game. After all, it not just about 4K graphics, online multiplayer, and microtransactions is it? You need to understand what your player wants - actually one of the reasons why players will take the time to play your game.

A web browser is a window into a world of immersive interactive content - players can play any time of game at the click of a button, from Street Fighter type games to Super Mario platformers. While consoles and high performance graphics are important, 20 years into the new millennium, players are still thirsting for retro type games. It's important to recognize how a well designed retro games is able to stimulate players. Previously, in the early days of computer games, developers had to squeeze every bit of work out the system (e.g., your average Sega Mega Drive or SNES cartridge had a maximum capacity of just 4MB - and limited graphics and sound capabilities) - yet with a web browser, you have access to limitless resources. When you design your retro game, you should be thinking of how to make a memorable game. People will play your game because they're interesting and novel (and of course fun).

3.3 Types of games (mazes, scrolling, ..)

Not all games are the same. Games such as Tetris or sonic the hedgehog appeal to different people. When you start to design your game, you'll be thinking of a specific *type*. There are many different types of video games, and typically, they're categorized by their characteristics or underlying objectives - not by the type of gameplay they contain.

While there are all sorts of game genres, your video gaming options are plentiful. Not only are there a number of different types of video games to explore, each with its own set of successful subgenres. Combined with the fact that technology is constantly evolving, and you've got a

fertile interactive landscape! For your imagination and anyone wanting to learn the ins and outs of game development (in hopes of creating the next big hit) exploring video game genres can be the start of something big. Nothing stopping you taking any game genre and making it unique to your design... and then some.

3.4 Essential Concepts

A few common concepts/techniques which you'll cover, include:

- Place images on the screen
 - Background
 - Environment
 - Movers (player/enemies)
- Animate images (GIF or Sprites)
- Interactive (user input, control)
 - Mouse/keyboard
- Collision detection (navigate and interact with the environment)
 - Detection collision (true/false)
 - Intersection (handling collisions)
- Reward/penalty system (goal or purpose)
 - Enemies (avoid/help)
 - Points (collect/reward)
- Physics
 - Gravity
 - Forces (mass/velocities)



4. CSS and HTML

This Chapter will review the essential *CSS* and *HTML* concepts before jumping into the programming aspect. *HTML* provides the basic structure, which is enhanced and modified by other technologies like *CSS* and *Javascript*. You'll use the *CSS* to control **presentation (visual)**, formatting and layout aspects. In the next Chapter, you'll dive into *JavaScript*, which gives you the ability to **control** the behavior of different elements. *Javascript* is the dynamic aspect (*interactive* logic).

4.1 Visual aspects

When we say visual aspects, this includes, transparency, background color, border and more. The *CSS* is about defining the *visual* aspects of the web page.

If you use *PNG* files for your graphics rather than *JPGs*, you can keep part of the image transparent

In a browser and in your game, the graphics (images) are important. When we define an element (game object on screen), what you'll be doing most of the time is saying to the browser *go and get that image*. In your game, you'll store your images in a directory (to keep things tidy). Ideally, a directly called **images**, so you remain organised! Then in your script, when you need your image called **tree.jpg** you'll be able to find it easily in your images folder. While you can use any image format you want (*BMP*, *GIF* or *JPG*), it is recommended that your images are *PNG* files, as that

file type provides good-quality images with **transparencies**. A free and powerful online image editor is: **www.lunapic.com** - enabling you to edit your images from any computer (no need for installation, and includes a variety of special effects and advanced features).

4.2 HTML is the content

When you place content on the screen, it will have an associated **style** - which is defined by the *CSS*. This can be as simple as the starting position or as complex as a key-frame animation sequence.

4.3 Classes and IDs

- (1) *ID* - defined using a *hash* symbol before the name in CSS (.) *Class* - defined using a *dot* symbol before the name in CSS

Class	HTML
# (Hash)	. (Dot)
Share multiple elements	Only one ID per element

```

1 .myname {
2   color: blue;
3 }
4 #anothername {
5   color: green;
6 }

```

:thinking_face: Can an element have the same name for the ID and Classname? **Yes**

While you can set and configure the majority of your CSS in Javascript, creating a *stylesheet* allows you to customize your game look without having to constantly modify the code (possibility of having multiple games that look differently by simply changing the style sheet (.css file)).

4.3.1 ID or Class

When you want to access or identify a specific object on screen, you use an ID, however, a class is used to identify *groups* of items with similar attributes. You might have lots of space ships on screen, with similar attributes, these could all share a common *class* definition.

4.4 CSS Animations

CSS allows *animation* of HTML elements without using Javascript. You'll use the CSS advanced properties to create CSS animations (without Javascript). The animations can be as simple as *pulsating* (growing and shrinking) for buttons, rotating/spinning (wheels), walking motion or eyes on a characters face.

4.4.1 CSS @ Symbol

The **@** symbol is for CSS rule**** statements that instructs **CSS** how to behave.

Name	Details
@keyframes	Specifies the animation code
animation	A shorthand property for setting all the animation properties
animation-delay	Specifies a delay for the start of an animation
animation-direction	Specifies whether an animation should be played forwards, backwards or in alternate cycles
animation-duration	Specifies how long time an animation should take to complete one cycle
animation-fill-mode	Specifies a style for the element when the animation is not playing (before it starts, after it ends, or both)
animation-iteration-count	Specifies the number of times an animation should be played
animation-name	Specifies the name of the @keyframes
animation-play-state	Specifies whether the animation is running or paused
animation-timing-function	Specifies the speed curve of the animation

4.4.2 @keyframes Rule

To get the CSS animation to work, you must bind the animation to an element using the *@keyframes* rule. You do this inside the CSS style. You then specify the animation details causing the style information to gradually change from the current style to the new style at certain times.

```

1 <html><head></head><body>
2 <style>
3 div {
4   width: 100px; /* div width */
5   height: 100px; /* div height */
6   position: relative; /* div position details */

```

```

7   background-color: red; /* div starting color */
8
9   animation-name: myrulename; /* which animation to use */
10  animation-duration: 4s; /* how long is the animation */
11  animation-iteration-count:infinite; /* loop forever */
12 }
13
14 /* The animation code */
15 @keyframes myrulename{ /* blend between */
16   /* keywords "from" and "to" (represents 0% (start) and 100% (complete)). */
17   0% {background-color: red; left:0px; } /* 0 seconds */
18   50% {background-color: yellow; left:200px;} /* 2 seconds */
19   100% {background-color: red; left:0px; } /* 4 seconds */
20 }
21 </style>
22
23 <!-- your div box which will be animated -->
24 <div>BOX</div>
25
26 </body></html>

```

4.5 Position (Relative, Fixed, Absolute)

When you add objects to the DOM they'll have an associated *position* associated to them. Defining how they'll be positioned on the screen. Positioning elements with CSS in web development isn't as easy as it seems. Things can get quickly complicated as your game gets bigger and without having a good understanding of how CSS deals with aligning HTML elements, you won't be able to fix your alignment issues.

There are different ways/methods for positioning elements with pure CSS. Using CSS **float**, **display** and **position** properties are the most common methods.

In this course, you'll primarily be concerned with the **position** property. The position property determines how your **position** values are interpreted (e.g., relative to the parent or absolute on the screen).

4.5.1 CSS Position and Helper Properties

So there are **five** main values of the **Position** property:

`position: static | relative | absolute | fixed | sticky`

and additional properties for setting the coordinates of an element (let's call them **helper properties**):

`top | right | bottom | left` AND the `z-index`

Helper properties don't work without a declared position, or with `position: static`.

This might be also a good time to mention the **z-index**. You have height and width (x, y) as 2 dimensions. **Z** is the 3rd dimension. An element in the web-page comes in front of other elements as its z-index value increases.

Z-index doesn't work with position: static or without a declared position

Elements are **ordered** from **back to front**, as their **z-index** increase

As mentioned earlier, there are **five** property values for **position**, each of them are discussed next, so you can see their pros and cons.

4.5.2 Static

The position: **static** is the **default value**. Whether we declare it or not, elements are positioned in a normal order on the web-page. Let's do through an example:

First, you define your HTML structure:

```
1 <body>
2   <div class="box-green"></div>
3   <div class="box-blue"></div>
4 </body>
```

Then, you create two boxes and define their widths, heights and positions:

```
1 .box-green{ // without any position declaration
2   background: green;
3   height: 100px;
4   width: 100px;
5 }
6
7 .box-blue {
8   background: lightskyblue;
9   height: 100px;
10  width: 100px;
11  position: static; // Declared as static
12 }
```

The same result with and without position: **static**.

As you should understand, defining position: **static** or not doesn't make any difference. The boxes are positioned according to the normal document flow.

4.5.3 Relative

The position: **relative**: An element's new position relative to its normal position.

Starting with position: **relative** and for all non-static position values, you'll be able to change an element's default position by using the helper properties mentioned previously.

Move the green box next to the blue one.

```
1 .box-green{
2   position: relative; // We are now ready to move the element
3   background: green;
4   width: 100px;
5   height: 100px;
6   top: 100px; // 100px from top relative to its old position
7   left: 100px; // 100px from left
8 }
```

Green box is moved 100px to bottom and right, relative to its normal position.

Using position: `relative` for an element, doesn't affect other elements' positions.

4.5.4 Absolute

In position: `relative`, the element is positioned relative to itself. However, an absolutely positioned element is relative to its parent.

An element with position: `absolute` is **removed from the normal document flow**. It is positioned automatically to the starting point (top-left corner) of its parent element. If it doesn't have any parent elements, then the initial document `<html>` will be its parent.

Since position: `absolute` removes the element from the document flow, other elements are affected and behave as the element is removed completely from the web-page.

Add a container as parent element:

```
1 <body>
2   <div class="container">
3     <div class="box-green"></div>
4     <div class="box-blue"></div>
5   </div>
6 </body>
7 .box-green{
8   position: absolute;
9   background: green;
10  width: 100px;
11  height: 100px;
12 }
```

The position: `absolute` takes the element to the beginning of its parent. Now it looks like the blue box has disappeared, but it hasn't. The blue box behaves like the green box is removed, so it shifts up to the orange box's place.

Move the green box by 5 pixels:

```
1 .box-green{
2   position: absolute;
3   background: green;
```

```
4 width: 100px;
5 height: 100px;
6 left: 5px;
7 top: 5px;
8 }
```

You should be able to see the blue box. The coordinates of an absolute positioned element are relative to its parent if the parent also has a non-static position. Otherwise, helper properties position the element relative to the initial `<html>`.

```
1 .container {
2   position: relative;
3   background: lightgray;
4 }
5
6 .box-green{
7   position: absolute;
8   background: green;
9   width: 100px;
10  height: 100px;
11  right: 5px; // 5px relative to the most-right of parent
12 }
```

4.5.5 Fixed

Like `position: absolute`, fixed positioned elements are also removed from the normal document flow. They are only relative to the `<html>` document, not any other parents. They are not affected by scrolling.

```
1 .container {
2   position: relative;
3   background: lightgray;
4 }
5
6 .box-green{
7   position: fixed;
8   background: green;
9   width: 100px;
10  height: 100px;
11  right: 5px; // 5px relative to the most-right of parent
12 }
```

Here in the example, you'll change the green box's position to fixed, and this time it is relative 5px to the right of the `<html>`, **not its parent** (container).

As you'd see if you run the example, scrolling the page doesn't affect the `fixed` positioned box. It is not relative to its parent (container) anymore.

4.5.6 Sticky

The `position: sticky` can be explained as a mix of `position: relative` and `position: fixed`.

It behaves until a declared point like position: **relative**, after that it changes its behavior to position: **fixed**.

[Warning] Position Sticky is not supported in Internet Explorer and earlier versions of other browsers. However, you can check the latest browser support details at ‘caniuse.com’.

The best way to understand the CSS position property is by practice. Keep coding until you have a better understanding. The majority of the examples in these notes, you’ll use position: **absolute**, however, as you grow and innovate you might use some of the other position properties for optimization/interactive solutions in your games/web-applications.

4.6 Transforms

Once you start to get objects on the screen, you’ll want to **transform** them - which means, position (*translate*), scale (*size*), rotate and more.

While we go through the low-level concepts scripting concepts here, there are a variety of free and easily accessible tools available online for experimenting and testing out various transforms/animations. For example:

<https://animista.net/> - Tool for creating CSS animation code to create effects (fade-in/out, attention, rotation, ...) <https://webdesigner.withgoogle.com/> - A tool for visually editing/creating web designs (advanced features, like time-line/animation)

4.7 Position (Translation) and Size

You set the *position* and *size* of elements using the CSS style options (left, top, width and height). A few things to be aware of, you need to specify the units (**px**) - if you don’t include the units (even in Javascript) you’ll have issues.

```
1 #mydiv {
2   position: absolute; /* position isn't relative */
3   left: 20px; /* position wrt the left of the element */
4   top : 100px; /* position wrt the top of the element */
5   width: 200px; /* width of element */
6   height: 200px; /* height of element */
7 }
```

If you forget to append the units for the left or top position when setting the style in Javascript - nothing will happen. While the width/height will change if you don’t include the units (also note the position type - *absolute*)

4.8 Rotation

You'll obviously at times want to *rotate* your game objects - either for dramatic effect (spinning coin) or for as part of the mechanic (face the *direction* you're going to shoot your bullets).

The CSS function is called `rotate()` - and defines the transformation that rotates your element around a fixed point on the 2D plane, without deforming it. Other transformation functions in addition to rotation, include scaling (zoom), skew, or moving an element.

```
1 #mydiv {  
2   transform: rotate(45deg); /* Equal to 45degrees */  
3 }
```

4.8.1 Rotation around (predefined point)

Initially, you might think the rotation would be around the centre of the object, however, what if you wanted to rotate the object around the corner? To address this, there is an additional CSS function for defining the rotation *origin* (i.e., point around which your element rotates).

```
1   transform-origin: -100% 50%;
```

For example, to rotate 20 degrees around the centre of your div element (with ID of `mydiv`):

```
1 #mydiv {  
2   transform: rotate(20deg); /* Equal to 45degrees */  
3   transform-origin: 50% 50%; /* Centre of the element */  
4 }
```

4.8.2 Direction and Rotation

You'll see later in Javascript you control the rotation (and direction) of an element programatically in Javascript - for instance, you might want the player to always face the direction they're moving or shooting.

4.9 Depth (Z-Index)

Often you'll want to overlap elements on top of one another. The question is then which should be drawn on top and which on the bottom? For example, the background scene image should be drawn at the back, while all the interactive player objects in front. To set this, you specify the CSS `z-index` value. Lower values are at the bottom and higher values at the top.

- Background image `z-index = -10`
- Clouds `z-index = 5`
- Player `z-index = 100`
- Menu `z-index = 500`

4.10 Sprite Sheets

4.10.1 Showing Part of an Image

At times, you'll have a single image with lots of sub-graphical elements (trees, bridges and walls all baked into a single graphics file). So you can then reference and show the different parts of the image when you need them.

```
1 .tree {  
2     position: absolute; /* Absolute position on the screen */  
3     width: 50px; /* Element width in the sprite sheet */  
4     height: 50px; /* Element height in the sprite sheet */  
5     display: inline-block; /* Display icon as inline block */  
6     background-position: -200px -301px; /* Background position in sprite */  
7     left: 10px; /* Display position on the screen */  
8     top: 10px; /* Display position on the screen */  
9 }
```

4.10.2 Animating the Sprite Sheet

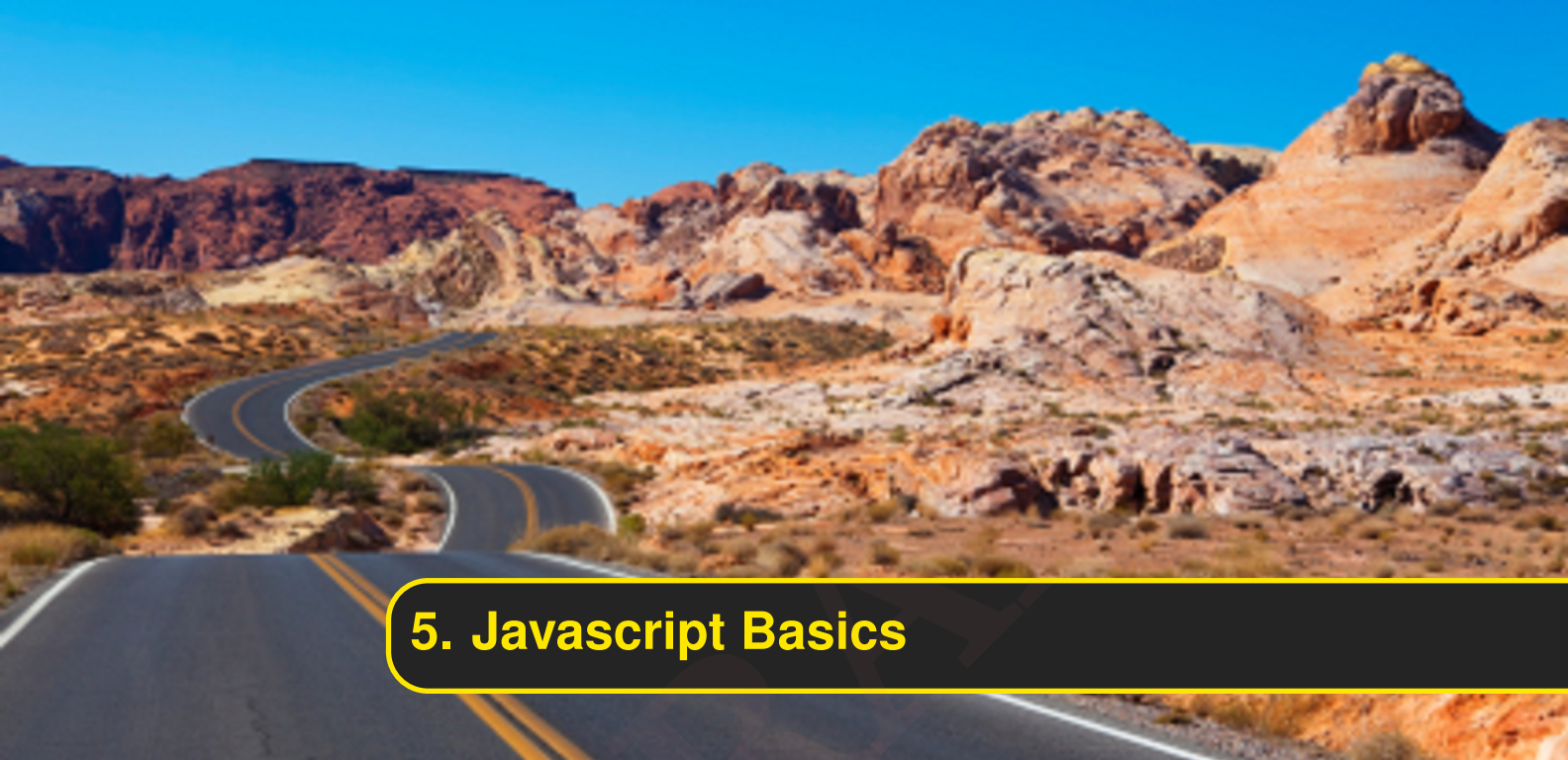
Combining the concept for selecting part of a region within an image for displaying with the CSS animation, you're able to create exciting sprite animations. In the example below, image is a long sequence of coin images (each 75x75px). Every 2 seconds, the `x` offset is incremented to create the illusion that the coin is rotating (key-frame animating the images). In the CSS we specify the start and end offset, then the CSS animation will *tween* the inbetween values for the offset positions.

```
1 .spinningCoin {  
2     width: 75px;  
3     height: 75px;  
4     background-image: url("./coin.png");  
5     animation: playX 2s steps(8) infinite;  
6 }  
7 @keyframes playX {  
8     from {background-position-x: 0px;}  
9     to {background-position-x: -600px;}  
10 }
```

4.11 CSS or Javascript

The CSS animation details can also be controlled in Javascript, as you'll see later. Just allows you to design and develop visual features which can easily be plugged into your game (i.e., you can change the CSS file for a completely different looking game, without having to modify the Javascript code which controls the game mechanics).

Each of the transforms defined in CSS can be done programatically in Javascript (i.e., rotate, translate, scale, ...). As your page contents will be changing on the fly (objects appearing and disappearing) depending on the users input/actions - requiring you to generate and manipulate CSS/HTML in Javascript.



5. Javascript Basics

5.1 Creating CSS elements on the fly (CreateElement)

On occasion you will want to create elements (e.g., divs) on the fly using purely Javascript. For example, you might have particles, weapons or streaming enemies - or maybe you want to allow the user to customize/edit the elements/design.

You access the CSS style information in Javascript for the element using the `style` properties. e.g., `element.style.id`. Be aware that some style properties are read only, also the names used in CSS and as accessors in Javascript might be slightly different, for instance, CSS: `background-color=blue`, while in Javascript: `element.style.backgroundColor="blue";`.

A simple example of how easy it is to add an element to a scene programmatically, is given below:

```
1  var tank = document.createElement("div");
2  tank.id = "track"; // set id for the created element
3  tank.style.backgroundPosition = "-589px -80px"; // offset region in image to draw
4  tank.style.position = "absolute"; // position is absolute (vs relative)
5  // positions can be calculated and updated on the fly (instead of hardcoded)
6  tank.style.top = (pos.y - 8) + 'px';
7  tank.style.left = (pos.x - 1) + "px";
8  tank.style.width = "82"; // size of the image
9  tank.style.height = "102"; // size of the image
10 tank.style.margin = '0px'; // don't want any margin
11 tank.style.zIndex = -12; // z-order (handle overlapping elements)
12 //tank.style.zoom = "70%"; // also option for zooming (instead of scaling)
13 // Concatenate transforms, rotation followed by a scale
```

```

14     var ang = 20;
15     tank.style.transform = "rotate(" + ang + "deg) scale( 0.7, 0.7 )";
16     tank.style.transformOrigin = "200 200"; // rotation/transform origin
17     tank.style.backgroundImage = "url('./tank.png')"; // image
18     document.body.appendChild(tank); // add element to the browser scene

```

You create the type of element using the `createElement` function. Then, you set the parameters, such as, the ID, `className` or any style (CSS) values. You must always remember to add your element to the scene - if the element isn't added to the scene then it won't be rendered or updated.

When setting the position of a element programatically in Javascript (i.e., `style.left` or `style.top` property) remember to include the units (e.g., `px`) otherwise it won't work (it won't give any error warning to why). For example: `tank.style.left=200;` is wrong and should be `tank.style.left=200px;`

5.2 Generate HTML for the page (document.write)

While you can append and remove elements from a web page dynamically in Javascript, you can also write the HTML for the web page directly using the `document.write` function.

5.3 Setting/Getting CSS Style information in Javascript

In Javascript you can access/modify the *style* information for the element through the `style` accessor. For example:

```

1 // Get the element object
2 var tank = document.getElementById("tank");
3 // Set the CSS background style for the object
4 tank.style.backgroundColor = "blue";

```

5.4 Variables let, var (or const)

Javascript variables are containers that holding pieces of data. There are **three keywords** used when declaring variable in Javascript, namely `var`, `let` and `const`. They generally follow this pattern or syntax:

```

1 var varName1 = varValue1;
2 let varName2 = varValue2;
3 const varName3 = varValue3;

```

In days gone by, **var** was the only way to declare a variable - that was before ES6. **ES6** stands for ECMAScript 6 and was created to standardize Javascript (**ES6** is the 6th version of ECMAScript - Javascript standard). This was to introduce more consistency and robustness into the language.

You can declare the **same variables** more than one time and can be updated.

```
1  var myFriend = 'Tom';
2  var myFriend = 'Peter'; // we're declaring the variable again
3  console.log(myFriend2); // 'Peter'
```

If you declare variable inside a block statement, the variable will leak outside.

This was the only way to declare variable before ES6. For example, you can declare the same variables more than once, each time, the variable is updated (no error or warning by default).

```
1  var myFriend = 'Tom';
2  var myFriend = 'Peter';
3  var myFriend = 'Alex';
4  console.log(myFriend); // 'Alex'
```

In most case **let** and **const** are almost same, the main difference is **const** cannot be updated but **let** can.

```
1  // let can be updated
2  let myFriend = 'Tom';
3  myFriend = 'Peter';
4  console.log(myFriend); // Peter
5
6  // const cannot be updated
7  const otherFriend = 'Alex';
8  otherFriend = 'Steven'; // Uncaught TypeError: Assignment to constant variable.
```

The variable is not leaked outside of the block statement if you use **let** or **const**. This is useful if you're concerned with scope (only want the variable to be used within a block statement).

```
1  const bodyWeight = 50;
2  if (bodyWeight > 49) {
3    const water = 1.4;
4    console.log(`A body weight of ${bodyWeight}kg, means you should drink ${water}litres
    ↳ of water`);
5  }
6  console.log(water); // Uncaught ReferenceError: water is not defined
```

When to use **var**, **let** and **const**

Always use **const** when declaring variable, use **let** or **var** when you want to update the variable (note your browser needs to support ES6 and above for **const** and **let**). Apparently **let** is the new **var**, the **only** difference is that **var** gets scoped to the current function, while **let** gets scoped to the current block.

One thing to always be aware of when using `var` is the scope monster - where you might inadvertently be changing (or saving) values globally.

Often good practice to encapsulate your code inside anonymous function and execute the function immediately:

```
1 (function(){
2     // var is limited to function scope
3     var abc = "test";
4 }());
5 console.log( abc ); // will give an error as abc is out of scope
```

Syntax (Aesthetics) Various ways to format and layout your variables and code, for example because there are no type defines (done automatically) you can declare your variables in a single line using commas to separate them:

```
1 var
2     g = .15,
3     windResistance = .0001,
4     angle = 0,
5     rollingAdditionalAngle = 0,
6     floorFriction = .1,
7     spritePos = 0;
```

or a `var` for each variable and line:

```
1 var g = .15;
2 var windResistance = .0001;
3 var angle = 0;
4 var rollingAdditionalAngle = 0;
5 var floorFriction = .1;
6 var spritePos = 0;
```

They both work the same - the most important point is which is the most readable and easy to maintain.

JSLint Can Help *JSLint* is a free online tool which can help you avoid common errors and get into good practices.

<http://www.jshint.com/>

Warning! JSLint may hurt your feelings. If you write sloppy code you will have to do a lot of fixing to make JSLint happy. Of course, if you JSLint it often you will develop robust and clear coding style discipline and it will become your friend that helps you spot minor mistakes.

5.5 Types of Variables

There are a specific **Types** of **primitive** for **data**, checked by `typeof` operator (as discussed in the next section). These include

1. `boolean`
2. `number`
3. `string`
4. `bigint`
5. `symbol` – available only from ES6

There are two special primitive types which cannot contain values

1. **`undefined`** sole value possessed by unassigned variables
2. **`null`** value denoting intentional lack of a value

There are other **Non Data** structures, which include:

1. **`object`**
2. **`function`**

Variables names (identifiers) are **case-sensitive** and are limited by the following constraints:

- start with alphabetic letter, underscore or dollar sign
- contain only alphanumerics, dollar signs and underscore
- have no type - type belongs to their value

5.5.1 `null`

The `null` type is the second primitive data type that has only one value: `null`. Javascript defines that `null` is an empty object pointer.

5.5.2 `undefined`

The `undefined` type is a primitive type that has one special value `undefined`. By default, when a variable is declared but not initialized, it is assigned the value of `undefined`.

5.5.3 `boolean`

The `boolean` type has two values: `true` and `false`, in lowercase. The following example declares two variables that hold boolean values.

5.5.4 `number`

JavaScript uses the IEEE-754 format to represent both integer and floating-point numbers.

NaN JavaScript has a special numeric value called `NaN`, which stands for **Not a Number**. In fact, it means an invalid number.

5.5.5 string

In JavaScript, a string is a sequence of zero or more characters. A literal string begins and ends with either single quote (') or double quotes ("). A string that starts with a double quote must end with a double quote and a string that begins with a single quote must end with a single quote.

5.5.6 symbol

JavaScript added a primitive type in ES6: the `symbol`. Different from other primitive types, the `symbol` type does not have a literal form.

5.5.7 objects

Javascript **Objects** are members of the type `Object` that have * prototype object * extensible enumerable list of proper ties (including functions) * include keyed, indexed collection, function types

5.6 typeof

Anytime you're unsure what type of variable/data you have on your hands, you can use the `typeof` operator. The `typeof` operator returns a string identifying the type (e.g., `number`, `string`, ...).

Type	typeof Result
Undefined	"undefined"
Null	"object"
Boolean	"boolean"
Number	"number"
BigInt (new in ECMAScript 2020)	"bigint"
String	"string"
Symbol (new in ECMAScript 2015)	"symbol"
Function object	"function"
Any other object	"object"

Some examples:

```

1 // Numbers
2 typeof 37 === 'number';
3 typeof 3.14 === 'number';
4 typeof(42) === 'number';
5 typeof Math.LN2 === 'number';
6 typeof Infinity === 'number';
7 typeof NaN === 'number'; // Despite being "Not-A-Number"
8 typeof Number('1') === 'number'; // Number tries to parse things into numbers
9 typeof Number('shoe') === 'number'; // including values that cannot be type coerced to
    ↪ a number
10
11
12 // Strings
13 typeof '' === 'string';
14 typeof 'bla' === 'string';
15 typeof `template literal` === 'string';
16 typeof '1' === 'string'; // note that a number within a string is still typeof string
17 typeof (typeof 1) === 'string'; // typeof always returns a string
18 typeof String(1) === 'string'; // String converts anything into a string, safer than
    ↪ toString
19
20
21 // Booleans
22 typeof true === 'boolean';
23 typeof false === 'boolean';
24 typeof Boolean(1) === 'boolean'; // Boolean() will convert values based on if they're
    ↪ truthy or falsy
25 typeof !(1) === 'boolean'; // two calls of the ! (logical NOT) operator are equivalent
    ↪ to Boolean()

```

5.7 Mutable or Immutable Data Types

A *mutable* object is an object whose state can be modified after it is created. An *immutable* object is an object whose state cannot be modified after it is created.

Some examples of native JavaScript values that are **immutable** are numbers and strings. Examples of native JavaScript values that are **mutable** include objects, arrays, functions and classes.

Mutable: - object - array - function

Immutable: All primitives are immutable.

- string
- number
- boolean
- null
- undefined
- symbol

5.7.1 Copy Array (or Object)

There are occasions when you'll want to duplicate or copy an object (or array) and not refer to the original. Simply assigning an object or an array using the assignment operator (=) will only get you a reference to the original.

When you need to copy an array, you can use the `slice` method. However, with ES6, you can also use the spread operator to duplicate an array. See below:

```
1  const letters = ['a', 'b', 'c'];
2
3  // Before ES6 way
4  const cloneLetters = letters.slice();
5
6  // ES6 way
7  const cloneLettersES6 = [...letters];
8  // ... is the syntax for the spread operator
9
10 // or another ES6 way
11 const cloneLettersES6 = Array.from(letters);
```

5.7.2 Pass-by-Value or Pass-by-Reference

Pass by Value In Pass by Value, Function is called by directly passing the value of the variable as the argument. Changing the argument inside the function doesn't affect the variable passed from outside the function. Javascript always pass by value so changing the value of the variable never changes the underlying primitive (String or number).

However, when a variable refers to an object which includes array, the value is the reference to the object.

Pass by Reference In Pass by Reference, Function is called by directly passing the reference/address of the variable as the argument. Changing the argument inside the function affect the variable passed from outside the function. **In Javascript objects and arrays follows pass by reference.** So if you are passing objects or arrays as arguments to your method, then there is a possibility that value of the object can change.

5.8 Conditional logic

Conditional logic includes, if, else and switch statements. Without conditional logic, your code would do the same thing every time. Depending on values (e.g., timer, mouse press, some random number), the code will execute different ways.

If (and *else*) statement syntax:

```
1  var val = 2;
2  if ( val > 2 )
```

```
3 {  
4   console.log('val is greater than 2');  
5 }  
6 else  
7 {  
8   console.log('val is less than or equal to 2');  
9 }
```

Or sometimes the more compact and readable *switch statement* syntax:

```
1 var val = 2;  
2 switch ( val )  
3 {  
4   case 0:  
5     console.log('val is 0');  
6     break;  
7   case 1:  
8     console.log('val is 1');  
9     break;  
10  case 2:  
11    console.log('val is 2');  
12    break;  
13  default:  
14    console.log('val is unknown / other');  
15 }
```

Be aware of type coercing in Javascript, as this could be a problem with a `switch` statement.

For example:

```
1 let a = "2";  
2  
3 // this line will print 'a==2' as a will be coerced  
4 // into a number  
5 if ( a==2 ) { console.log(" a == 2 " ); }  
6  
7 // the switch statement will fail, as `a` is a  
8 // string not a number  
9 switch ( a )  
10 {  
11   case 1:  
12     console.log(" a == 1 " );  
13     break;  
14   case 2: // fails as `a` is a string  
15     console.log(" a == 2 " );  
16     break;  
17 }
```

Tip: Add an additional `typeof` check to the start of your switch statement to ensure the value types getting checked match (e.g., numbers, strings).

5.9 Loops

As you'll guess, some code needs to be executed repeatedly. The three main types of loop are:

- *for loop*
- *while loop*
- *do while loop*

The syntax for the three loops are as follows (each printing the value from 0 to 9 to the console log window):

5.9.1 For loop

```
1 for (var i=0; i<10; ++i)
2 {
3     console.log( i ); // print i out 10 times
4 }
```

5.9.2 While loop

```
1 var i = 0;
2 while (i<10)
3 {
4     console.log( i ); // print i out 10 times
5     i++;
6 }
```

5.9.3 Do while loop

```
1 var i = 0;
2 do
3 {
4     console.log( i ); // print i out 10 times
5     i++;
6 } while (i<10)
```

5.10 Functions

You don't want to keep rewriting the same code again and again. Instead you'll write the code and keep it inside a *function*. When you need to execute that code, you call the function.

```
1 function someFunction()
2 {
3     // function code
4 }
5
6 // Call the function
7 someFunction();
```

5.10.1 Modularize - functions to contain task

Modularization is a general programming good practice. You should make sure that you create functions that fulfill one job at a time makes it easy for other developers to debug and change your code without having to scan through all the code to work out what code block performs what function.

This also applies to creating helper functions for common tasks. If you find yourself doing the same thing in several different functions then it is a good idea to create a more generic helper function instead, and reuse that functionality where it is needed.

Also, one way in and one way out makes more sense than forking the code in the function itself. Say you wanted to write a helper function to create new links. You could do it like this:

```
1 function addLink(text,url,parentElement){
2   var newLink = document.createElement('a');
3   newLink.setAttribute('href',url);
4   newLink.appendChild(document.createTextNode(text));
5   parentElement.appendChild(newLink);
6 }
```

5.11 Classes and Objects

Grouping together **code** and **data** (*encapsulation*). Makes your code more flexible and reliable. Encapsulating your code and data is **Not just a syntactic sugar**. Sometimes people say that a *class* is just *syntactic sugar* (syntax that is designed to make things easier to read, but doesn't introduce anything new), because we could actually declare the same without the `class` keyword at all. However, as your game projects start to get larger and more complex, classes make it easier for you to modularize and plugin/out components. As you'll see later on - it's easy for you to swap out one menu system for another as long as the class has the same publically accessible methods (i.e., methods and data available outside the *class*).

5.11.1 Four ways to define a Javascript object (class)

Javascript is a very flexible object-oriented language when it comes to syntax. In this article you can find three ways of defining and instantiating an object. Even if you have already picked your favourite way of doing it, it helps to know some alternatives in order to read other people's code.

It's important to note that there are no classes in Javascript. Functions can be used to somewhat simulate classes, but in general Javascript is a class-less language. Everything is an object. When it comes to inheritance, objects inherit from objects, not classes from classes as in the "class"-ical languages.

5.11.2 1. Function definition

This is probably one of the most common ways. You define a normal Javascript function and then create an object by using the `new` keyword. To define properties and methods for an object created using `function()`, you use the `this` keyword, as seen in the following example.

```
1 function Apple (type) { // constructor function
2   this.type = type;
3   this.color = "green";
4   this.getInfo = getAppleInfo;
5 }
```

To instantiate an object using the Apple **constructor** function, set some properties and call methods you can do the following:

```
1 var apple = new Apple('sweet');
2 apple.color = "reddish";
```

Methods defined internally To add a method to class Apple above, such as, `getInfo()`, you'd defined a separate method called `getInfo()`. To connect this method with your class, you define your methods within the constructor function, like this:

```
1 function Apple (type) {
2   this.type = type;
3   this.color = "green";
4
5   // Method for the class Apple
6   this.getInfo = function() {
7     return this.color + ' ' + this.type + ' apple';
8   };
9 }
```

Using this syntax changes nothing in the way you instantiate the object and use its properties and methods.

Methods added to the prototype A drawback of the previous approach, is that the method `getInfo()` is recreated every time you create a new object. Sometimes that may be what you want, but it's rare. A more inexpensive way is to add `getInfo()` to the *prototype* of the constructor function.

```
1 function Apple (type) {
2   this.type = type;
3   this.color = "red";
4 }
5
6 Apple.prototype.getInfo = function() {
7   return this.color + ' ' + this.type + ' apple';
8 };
```

You'd access the object in the same way as before.

5.11.3 2. Object literals

Literals are shorter way to define objects and arrays in Javascript. To create an empty object using you can do:

```
var o = {};
```

instead of the “normal” way:

```
var o =new Object();
```

For arrays you can do:

```
var a = [];
```

instead of:

```
var a =new Array();
```

So you can skip the class-like stuff and create an instance (object) immediately. Here’s the same functionality as described in the previous examples, but using object literal syntax this time:

```
1 var apple = {  
2   type: "sweet",  
3   color: "red",  
4   getInfo: function () {  
5     return this.color + ' ' + this.type + ' apple';  
6   }  
7 }
```

In this case you don’t need to (and cannot) create an instance of the class, it already exists. So you simply start using this instance.

```
1 apple.color = "reddish";  
2 alert(apple.getInfo());
```

Such an object is also sometimes called *singleton*. In “classical” languages such as Java, **singleton** means that you can have only one single instance of this class at any time, you cannot create more objects of the same class. In Javascript (no classes, remember?) this concept makes no sense anymore since all objects are singletons to begin with.

5.11.4 3. Singleton using a function

The third way presented in this section is a combination of the other two you already saw. You can use a function to define a singleton object. Here’s the syntax:

```
1 var apple = new function() {  
2   this.type = "sweet";  
3   this.color = "red";  
4   this.getInfo = function () {  
5     return this.color + ' ' + this.type + ' apple';  
6   }  
7 }
```

So you see that this is very similar to the example above, but the way to use the object is still the same.

```
1 apple.color = "pink";
2 alert(apple.getInfo());
```

`new function(){...}` does two things at the same time: define a function (an anonymous constructor function) and invoke it with `new`. It might look a bit confusing if you're not used to it and it's not too common, but it's an option, when you really want a constructor function that you'll use only once and there's no sense of giving it a name. While these three main ways are the most popular ways for creating objects in Javascript, there is also the *newer* approach (which is technically more correctly object orientated).

5.11.5 4. Classes (class keyword)

As we've shown, you can declare and manage your objects in Javascript using functions. The latest Javascript supports the keyword **class** for declaring and creating **proper** objects. In recent years, objects orientated concepts have become more important. Hence there was a push to have Javascript embrace object orientated principles (constructors and the class keyword). The later versions of Javascript enable you to create **classes** using the `class` keyword. It's no longer correct to say that all classes are functions in Javascript. There are now classes (using an actual `class`). However, be aware, that this new approach is not as well supported by browser implementations (yet) but in this fast changing age - the `class` way of creating classes will continue to gain momentum and become a more core concept of the language. For completeness, the syntax for creating classes/objects using the `class` keyword/syntax is shown below:

```
1 class Cat {
2   constructor(name) { // supports constructor keyword
3     this.name = name;
4   }
5
6   speak() {
7     console.log(this.name + ' makes a noise.');
```

5.12 use strict;

Adding `"use strict";` to the top of your Javascript files lets you catch certain errors (possible bugs). *Strict mode* makes it easier for you write *secure* Javascript.

Strict mode discourages previously accepted *bad syntax* issues into errors. As an example, in normal Javascript, if you mistyped a variable name, it would create a new global variable. In *strict mode*, this would throw an error, preventing you accidentally creating a global variable.

In normal Javascript, a developer will not receive any error feedback assigning values to non-writable properties. In strict mode, any assignment to a non-writable property, a getter-only property, a non-existing property, a non-existing variable, or a non-existing object, will throw an error.

```
1  "use strict";
2  var val = 2.3;
3
4  vaL = 3; // this would give a error in strict mode
5  // as val not the same as vaL (case sensitive)
6
7  alert( val ); // you might expect this line to display 3
8  // however, without strict there would be no warnings
9  // and could result in you spending many hours trying to
10 // debug the reason why (i.e., simple typing error)
```

5.13 debug assert (console)

A useful sanity check in most programming languages, including Javascript, is the *assert* check. The assert check provides a safety net that is triggered in your code as soon as your check fails. For example, if a calculation returns an undefined number, like infinity, then you can add an assert check.

```
1  var num = 2;
2  // assert check - number must always be greater than 0
3  console.assert( num > 0 );
4
5  num = -2
6  // assert check - this line will trigger an assert warning
7  // as the assert check would *fail*
8  console.assert( num > 0 );
```

5.13.1 Console.log isn't available?

Be aware that code may brake if *console* is not present (e.g., it may be turned off or not supported by a specific browser). To avoid errors replace it with fake console object and methods, when console is not present:

```
1  var console = window.console || {};
2  console.log = console.log || function(){};
3  console.warn = console.warn || function(){};
4  console.error = console.error || function(){};
5  console.info = console.info || function(){};
```

5.14 Visually Debugging

Due to the interactive nature of a game, sometimes it might not be possible (or difficult) to debug or test a feature algorithmically. Instead, you might need to add some additional *debug drawing* code in to show specific information (e.g., collision box for a shape, contact or hit point, velocity or timer details).

5.15 Creating Elements

You can directly define objects in the HTML and reference them using their ID or Class name. However, you can also create objects on the fly and add them to the scene using the *createElement* method. The following code snippet will create a *div* object:

```
1 var d = document.createElement('div')
2 d.id = 'mydiv';
3 d.className = 'myClassName';
4 d.style.position = 'absolute';
5 document.body.appendChild( d );
```

Remember, it is important to add your created object to the Document Object Model hierarchy using *appendChild* otherwise you won't be able to see it.

5.16 Debug Draw

Debug draw borders for all your div elements on the screen (e.g., collision detection or show transparent hidden objects).

```
1 /*
2 Debug option - comment in to see all the div dimensions/details
3 */
4 * {
5 border-style: dotted !important;
6 }
```

5.17 Debug Output

The main ways for passing information for debug include:

1. `alert(...)` - stop the program to display information
2. `console.log(...)` - write information to the console window log
3. write information to the window direction (e.g., custom debug div element, `div.innerHTML ="";`)

4. also the option to send debugging data asynchronously to an active server in the background (e.g. using AJAX calls)

5.18 Drawing Lines without Canvas

There might be an occasion when you want to draw line information, for example, you have a maze and you want to draw edges or a cross for the cursor location.

```

1 function createLine(ax,ay, bx,by)
2 {
3     if(ay>by)
4     {
5         bx=ax+bx;
6         ax=bx-ax;
7         bx=bx-ax;
8         by=ay+by;
9         ay=by-ay;
10        by=by-ay;
11    }
12    // Detailed to include the browser syntax variations (e.g.,
13    // transform and webkit-transform, ...)
14
15    let calc = Math.atan((ax-bx)/(by-ay));
16    calc = calc*180.0/Math.PI;
17    let length = Math.sqrt((ax-bx)*(ax-bx)+(ay-by)*(ay-by));
18
19    let div = document.createElement('div');
20    div.style.height = length + "px";
21    div.style.width = "3px";
22    div.style.left = ax + "px";
23    div.style.top = ay + "px";
24    div.style.transform = "rotate(" + calc + "deg)";
25    div.style.transformOrigin = "0% 0%";
26    div.style.backgroundColor = "blue";
27    div.style.position = "absolute";
28    document.body.appendChild( div );
29    return div;
30 }
```

A simple test function which draws a line on screen (blue line 3 pixels thick), and follows the mouse cursor around the screen. This simple test will allow you to check that your code works as expected. Always test your code!

```

1 <script>
2 // 1 create our div line, anchor one point
3 // somewhere on the screen and the other end will
4 // follow the mouse around
5 let startPos = {x:300, y:500};
6 let endPos = {x:100,y:-200};
7 let linediv = createLine(startPos.x, startPos.y,
8                           endPos.x, endPos.y);
9
10 function updateLine(div, ax,ay, bx,by)
11 {
12     if(ay>by)
13     {
14         bx=ax+bx;
15         ax=bx-ax;
16         bx=bx-ax;
```

```

17     by=ay+by;
18     ay=by-ay;
19     by=by-ay;
20 }
21 let calc = Math.atan((ax-bx)/(by-ay));
22 calc = calc*180.0/Math.PI;
23 let length = Math.sqrt((ax-bx)*(ax-bx)+(ay-by)*(ay-by));
24 div.style.height = length + "px";
25 div.style.left = ax + "px";
26 div.style.top = ay + "px";
27 div.style.transform = "rotate(" + calc + "deg)";
28 }
29
30 document.onmousemove = function(e){
31     updateLine( linediv,
32                 startPos.x, startPos.y,
33                 endPos.x, endPos.y);
34     endPos = {x:e.pageX, y:e.pageY};
35 }
36 </script>

```

The following helper function, also provides a version of the write line code, but instead of adding elements directly to the Document Model Hierarchy (i.e., using `document.createElement(..)`), the line is created using the `document.write(..)` function.

```

1 // Small helper function, so if you need to draw lines once using
2 // the document.write(..). You can call this function with the
3 // coordinates for the start and end of your line
4 function writeLine(ax,ay,bx,by)
5 {
6     if(ay>by)
7     {
8         bx=ax+bx;
9         ax=bx-ax;
10        bx=bx-ax;
11        by=ay+by;
12        ay=by-ay;
13        by=by-ay;
14    }
15    // Details include the browser syntax variations (e.g.,
16    // transform and webkit-transform, ...)
17
18    let calc=Math.atan((ax-bx)/(by-ay));
19    calc=calc*180.0/Math.PI;
20    let length=Math.sqrt((ax-bx)*(ax-bx)+(ay-by)*(ay-by));
21    let sss = "<div style='height:" + length +
22        ↪ "px;width:1px;background-color:pink;position:absolute;top:" + (ay) +
23        ↪ "px;left:" + (ax) + "px;transform:rotate(" + calc +
24        ↪ "deg);-ms-transform:rotate(" + calc + "deg);transform-origin:0%
25        ↪ 0%;-moz-transform:rotate(" + calc + "deg);-moz-transform-origin:0%
26        ↪ 0%;-webkit-transform:rotate(" + calc + "deg);-webkit-transform-origin:0%
27        ↪ 0%;-o-transform:rotate(" + calc + "deg);-o-transform-origin:0% 0%;></div>"
28    document.write(sss);
29 }

```

5.19 User input

The two main ways for getting user input from your browser, is via the keyboard or mouse.

Newer mobile devices this might include navigation or gyroscope information.

5.19.1 Mouse input

```

1  var cursorX;
2  var cursorY;
3  document.onmousemove = function(e){
4      cursorX = e.pageX;
5      cursorY = e.pageY;
6  }

```

5.19.2 Keyboard Input

Ascii, Unicode and Key Code Values When you start to work with the keyboard, you need to be aware of the difference between character and key code values. The difference between the two code types:

- Character codes - A number which represents an *ASCII* character
- Key codes - A number which represents an actual key on the *keyboard*

A simple example for you to type, and see the value for each of the keys on your keyboard:

```

1  <p>onkeypress - <span id="keya"></span></p>
2  <p>onkeydown - <span id="keyb"></span></p>
3
4  <input type="text"
5      onkeypress="uniCharCode(event)"
6      onkeydown="uniKeyCode(event)">
7
8  <script>
9  function uniCharCode(event) {
10     var char = event.which || event.keyCode;
11     document.getElementById("demo").innerHTML = "Unicode: " + char;
12 }
13
14 function uniKeyCode(event) {
15     var key = event.keyCode;
16     document.getElementById("demo2").innerHTML = "KEY code: " + key;
17 }
18 </script>

```

5.19.3 Reacting to Key Events

Once you're comfortable with which keys do what, you'll want to capture keyboard **events** - so your game game respond appropriately to input from the user (firing bullets or steering animated character on screen).

```

1  function checkKey(e) {
2      e = e || window.event;
3

```

```
4   if (e.keyCode == '38') { dir.y = -1; } // up arrow
5   else if (e.keyCode == '40') { dir.y = 1; } // down arrow
6   else if (e.keyCode == '37') { dir.x = -1; } // left arrow
7   else if (e.keyCode == '39') { dir.x = 1; } // right arrow
8 }
9 document.onkeydown = checkKey;
```

5.20 Timer Events (Timing Information)

You can define callback timer events in Javascript using different methods. For example, `requestAnimationFrame`, `setInterval` (or `setTimeout`). You can even measure the time taken for a particular piece of code to have run using the console timing function:

```
1 console.time('refA');
2
3 myFunction(); // Timing between the two "console.time"
4
5 console.timeEnd('refA');
```

The string being pass to the `time()` and `timeEnd()` methods must match (for the timer to finish as expected).

5.21 Events and Elements

You can connect events to specific elements (e.g., *button clicks* and *key presses*) which are only **called** when the element has the focus. For example, making a specific `div` element respond to cursor clicks:

```
1 var something = document.getElementById('something');
2
3 something.style.cursor = 'pointer';
4 something.onclick = function() {
5     // do something...
6 };
```

Changing the background color of the element as the *mouse moves* over the `div` element:

```
1 something.onmouseover = function() {
2     this.style.backgroundColor = 'red';
3 };
4 something.onmouseout = function() {
5     this.style.backgroundColor = '';
6 };
```

5.22 Game Loop

The `requestAnimationFrame()` to write that main loop

```
1 function mainLoop() {  
2   update();  
3   draw();  
4   requestAnimationFrame(mainLoop);  
5 }  
6  
7 // Start things off  
8 requestAnimationFrame(mainLoop);
```

As you can see, `requestAnimationFrame()` is actually very simple in concept and execution, once you understand its purpose and common patterns. You'll utilize `requestAnimationFrame()` internally for all your dynamic operations.

5.22.1 `requestAnimationFrame` vs `setInterval` (or `setTimeout`)

Typically, the reason to use `requestAnimationFrame` over `setInterval` is down to animation quality. Typically, `requestAnimationFrame` produces higher quality (smoother) animation by eliminating flicker and shear that can happen when using `setTimeout` or `setInterval`, and reduce or completely remove frame skips.

However, it's important to note, that neither `requestAnimationFrame` or `setInterval` provide guaranteed exact timing calls, the call times can deviate quite a lot. A solution is to make use of the **timestamp** (which you'll learn more about later when we dig into the principles of physics). The timestamp is provided as the first argument of the `requestAnimationFrame` callback and can be used for the kinematic / physics based calculations (i.e., difference between the time of the last and the current callback execution).

5.22.2 Document Object Model (Ready?)

You can add an event listener to inform you (call your function) when the browser has finished *loading* (e.g., loading and parsing any content).

```
1 // The window load event listener  
2 window.addEventListener("load", docReady);
```

5.22.3 Timing Problems

Thrashing your browser, not to mention, all of the other applications running in the background, you can't expect the time frame to remain consistent?

```

1  var lastFrameTimeMs = 0, // The last time the loop was run
2      maxFPS = 10; // The maximum FPS we want to allow
3
4  function mainLoop(timestamp) {
5      // Throttle the frame rate.
6      if (timestamp < lastFrameTimeMs + (1000 / maxFPS)) {
7          requestAnimationFrame(mainLoop);
8          return;
9      }
10     lastFrameTimeMs = timestamp;
11
12     // ...
13 }

```

5.23 Rotating Elements to Face a Particular Direction

You learned about the CSS transform feature in in the CSS and HTML section. You can expand on this concept using vector maths to modify the rotation on the fly in Javascript to rotate the element to face a particular direction (such as a tank turret *facing the direction* it would shoot).

```

1  /*
2  Takes a div element, a direction vector you want the element
3  to face towards (e.g., dir = {x:0, y:1}). This function requires
4  the direction vector be normalized (also the vector can't be null or zero length)
5  */
6  function RotateDirection(div, dir)
7  {
8      var up = {x:0, y:1}; // reference direction
9      var dot = up.x*dir.x + up.y*dir.y; // dot product
10     var ang = Math.acos( dot ); // angle
11     var cross = up.x*dir.y - up.y*dir.x; // cross product
12     if ( cross < 0 ){ ang *= -1.0; } // rotate left or right
13
14     // apply the rotation to the element style (radians)
15     div.style.transform = 'rotate(' + ang + 'rad)';
16 }

```

For example, a simple implementation to test out the RotateDirection function, is shown below. The div shape rotates as the mouse moves around the window (always face towards the mouse cursor).

```

1  <html><head></head><body>
2  <div id="mydiv"
3      style="position: absolute;
4          background-color:blue;
5          left: 120px;
6          top: 200px;
7          width: 30px;
8          height: 60px;">Test</div>
9
10 <script>
11 function RotateDirection(div, dir)
12 {
13     var up = {x:0, y:1}; // reference direction
14     var dot = up.x*dir.x + up.y*dir.y; // dot product
15     var ang = Math.acos( dot ); // angle
16     var cross = up.x*dir.y - up.y*dir.x; // cross product

```

```

17   if ( cross < 0 ){ ang *= -1.0; }
18   div.style.transform = 'rotate(' + ang + 'rad)';
19 }
20
21 document.onmousemove = function(e){
22   var mpos = {x:e.pageX, y:e.pageY};
23   var mydiv = document.getElementById( 'mydiv' );
24   var xp = mydiv.style.left; // returns '120px'
25   var yp = mydiv.style.top; // returns '200px'
26   var w = mydiv.style.width; // returns '30px'
27   var h = mydiv.style.height; // returns '60px'
28   xp = parseFloat( xp ); // convert string to number
29   yp = parseFloat( yp ); // convert string to number
30   w = parseFloat( w ); // convert string to number
31   h = parseFloat( h ); // convert string to number
32   var dpos = {x:xp+w*0.5,
33               y:yp+h*0.5};
34
35   var dir = {x:dpos.x-mpos.x, y:dpos.y-mpos.y};
36   // Normalize the direction vector
37   var len = Math.sqrt(dir.x*dir.x + dir.y*dir.y);
38   dir.x = dir.x / len;
39   dir.y = dir.y / len;
40   // Convert the direction to a rotation
41   RotateDirection( mydiv, dir );
42 }
43 </script>
44 </body></html>

```

Remember when you extract position information from your elements, e.g., `mydiv.style.left` the return value is a string, such as, `20px`. If you want to use this value in your *calculations* you need explicitly convert your value to a number (using `parseFloat` or `parseInt`).



6. Javascript Advanced

6.1 Data (JSON)

As your games become more complex (more players and mechanics), you'll discover times when you need to store/restore data. For example, menu systems, save/load, multiplayer games and more. A popular data format, is the JSON format. JSON_ stands for *Javascript Object Notation (JSON)*. *JSON* is a lightweight format for storing and transporting data. *JSON* is often used when data is sent from a server to a web page. *JSON* is *self-describing* and easy to understand.

6.1.1 Javascript JSON helper functions

Two important Javascript JSON functions for converting too and from the JSON format are `JSON.parse` and `JSON.stringify`:

```
1 var sobj = '{ "name":"John", "age":30, "city":"New York"}';
2 // Convert string to JSON object
3 var obj = JSON.parse( sobj );
4
5 // Convert Javascript object to string
6 var myJSON = JSON.stringify(obj);
7
8 console.log( myJSON );
9 // Should display the original sobj
```

Some of the main reasons to use the JSON format are:

- It's a popular format
- It's human readable
- It's easy to debug and integrate into your game
- Supported by Javascript and other online languages such as, PHP

6.2 Pausing and Continuing Animations (Overriding CSS)

You alter the animation state by changing the `animationPlayState` variable (i.e., **paused** or **running**).

```

1  animationPlayState=false
2  // Call this function alter the animation state of the
3  // element so it *paused*
4  function pause() {
5      var animationElement = document.getElementById('myid');
6      animationElement.style.animationPlayState='paused';
7  }
8  function resume() {
9      var animationElement = document.getElementById('myid');
10     animationElement.style.animationPlayState='running';
11 }

```

6.3 Rotating and Animating (with CSS)

Define the class for the animation (class name `hi`). Specifies the file for the image (`tank.png`). Then you specify the animation speed (`.8s`) between frames. You then define the element should be rotated 10 degrees using the `transform: rotate(10deg);` line. The `myplay` defines the animation detailed set out in the `.hi` style (`animation: myplay .8s steps(8)infinite;`)

```

1  <style>
2  .hi {
3      position: absolute;
4      width: 83px;
5      height: 83px;
6      background-image: url("tank.png");
7      animation: myplay .8s steps(8) infinite; /* loop forever */
8
9      /* rotate around middle of the image */
10     transform-origin: 50% 50%;
11     transform: rotate(10deg);
12 }
13
14 @keyframes myplay {
15     from { background-position: 0px 0px; }
16     to { background-position: -672px 0px; }
17 }
18
19 /*
20  plus others for compatability, -webkit-keyframes, -moz-keyframes, -ms-keyframes,
  ↪ -o-keyframes

```

```

21 */
22 </style>

```

You can test out the animation class, by adding an element to the page:

```

1 <div class="hi"></div>

```

6.4 Rotating Programmatically

You set the orientation of the object using the `rotate` style - this can be done in the CSS style or you can alter it on the fly in the Javascript. The following Javascript will create a `timer` callback event. Each time the callback function is triggered, the angle is incremented and rotated.

```

1 <script>
2 var dd = document.getElementById( 'hi' );
3
4 dd.style.left = "230pt";
5 dd.style.top = "300pt";
6 dd.style.transform = "rotate(7deg)";
7
8 var ang = 7;
9 function animateCallback() {
10     ang += 10;
11     var ss = "rotate(" + ang + "deg)";
12     dd.style.transform = ss;
13 }
14
15 window.onload = function()
16 {
17     // animation tick/update
18     setInterval(animateCallback, ( 100 ));
19 }
20 </script>

```

While the rotation examples use `deg` (for degrees), you can also specify different units, for example *radians* using `rad`

6.5 Memory Management

While Javascript has its own *Memory Manager* (automatically manages memory allocation/release), there may be cases when you need to know the **usage** statistics. For example, you might detect some strange slowdowns/performance issues, and it might be an issue with some memory not getting released, fragmented or poorly structured?

One solution, is `window.performance.memory`, which is a propriety extension in **Google Chrome** that offers a glimpse into how your JavaScript applications are using browser memory. It exposes the following metrics:

```
1 {  
2   totalJSHeapSize: 18400000,  
3   usedJSHeapSize: 14200000,  
4   jsHeapSizeLimit: 1430000000  
5 }
```

This information, even if browser specific, enables you to extrapolate some high-level idea of how your application is using memory. Of course, if you try this in your browser console, you may notice that the numbers don't change. This is a **security protection** Chrome doesn't want to expose its internals to just anyone who might be listening. To enable more accurate memory monitoring, you need to start Chrome with the `--enable-precise-memory-info` flag.

The memory usage statistics can provide some interesting information as you advance. For instance, the memory usage is a useful piece of metadata about other JavaScript errors that occur (performance). Perhaps your application exhibits a different set of timings under heavy memory pressure, or fails in unique ways. This is pretty useful, especially when debugging modern interactive web applications (games), but it's **limited to Chrome with custom flags currently**.



7. Collisions and Intersections

7.1 Collision vs Intersection

We define a *collision* as the boolean check if two objects are touching or overlapping (true or false), while the *intersection* test is more complex and provides *contact* information (point the two objects are touching and the overlap amount).

You might use the collision detection check for *triggering* events, such as, particles, noises or menus in the game. While the intersection check, you'll use to prevent objects passing through walls or penetrating other objects (such as a car hitting a wall - slide along the wall but not actually penetrate).

7.2 Elements On Screen (Screen-Box-Area)

The steps to determine if an element is on screen are:

- Determine the size of the screen (width/height)
- Determine the position and size of the element (left/top/width/height)
- You then do the mathematics to calculate if the element is within the bounds.

7.3 Mouse-Element Check (Point in Box)

The steps to check if the mouse cursor is within the region of an element:

- Get mouse position
- Get the element position and dimensions
- Calculate if the point is inside or outside of the box

7.4 Axis-Aligned Bounding Box (AABB)

An example implementation for detecting the *Axis Aligned Bounding Box (AABB)* collision of two elements is given below:

```

1 function checkCollisionAABB(divIDA, divIDB)
2 {
3     // we get the elements from the DOM
4     dA = document.getElementById(divIDA);
5     dB = document.getElementById(divIDB);
6     // extract the elements positions/sizes
7     var wa = dA.offsetWidth;
8     var wb = dB.offsetWidth;
9     var ha = dA.offsetHeight;
10    var hb = dB.offsetHeight;
11    var xa = dA.offsetLeft;
12    var xb = dB.offsetLeft;
13    var ya = dA.offsetTop;
14    var yb = dB.offsetTop;
15
16    // calculate if the elements are overlapping
17    var dx = Math.abs(xa - xb) - (wa+wb)*0.5;
18    var dy = Math.abs(ya - yb) - (ha+hb)*0.5;
19    if ( dx < 0 && dy < 0 )
20    {
21        // code to deal with intersection
22        // move object dB away out of intersection
23        // using the overlap calculation. Pick the largest
24        // overlap
25        if ( dy < dx ) { dB.style.left = ( xb - dx ) + 'px'; }
26        else { dB.style.top = ( yb - dy ) + 'px'; }
27
28        return true; // Collision detected
29    }
30    return false; // No collision
31 }
```

7.5 Sphere-Sphere

An example for calculating the *sphere-sphere intersection* of two elements is given below:

```

1 function checkCollisionSphere(divIDA, divIDB)
2 {
3     // get the elements from the DOM
```

```

4    dA = document.getElementById(divIDA);
5    dB = document.getElementById(divIDB);
6    // extract the elements location/sizes
7    var wa = dA.offsetWidth;
8    var wb = dB.offsetWidth;
9    var ha = dA.offsetHeight;
10   var hb = dB.offsetHeight;
11   console.assert( wa===ha, "width and height must be the same" );
12   console.assert( wb===hb, "width and height must be the same" );
13   var xa = dA.offsetLeft;
14   var xb = dB.offsetLeft;
15   var ya = dA.offsetTop;
16   var yb = dB.offsetTop;
17
18   // centre of each circle
19   var xca = xa + wa*0.5;
20   var xcb = xb + wb*0.5;
21   var yca = ya + ha*0.5;
22   var ycb = yb + hb*0.5;
23   var ra = wa*0.5;
24   var rb = wb*0.5; // wb and hb
25
26   // use pythagoras - to calculate the radius
27   var len = Math.sqrt( (xca-xcb)*(xca-xcb) + (yca-ycb)*(yca-ycb) );
28
29   // do the sphere-sphere check if they're touching
30   var delta = len - (ra+rb);
31   if ( delta < 0 )
32   {
33       // collision
34       var cdir = { x: (xca-xcb)/(len), y: (yca-ycb)/(len) };
35
36       dB.style.left = (xb + cdir.x * delta) + 'px';
37       dB.style.top = (yb + cdir.y * delta) + 'px';
38       return true;
39   }
40   return false; // No collision
41 }

```

While the majority of the time you'll only be concerned with simple approximations (rectangles and spheres). However, it's possible for more accurate representations using **Convex** and **Concave** shapes. This means you'll move into some of the advanced collision detection algorithms, such as, line-line collision detection, separating-axis theorem and more.

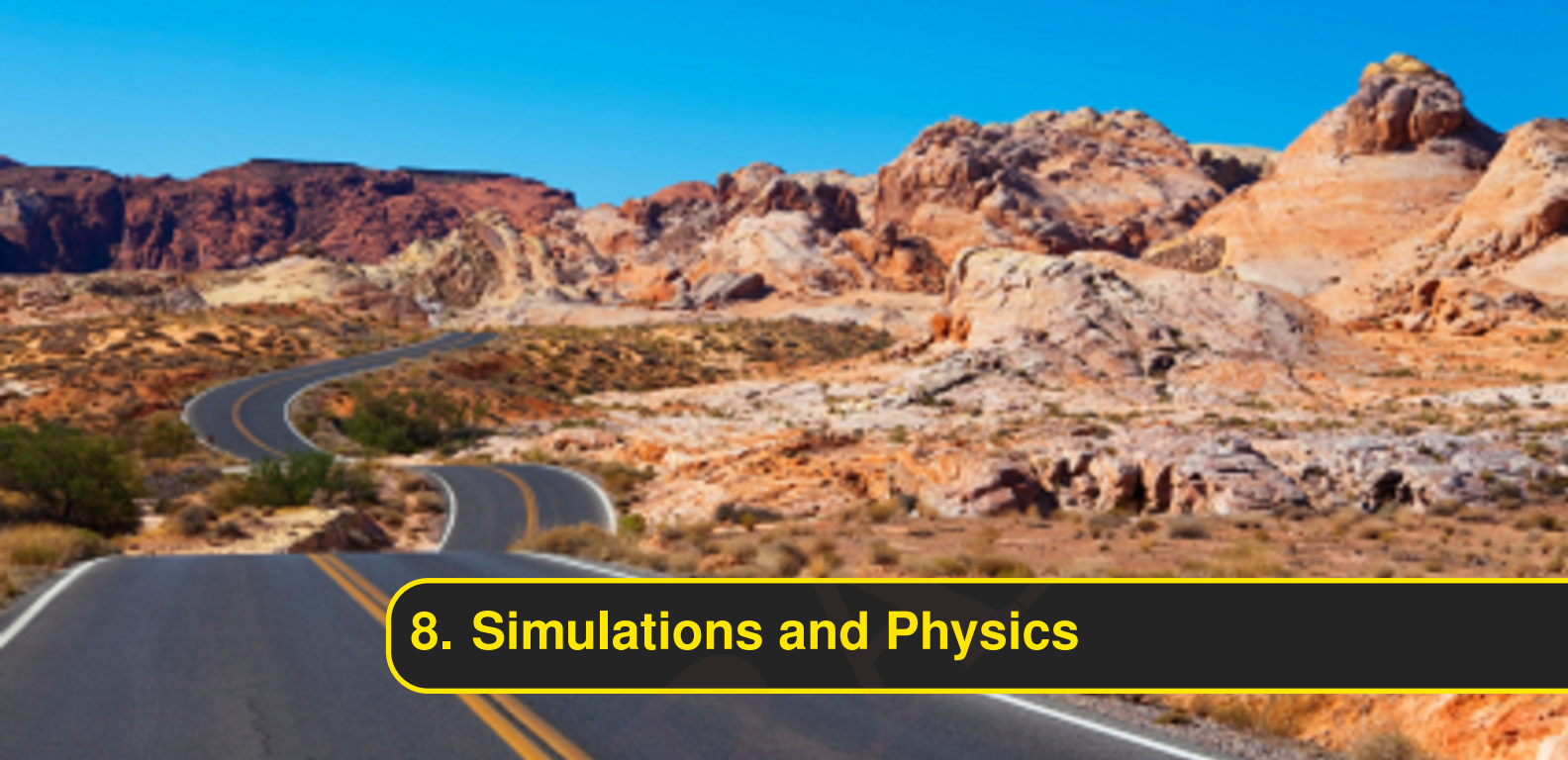
7.6 Shape Hierarchies/Decomposition

Sphere and cubes are popular collision detection algorithms, due to their simplicity and reliability. However, there may be occasions when you have a shape which isn't circular or square and requires *accurate* collision/intersection information. Instead of developing a complex accurate collision detection algorithm (e.g., triangle-triangle), you can just use more simple primitive collision tests. Use lots of smaller spheres to represent the shape geometry. For example a concave shape with lots of holes would benefit from this technique.

7.7 Problems

Developing a collision detection/intersection algorithm is only part of the solution, there are a variety of issues you need to be aware of in interactive gaming environments, such as:

- Tunnelling
- Overlap (unable to recover in a natural/realistic way)
- Performance (space partitioning techniques)



8. Simulations and Physics

Adding gravity, momentum, forces, torques, ... means you'll have to run some sort of simulation. You can't pre-define the positions and movement of objects as you don't know how the player will react and interact with the game and objects.

Physics in games ensures your game has that right feel. The feel is about how the game feels to the player. It's the user experience of game mechanics and dynamics. It's one thing to know physics, and it's another to make physics work and feel well in a game. Of course, it depends on the type of game you're making. However, for an interactive game, with multiple elements reacting with one another in a realistic way (gravity and collisions) - basing their moving on physical laws gives them a more natural motion.

Not just about realistic physics, it's about what works for your game. For basic games (not about solving puzzles but have interactive aspects and animations), realistic physics isn't all that matters, what matters is the movement is **convincing** and satisfying for the player.

For instance, animation (or visible motion) is important in games where the players need to know to avoid objects coming at them with a lot of momentum, and crave visceral feedback for a well-executed move.

Your physics system doesn't need to be scientifically accurate - it just needs to behave in a way that you need. For instance, it might have very realistic aspects, causing objects to squash and stretch, bounce and flail, excite and ricochet with reckless abandon (or this might be too complex and computationally expensive for what your game requires).

The important thing isn't realism, it's consistency with the rest of your game's animation and artistic style, however, physics can do so much more than improve cosmetics (game design, constraints, laws and consistency).

8.1 Newton's Laws (Basics)

At the heart of classical mechanics and physical laws are Newton's principles (or laws). Newton's laws form the foundation of mechanics (i.e., large objects). These laws are relatively straightforward to digest and comprehend (you just have to think about them). *Newton's three laws* of motion can be stated as follows:

- Every object in a state of uniform motion will remain in that state of motion unless an external force acts on it
- Force equals mass times acceleration
- For every action there is an equal and opposite reaction

8.2 Relationship between position, velocity and acceleration

The famous equation and the means for calculating the relationship between force and access is:

$$f = m a$$

where *f* is *force*, *m* is *mass* and *a* is *acceleration*.

Acceleration, velocity and position are **connected** through *calculus* (rate of change - **calculus**). As a change in acceleration, causes a change in velocity which leads to a change in position.

An example implementation of $f=ma$ with velocity and position in action is given below:

```

1  function update( dt ) // dt - timestep (duration between last update)
2  {
3      // f - force
4      // minv - inverse mass (1/mass) - better for computers
5      // as we can set minv to zero if we want objects 'immovable'
6      // avoid divide by zero errors
7      a = (f * minv); // f = m a
8      v = v + a * dt; // v - velocity, a - acceleration
9      p = p + v * dt // p - position
10
11     f = 0; // clear forces
12 }
```

where the `update` function would get called repeatedly in the main game loop. Forces would be added to the objects to control them. Important to note that after a force has been applied, you'd clear it. Otherwise the force would repeatedly keep getting added to keep increasing the

velocity and keep making the object move faster and faster. When you apply a force, it's been processed, so you clear the accumulated forces.

The basic *simulator* example shows a box falling due to a downward force (e.g., *gravity*). You should try adding in an additional sideways force (only one extra line), `that.element.fx = 1.2;`, so the object also moves to the right as well as falling down.

8.3 Forces to Control Movement

Once you've got your update *integration* loop running, to inject *movement* into scene (move any movable objects) you apply a force. This force is *accumulated*, as you can add multiple forces from multiple sources (e.g., control force from the keyboard, a collision forces, a force from wind or gravity). All the forces are accumulated and then *integrated* into the velocity and position in the update loop.

```
1 function addForce( force )
2 {
3     f += force;
4 }
```

8.4 Vectors (2-dimensional movement)

In the previous discussion, you learned about how to add physics into your game. However, forces, velocities and position are *vector quantities*. Hence, you'll apply the concepts in parallel to both *x* and *y* components.

```
1 <html><head></head>
2 <body onload="startSimulation()">
3
4 // So our simple test object (div) is a blue box
5 <style>
6     #mybox {
7         background-color: blue;
8         width : 50;
9         height : 50;
10        position : absolute;
11    }
12 </style>
13
14 <script>
15
16 var simulator = {
17     // element and timer
18     element : 0,
19     interval : 0,
20     that : 0, // object instance
21     // store element details - setup default position, update loop
22     start : function() {
23         // Object is only a declaration. You need to actually
24         // specify the instance, i.e., either call functions
25         // using simulator.stop() or store the instance as `that`
```

```

26      // so we can call functions like that.stop()
27      that = this;
28      this.element = document.getElementById( 'mybox' );
29      // set the starting position of the element
30      this.element.x = 300;
31      this.element.y = 100;
32      this.element.vx = 0;
33      this.element.vy = 0;
34      this.element.fx = 0;
35      this.element.fy = 0;
36      this.interval = setInterval(this.update, 20);
37  },
38  stop : function() {
39      clearInterval(this.interval);
40  },
41  // integrate force/vel/pos and apply to the elment
42  integrate : function() {
43      // mass 1.0
44      // timestep of 1.0
45      this.element.vx += this.element.fx;
46      // positive upward/negative down on screen
47      this.element.vy -= this.element.fy;
48
49      // clear forces
50      this.element.fx = 0;
51      this.element.fy = 0;
52
53      this.element.x += this.element.vx;
54      this.element.y += this.element.vy;
55
56      this.element.style.left = this.element.x + "px";
57      this.element.style.top = this.element.y + "px";
58  },
59  // keep updating, adding forces and integrating
60  // force and velocity
61  update : function () {
62      that.element.fy = -0.2;
63      that.integrate();
64
65      // fallen off screen, reset and restart simulator
66      if ( that.element.y > screen.height )
67      {
68          that.stop();
69          that.start();
70      }
71  }
72 }
73 // start the simulation going when loaded
74 function startSimulation()
75 {
76     simulator.start();
77 }
78 </script>
79
80 <!-- put our element on screen -->
81 <div id="mybox"></div>
82 <p>Simple object falling due to gravity</p>
83
84 </body></html>

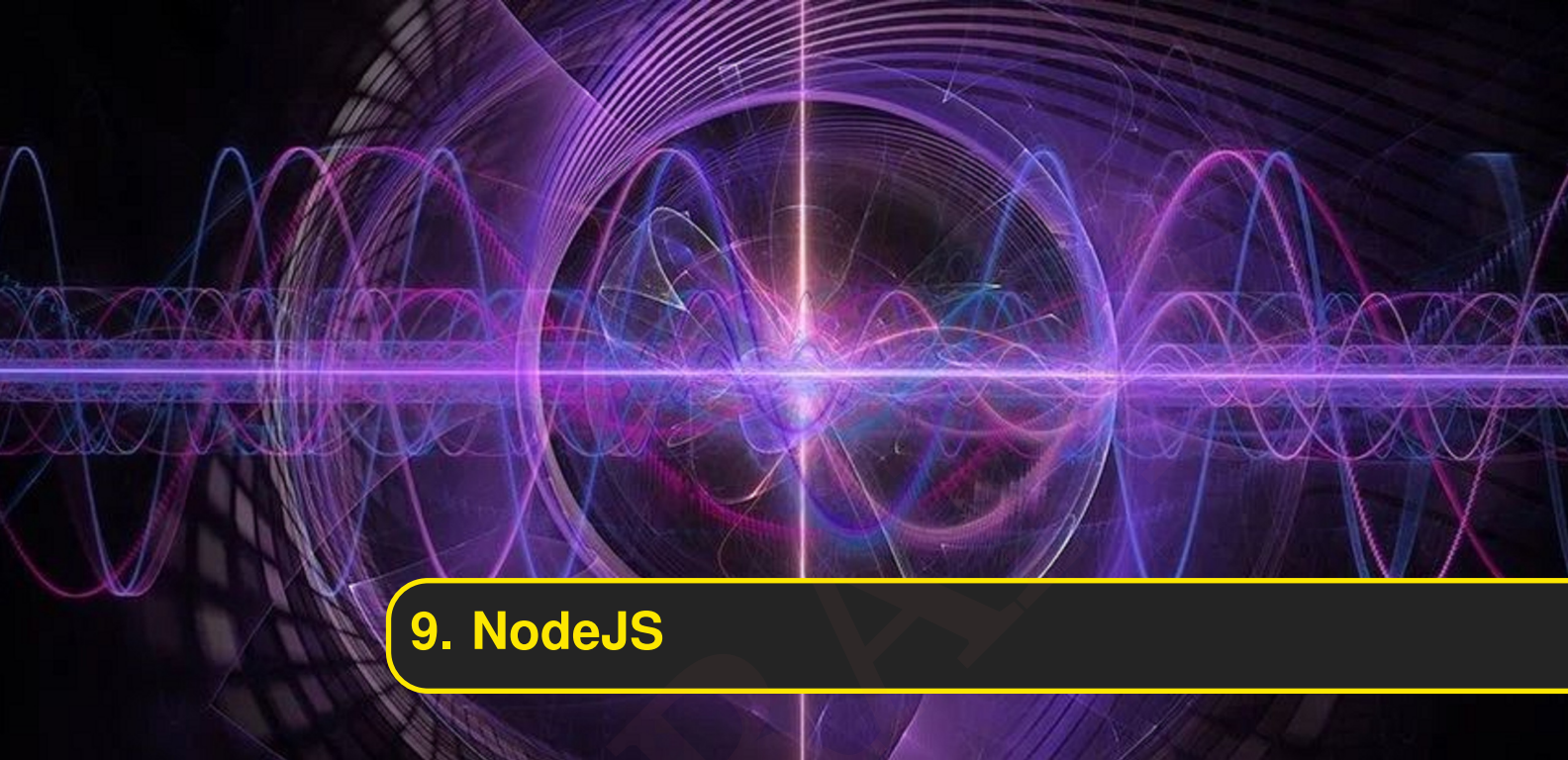
```

Another enhancement, currently when the box falls off the bottom of the screen the simulation is reset (object put back to the starting position). Instead you could apply an upward collision force - so it bounces along the bottom of the screen.

8.5 Time Step (Fixed)

Why should the update time be fixed you may ask? Having a fixed *time step*, means your game and simulations will be *deterministic* - each time you run your game simulation with the same input parameters, you get the same results/output (don't want randomness). You don't want it to work sometimes and not other times because of fluctuations with processor speed or other applications running at the same time.

Each time you update your simulator, you'll pass in a fixed **duration**, such as, 100th of a second (0.01). To make sure your simulation stays in sync with the real world, you call the update more or less times (for example, if 1 second has passed you'd call the update 100 hundred times). So the simulation is real-time, however, the simulation duration is fixed - so you don't get numerical issues or errors as the time frame jumps around in your browser.



9. NodeJS

9.1 What is NodeJS?

NodeJS is a popular open-source, cross-platform run-time environment used for developing server-side web applications. NodeJS applications are written in JavaScript which means it can run on a wide variety of operating systems (Android, Windows, Apple's MacOS and Linux). This is because the NodeJS run-time environment is built on Chrome's celebrated V8 Javascript engine.

9.2 Why Learn and Use NodeJS?

NodeJS's popularity booming, not to mention, the tons of NodeJS resources all over the internet. NodeJS development is extremely powerful, fun and satisfying. NodeJS is programmed in Javascript, which means you'll be using a single language across the stack (unified language and data format) allowing you to optimally develop and reuse resources. NodeJS gives you a sleek tool for working in non-blocking, event-driven I/O paradigms.

A **web stack**, also called a web application stack, is a compilation of software set up especially for implementing websites and web applications. The term, 'stack', refers to the fact that the system's individual components are built upon one another



Figure 9.1: **NodeJS(R)** - Official NodeJS Logo/Trademark.

In a nutshell, NodeJS is useful for building fast and scalable server-side networking applications.

You'll not only learn one of the most in-demand modern languages, environments, and tools, but also how to apply them to your own projects and real-world situations. Additionally, as you progress through these short notes, you use small projects to help you develop a rock-solid professional portfolio that can be used to impress potential employers.

NodeJS shines in real-time web applications employing push technology over web-sockets. What is so revolutionary about that? Well, after over 20 years of stateless-web based on the stateless request-response paradigm, we finally have web applications with real-time, two-way connections, where both the client and server can initiate communication, allowing them to exchange data freely.

NodeJS supports both client and server side applications. It is based on JavaScript and is very fast in operation. These distinctive features make NodeJS one of the most powerful frameworks. NodeJS allows you to build real-time, scalable mobile and web applications (as demonstrated in these notes).

To summarize the juicy points that make NodeJS so popular among developers:

- **It is fast**- NodeJS is powered by the V8 javascript engine which is developed by Google so compiles and executes Javascript at a brisk pace
- **It is lightweight**- NodeJS is an event driven architecture, which means every single operation is a series of asynchronous callbacks, this allows NodeJS to run on a single thread unlike other

web technologies

- Since it's **Javascript**, you can use almost every feature of Javascript in NodeJS, every advancement in Javascript makes coding in NodeJS more easier (a single language for all layers)
- **Open Source** Libraries- NodeJS has a lot of libraries! This helps in performing various operations, this is largely due to a massive community of developers
- **Footprint**- Since it has a small footprint it is used in a lot IoT (Internet of Things) Applications
- It's **easy to modify and maintain** - traditional applications are less adaptive and rigid over time as new requirements are fed. Eventually, they start creaking under the stress they were not built for. However, developing new services using NodeJS is comparatively easier. As with Node, a bunch of small applications is built instead of a single, large application. This allows making changes or adding a new functionality much ease, without having to make changes deep inside the code-base

9.3 Who's using NodeJS?

NodeJS is used by many large companies, for example, to name a few of them:

- **Paypal** ? A lot of sites within Paypal have also started the transition onto NodeJS
- **LinkedIn** - LinkedIn is using NodeJS to power their Mobile Servers, which powers the iPhone, Android, and Mobile Web products.
- **Mozilla** has implemented NodeJS to support browser APIs which has half a billion installs.
- **eBay** hosts their HTTP API service in NodeJS



Figure 9.2: *Who Uses NodeJS?* - A large number of companies use NodeJS (a few you might have heard of)

9.4 NodeJS Architecture (Single-Threaded)

NodeJS uses a Single Threaded Event Loop Model Architecture which means all the client requests on NodeJS are executed by a single thread. This is important, as this architecture is not just single-threaded, but **event-driven** as well which helps NodeJS in handling multiple clients concurrently. As NodeJS is based on a single-threaded and event-driven architecture which supports concurrency.

9.4.1 Events

As you'll learn in these notes, one of the reasons NodeJS is so popular is because of its **event-driven approach**. Events enable you to develop efficient non-blocking implementations. As your event-driven solutions written in NodeJS respond to actions generated by the system instead of polling or waiting for state changes resulting in a more efficient approach.

9.5 Getting Started with NodeJS?

There are two ways to run NodeJS your examples. First you could download and install NodeJS on your computer (that is by running examples locally). Alternatively, you can run your solutions on one of the many online tools for creating, managing and hosting NodeJS environments.

9.6 Installing and Testing NodeJS

9.6.1 Locally

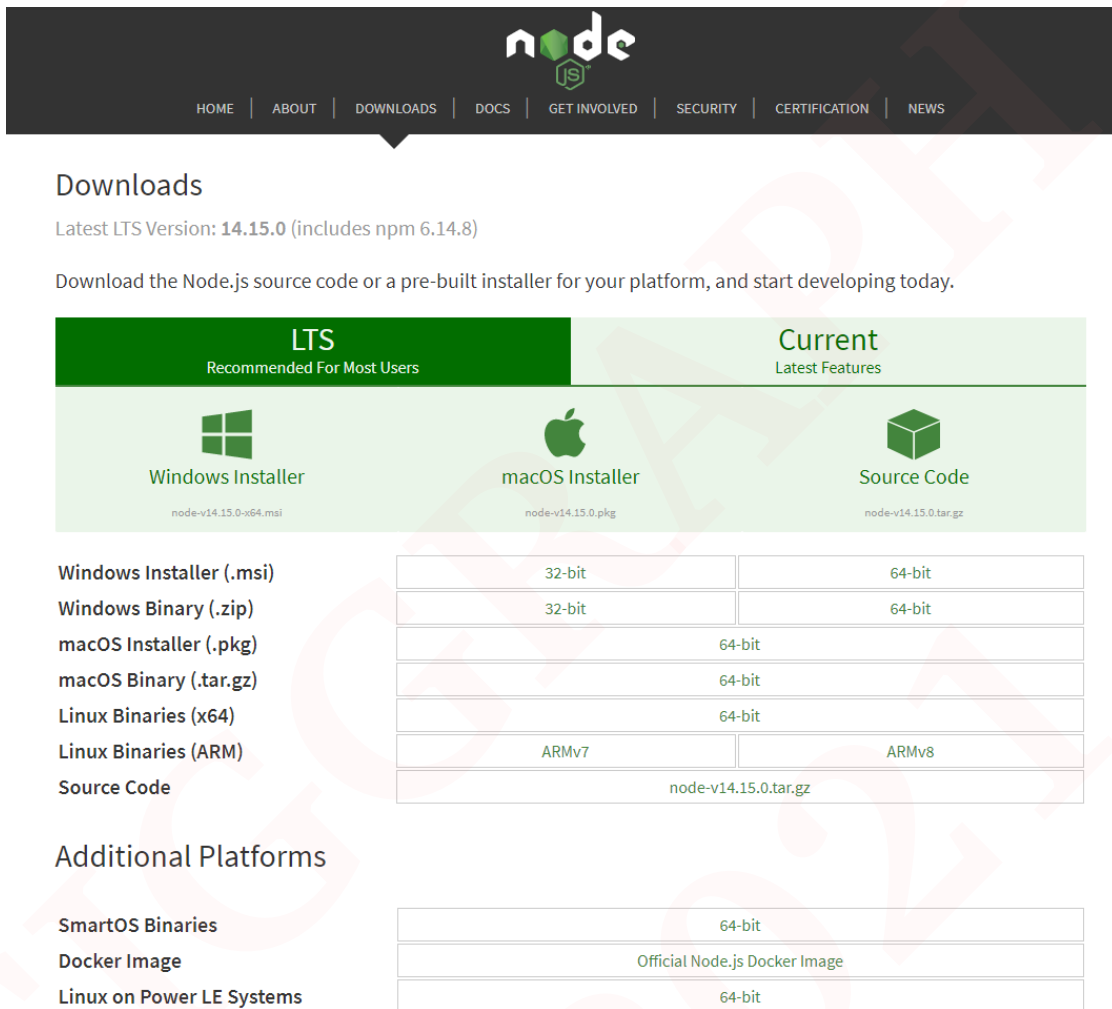
First you'll learn how to setup and run a NodeJS project locally on your personal computer.

9.6.1.1 Where to find NodeJS?

Open your web browser and navigate to the NodeJS homepage where you can download and install NodeJS for your system.

- Download from <https://nodejs.org/en/download/>
- Supports Windows, macOS and Linux

Node Version The easiest thing to do as a quick test of your install is to get the version of NodeJS that you've installed. Type the following:



The screenshot shows the Node.js homepage with a dark header containing the Node.js logo and navigation links: HOME, ABOUT, DOWNLOADS, DOCS, GET INVOLVED, SECURITY, CERTIFICATION, and NEWS. Below the header, the 'Downloads' section highlights the 'Latest LTS Version: 14.15.0 (includes npm 6.14.8)'. It instructs users to download the Node.js source code or a pre-built installer. The page is divided into two main sections: 'LTS Recommended For Most Users' and 'Current Latest Features'. Under 'LTS', there are links for 'Windows Installer' (node-v14.15.0-x64.msi), 'macOS Installer' (node-v14.15.0.pkg), and 'Source Code' (node-v14.15.0.tar.gz). Under 'Current', there are links for 'Windows Installer (.msi)', 'Windows Binary (.zip)', 'macOS Installer (.pkg)', 'macOS Binary (.tar.gz)', 'Linux Binaries (x64)', 'Linux Binaries (ARM)', and 'Source Code'. A table lists the available binaries for each platform and architecture. Below the 'Downloads' section, the 'Additional Platforms' section lists 'SmartOS Binaries', 'Docker Image', and 'Linux on Power LE Systems', each with a corresponding download link.

Platform	Architecture	Download Link
Windows Installer (.msi)	32-bit	node-v14.15.0-x64.msi
	64-bit	node-v14.15.0-x64.msi
Windows Binary (.zip)	32-bit	node-v14.15.0-x64.msi
	64-bit	node-v14.15.0-x64.msi
macOS Installer (.pkg)	64-bit	node-v14.15.0.pkg
	64-bit	node-v14.15.0.pkg
macOS Binary (.tar.gz)	64-bit	node-v14.15.0.pkg
	64-bit	node-v14.15.0.pkg
Linux Binaries (x64)	64-bit	node-v14.15.0.pkg
	64-bit	node-v14.15.0.pkg
Linux Binaries (ARM)	ARMv7	node-v14.15.0.pkg
	ARMv8	node-v14.15.0.pkg
Source Code		node-v14.15.0.tar.gz

Platform	Architecture	Download Link
SmartOS Binaries	64-bit	node-v14.15.0.pkg
	64-bit	node-v14.15.0.pkg
Docker Image	Official Node.js Docker Image	node-v14.15.0.pkg
	Official Node.js Docker Image	node-v14.15.0.pkg
Linux on Power LE Systems	64-bit	node-v14.15.0.pkg
	64-bit	node-v14.15.0.pkg

Figure 9.3: **NodeJS** - NodeJS Homepage (<https://nodejs.org/en/download/>.)

```
1 node --version
```

That should show you something along the lines of this:

```
1 v12.14.1
```

Welcome After you've installed NodeJS, open a command prompt and type:

```
1 \> node
```

If your installation when successfully, you should get a message saying 'Welcome to NodeJS' and the version of NodeJS that you've installed.

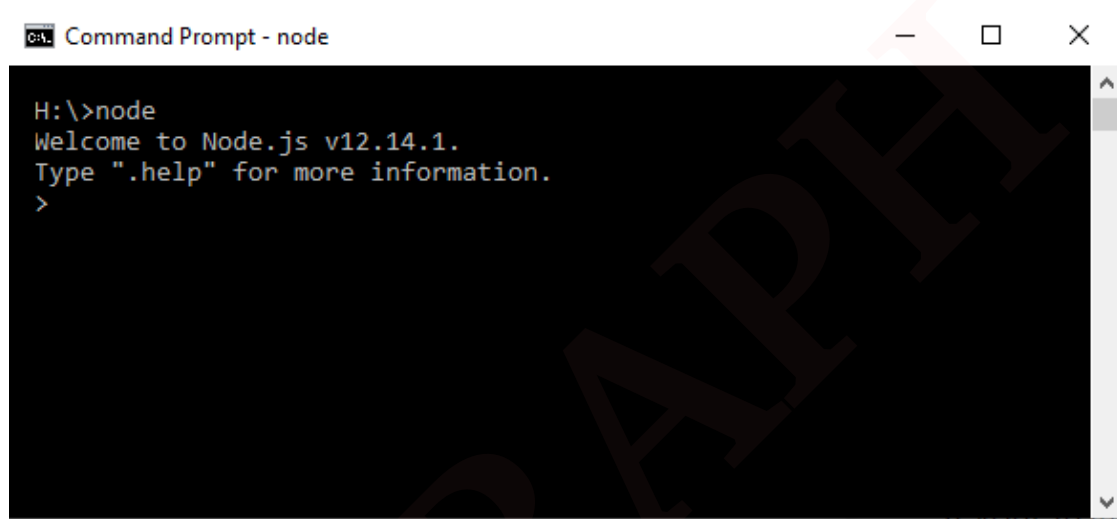


Figure 9.4: *Command Prompt* - Screenshot of the output.

9.6.1.2 Hello NodeJS

In your command prompt, after typing `node`, type the following command:

```
1 console.log('Hello Node');
```

You should see your message returned below the command:

```
1 Hello Node
2 undefined
```

⚠ Don't worry about the `undefined` word. You'll learn more about that later. Essentially, it's telling you that the command you called (`log`) didn't return anything.

9.6.1.3 Text Editors and '.js' Files

Once you've got NodeJS installed and setup, you're ready to start writing some simple scripts. Any text editor is fine. For example, `Notepad++` is a popular option as it's free, open source and has syntax coloring.

Download from <https://notepad-plus-plus.org/downloads/>

Since NodeJS uses the Javascript language, as you'll probably guess, the NodeJS script files are stored with the `.js` extension (indicating they're Javascript files).

Create a new file called `'test.js'` and type the following lines of code:



Figure 9.5: *Notepad++ - Application logo/icon.*

```
1 let a = 1;  
2 a++;  
3 console.log('a is:', a);
```

The script declares a variable called `a` using the keyword `let`. The variable is assigned the value `1`. On the following line, you then increment the variable using the `++` operator. Finally, you output the value to the console using the `console.log` function.

Open a command prompt and navigate to the folder you've got your `test.js` script saved to. Then type the following command to run your script:

```
1 \>node test.js
```

You should get the output telling you that `a` is `2`.

9.6.2 Online Tools

You can write and develop web applications in NodeJS using online tools through your web browsers, such as:

Visit: <https://codesandbox.io/>

Goto 'codesandbox.io' and create a new sandbox, be sure you select a NodeJS template:

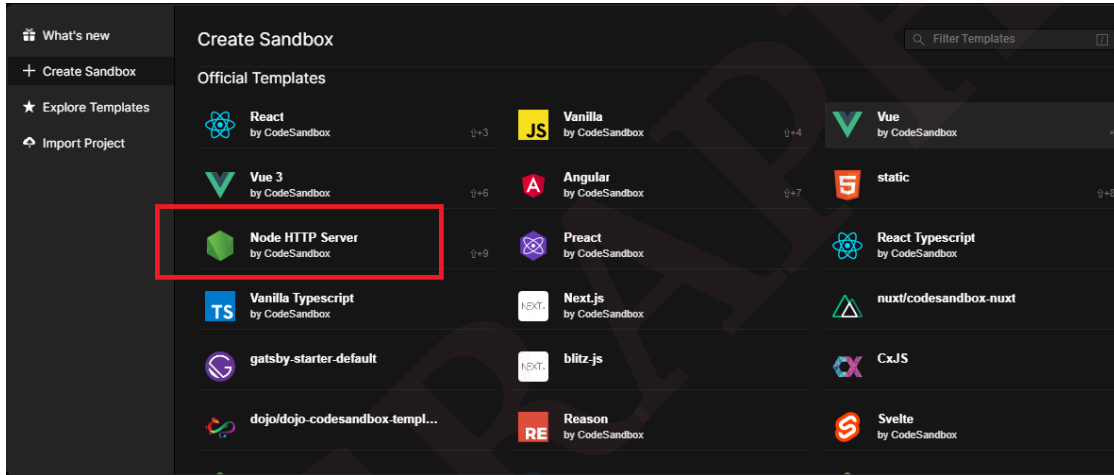


Figure 9.6: *CodeSandBox* - Creating a new NodeJS project space.

Once you've selected your sandbox type, you'll be given everything you need to start developing your NodeJS applications.

You can directly type your scripts in the browser window. You can add files (css, html, js script and other resources such as imgs). You can even see the generated output (preview window).

9.6.2.1 Simple Online Test

As you're only just starting out, let's repeat a simple test as discussed in previous sections. For the online editor, delete the generated example from 'index.js' and type the following code:

```
1 let a = 10;
2 a++;
3 console.log("a is", a);
```

Then press the `Ctrl` and `S` keys to save your file. After you save your file, the online editor will detect your changes and run your script. You should see the console output in the browser display your script result (i.e., `a is 11`).

9.6.2.2 Online Editor

Initially, you might get overwhelmed by the online editor, as there are so many options. However, as you start to become more proficient at NodeJS, you'll start to see that all these extra options

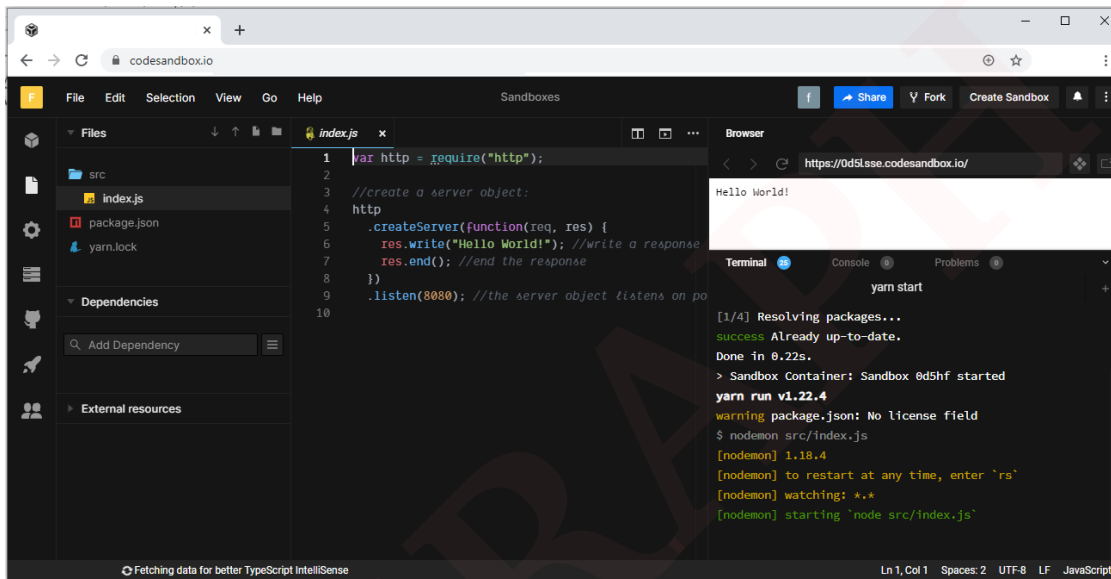


Figure 9.7: *CodeSandBox* - Default project space for a newly created NodeJS project.

are important. For example, some of the most important ones that you'll use in these notes, include:

- (a) File structure (upload, edit and select files/folders)
- (b) Dependencies (packages such as, 'socket.io', 'express', ...)
- (c) Editor (file you've got open and are editing)
- (d) Editor window (lines and code for the file you're editing)
- (e) Browser preview output (also copy and paste the url to any other browser window)
- (f) Terminal window (debug output/server status)

9.6.3 Alternatives to NodeJS (e.g., NodeJS vs PHP)

NodeJS isn't the only server-side solution for developing your applications. There are other, such as:

- Perl,
- ASP.NET,
- Deno,
- PHP,
- ...

In short, although there are other run-time environment, NodeJS is a popular and well-established technology that will be around for a long time.

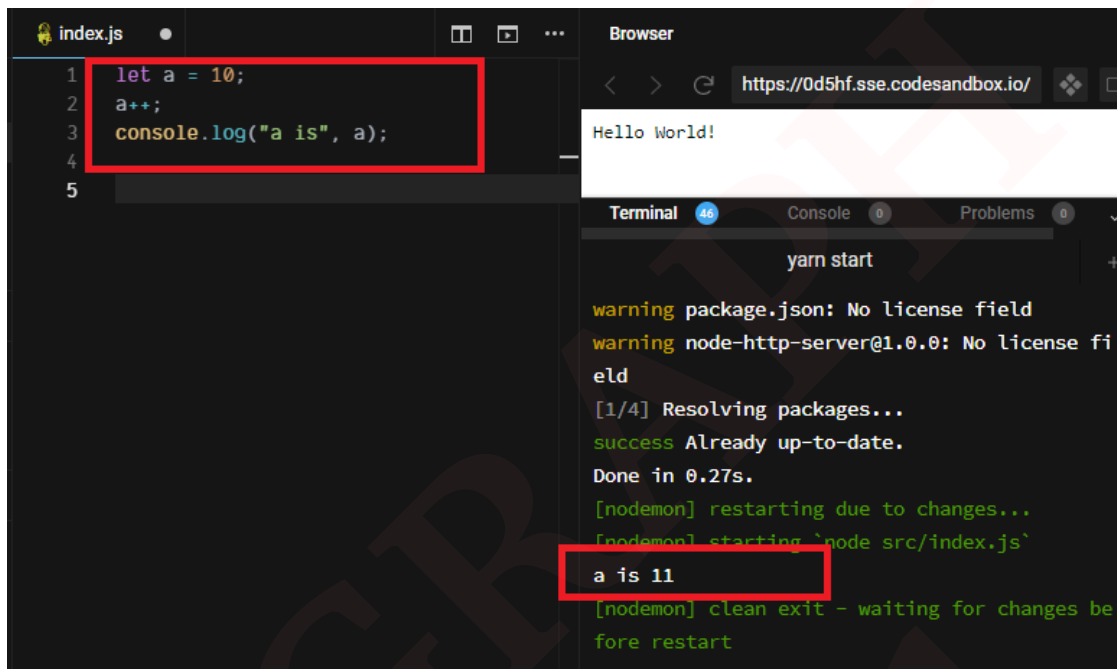


Figure 9.8: **Online Editor** - Show the key windows for editing, compiling and running NodeJS programs.

9.7 Summary

You're living in the age of web technologies. In every industry, the demand for web developers is high and continues to rise, yet there's a shortage of skilled NodeJS developers. As a result, there's a huge number of exciting opportunities out there for NodeJS developers who want to take them. As you master the skills taught in these notes, you'll prepare yourself for roles in a wide range of industries that use NodeJS. Whether you want to work in a startup game studio or develop web pages, you'll learn in-demand knowledge and techniques that will help you succeed.

9.8 Questions

These notes **quickly** go through NodeJS with multiple hands-on examples that enable you to create web sites, chat-clients and simple online board-games. To check your basic 101 knowledge of NodeJS, try the following questions.

Question: What is NodeJS? NodeJS is a Server side scripting which is used to build scalable programs. Its multiple advantages over other server side languages, the prominent being non-blocking I/O.

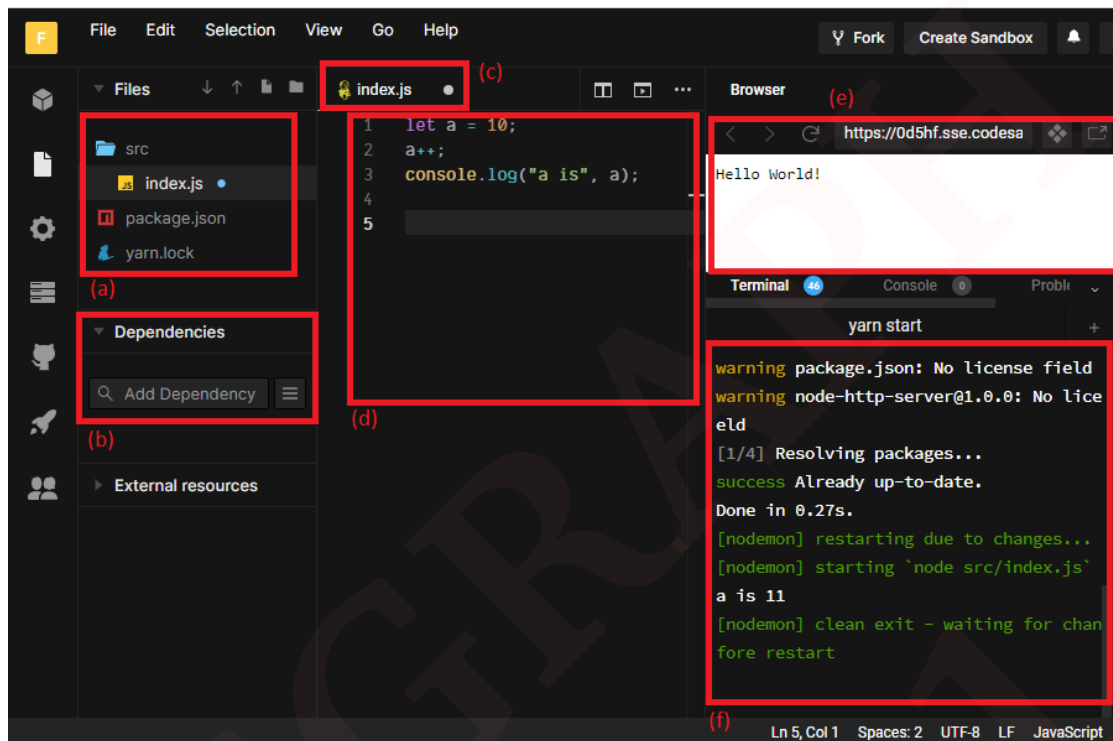


Figure 9.9: *CodeSandBox* - Online project space windows for selecting, editing, configuring and running your NodeJS projects.

Question: How does NodeJS work? NodeJS works on a v8 environment, it is a virtual machine that utilizes JavaScript as its scripting language and achieves high output via non-blocking I/O and single threaded event loop.

Question: What do you mean by the term I/O? I/O is the shorthand for input and output, and it will access anything outside of your application. It will be loaded into the machine memory to run the program, once the application is started.

Question: What does event-driven programming mean? In computer programming, event driven programming is a programming paradigm in which the flow of the program is determined by events like messages from other programs or threads. It is an application architecture technique divided into two sections 1) Event Selection, and 2) Event Handling.

Question: Where can you use NodeJS? NodeJS can be used for the following purposes.

- Web applications (especially real-time web apps)
- Game servers
- Network applications

- Distributed systems
- General purpose applications

Question: What is the advantage of using NodeJS?

- It provides an easy way to build scalable network programs
- Generally fast
- Great concurrency
- Asynchronous everything
- Almost never blocks

Question: What are the two types of API functions in NodeJS? The two types of API functions in NodeJS are:

- Asynchronous, non-blocking functions
- Synchronous, blocking functions

Question: Why is NodeJS single-threaded? For async processing, NodeJS was created explicitly as an experiment. It is believed that more performance and scalability can be achieved by doing async processing on a single thread under typical web loads than the typical thread based implementation.

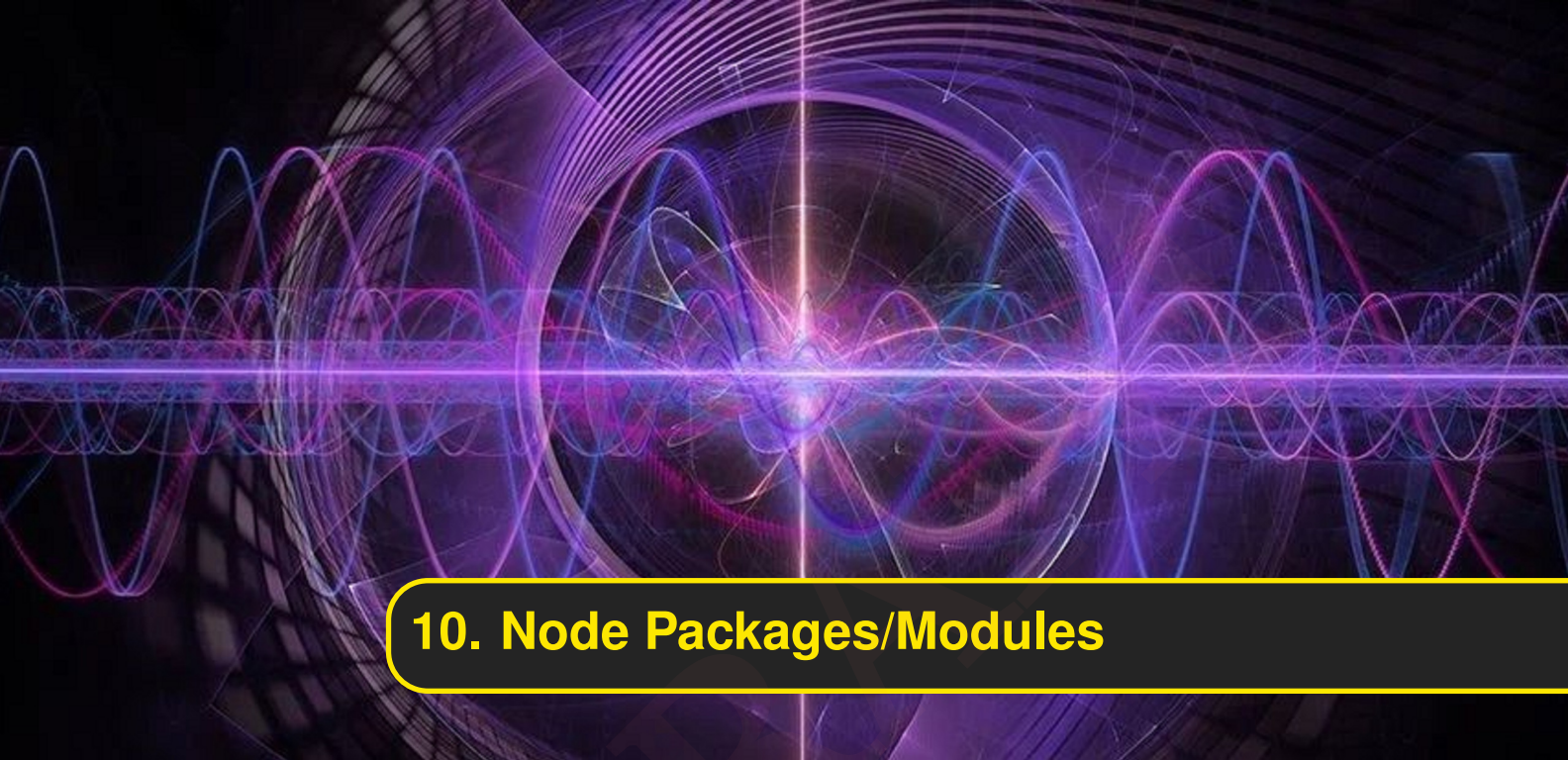
Question: Does node run on windows? Yes ? it does. Download the MSI installer from <http://nodejs.org/download/>

Question: Why is NodeJS quickly gaining attention from web programmers? NodeJS is quickly gaining attention as it is a loop based server for JavaScript. NodeJS gives user the ability to write the JavaScript on the server, which has access to things like HTTP stack, file I/O, TCP and databases.

Question: How does NodeJS overcomes the problem of blocking of I/O operations? NodeJS solves this problem by putting the event based model at its core, using an event loop instead of threads.

Question: What are the Challenges with NodeJS? Emphasizing on the technical side, it's a bit of challenge in NodeJS to have one process with one thread to scale up on multi core server.

Question: What does it mean ‘non-blocking’ in NodeJS? In NodeJS ‘non-blocking’ means that its IO is non-blocking. Node makes a non-blocking request and upon a request, it queues it within the event loop which call the JavaScript ‘callback’ on the main JavaScript thread.



10. Node Packages/Modules

10.1 What is a Module?

Consider a module to be the same as a JavaScript library which contains a set of functions you want to include in your application.

10.1.1 Built-in Modules

NodeJS has a set of built-in modules which you can use without any further installation. For example, some of the built-in modules (and ones you'll use in these notes) include:

- **fs** To handle the file system
- **http** To make NodeJS act as an HTTP server
- **url** To parse URL strings
- **events** To handle events

You can lookup the full list of Built-in Modules in the NodeJS documentation.

10.2 Difference between a package vs module?

While navigating the NodeJS documentation and online resources, you might come across the terms **modules** and **packages**. They're not the same!

A **module** is a single JavaScript file that has some reasonable functionality. For example, a module can be any file or directory that can be loaded by the NodeJS `require()` function.

A **package** is a directory with one or more modules inside of it and a 'package.json' file which has metadata about the package.

10.2.1 What is the file 'package.json'?

You'll stumble across a package file called ('package.json') which is usually in the project root. This file holds various metadata relevant to your project. This file is used for information to identify your project as well as handling your project dependencies. It can also contain other metadata such as a description of your, the version of your project, any licensing information, and even configuration data. All of which can be vital for your running implementation and to the end users.

Here is what your 'package.json' file might look like:

```
1 {  
2   "name" : "skeletonexample",  
3   "version" : "0.0.0"  
4 }
```

The file uses the `json` format (JavaScript Object Notation), hence the curly brackets, colons and commas. Over time, you'll update your file, such as, adding dependencies, keywords, descriptions, version details, and so on.

10.3 What is NPM (Node Package Manager)?

The Node Package Manager (aka NPM) manages your NodeJS packages (and modules). When you installed NodeJS, it also installed NPM onto your computer.

For example to check that NPM is installed on your computer. Open the command terminal and type:

```
1 \>npm -v
```

This should return the version of NPM you have installed.



Figure 10.1: NPM - Node package manager.

10.4 Initializing your project

Create a folder and navigate to this new directory in your command prompt. In the command prompt type:

```
1 \>npm init
```

You should then be taken through the steps for setting up your project package. Most of the time you can just click 'ok' to continue with the defaults. Here is what you will see:

```
1 \>npm init
2
3 This utility will walk you through creating a package.json file.
4 It only covers the most common items, and tries to guess sane defaults.
5
6 See `npm help json` for definitive documentation on these fields
7 and exactly what they do.
8
9 Use `npm install <pkg> --save` afterwards to install a package and
10 save it as a dependency in the package.json file.
11
12 Press ^C at any time to quit.
13 package name: (draw)
14 version: (1.0.0)
15 description:
16 entry point: (server.js)
17 test command:
18 git repository:
19 keywords:
20 author:
21 license: (ISC)
22 About to write to D:
23 ode-tests\draw\package.json:
24
25 {
26   "name": "draw",
```

```

27   "version": "1.0.0",
28   "description": "",
29   "main": "server.js",
30   "scripts": {
31     "test": "echo \"Error: no test specified\" && exit 1",
32     "start": "node server.js"
33   },
34   "author": "",
35   "license": "ISC"
36 }
37
38 Is this OK? (yes)

```

After this completes, you should notice that you have a ‘package.json’ file in your directory. You’re now all setup and ready to install modules and run your node applications.

Note: You can create and update your ‘package.json’ manually, however, if you let the amazing NPM package manager create your package.json for you, then every time you install modules using the NPM package manager it will also keep your package.json file updated with all the modules you are using. Why bother manually creating and keeping your ‘package.json’ to update with all your dependencies when NPM will do this for you! Simple run `pm install - save module_name` and npm will automatically update your ‘package.json’ with the correct module and version name

10.4.1 Installing modules

You can download and install package very easy. Let’s say you want to add the module called ‘socket.io’. Open the command line interface and tell NPM to download the package you want.

```
1 >npm install socket.io
```

Now you have downloaded and installed your first package!

NPM creates a folder named ‘node_modules’, where your package will be placed. All packages you install in the future will be placed in this folder.

Your project folder structure would be like this:

```

1  \package.json
2  \index.js
3
4  node_modules\socket.io

```

Warning - when you install modules be sure that you navigate to the project folder in the command prompt (i.e., you install modules for a specific project, not for all projects)

10.4.2 Using Modules

Once your modules is, it is ready to use. You do this using the `require` call.

```
1 var sc = require('socket.io');
```

10.4.3 Standard Modules

You can run simple NodeJS examples (e.g., web server) without installing any modules. As there are a certain number of pre-installed modules that come with NodeJS by default.

- HTTP (built-in)
- Express (install)
- Socket.io (install)
- FS (built-in)

10.4.4 Creating your own custom modules

You can create your own modules, and easily include them in your application. The following example creates a module that returns a date and time object:

As an example, create a simple module that returns the current date and time, as shown below:

```
1 exports.myDateTime = function () {  
2   return Date();  
3 };
```

The `exports` keyword to make properties and methods available outside the module file. Save the code above in a file called `'myfirstmodule.js'`. Now you can include and use the module in any of your NodeJS projects.

You use your module (i.e., `myfirstmodule`) in your NodeJS project as you do with other modules, i.e., you import your module using the `require` call:

```
1 var dt = require('./myfirstmodule');  
2 console.log( "The date and time are: " + dt.myDateTime() );
```

Note you can even publish your modules online for others to install and use (see `npm publish` for details). Which is NPM is able to provide more than 800,000+ prebuilt libraries which act as great tools for the NodeJS developers and speeds up the complete application development process.

10.5 Questions

Question: What is the command that is used in NodeJS to import external libraries?

Command `require` is used for importing external libraries, for example, `var http=require (http)`. This will load the http library and the single exported object through the http variable.

Question: What is the framework most commonly used in NodeJS? Express is the most common framework used in NodeJS.

11. Building a Simple Website

You've got NodeJS installed and tested! Great! Now it's time to start building a website.

11.1 Creating the web-server

The first you'll learn, is that with NodeJS there is no web-server until you create one! That's what you'll do with the following lines of code:

```
1  /*  
2   Minimum working server example  
3  */  
4  var http = require('http');  
5  http.createServer(function (req, res) {  
6    res.writeHead(200, {'Content-Type': 'text/html'});  
7    res.write('Hello World<br>How are you?');  
8    res.end();  
9  }).listen(1234, "127.0.0.1");  
10 console.log('Server running at http://127.0.0.1:1234/');
```

11.1.1 Calling the HTTP module

- A Starting at the top, the very first thing you do is load in the pre-prepared HTTP that ships as part of NodeJS. You'll see this type of syntax a lot, as NodeJS programming uses a lot of

modules. You might want to practice giving your variable names easy to identify values (e.g., `http` is the handle for the `http` module - makes your code more readable).

```
1 var http = require('http');
```

11.1.2 Create the server

- ⓑ Stripping out the callback function to make it a little more clear, you have the `createServer` method of HTTP like this:

```
1 http.createServer(function(req, res){ }).listen(1234, '127.0.0.1');
```

What you're doing here is creating a HTTP server to listen to requests (`req`) made to port 1234 on the IP address '127.0.0.1', and also send response objects (`res`) back to the same IP and port. The IP address '127.0.0.1' is common across many systems as is known as the 'localhost' server.

You also output a message to the console to show you've reached the end of the script and that the server is running:

```
1 console.log('Server running at http://127.0.0.1:1234/');
```

11.1.3 Sending responses

- ⓒ The rest of the code is for responding to any request.

```
1 res.writeHead()
```

The first part of the response object is used to send the HTTP headers. Here you're just sending the HTTP status code and content type. So you say that the request was successful, and that you're sending back HTML content we type:

```
1 res.writeHead(200, {'Content-Type': 'text/html'});  
2 res.write()
```

Next you want to send some content back to the response object. Here you're doing it all as one string:

```
1 res.write('Hello World<br>How are you?');  
2 res.end()
```

To tell the server that you've reached the end of the response object you type:

```
1 res.end()
```

11.1.4 Run your NodeJS site!

- 10 Starting your NodeJS server is simple. Open a command prompt, and navigate to the directory you saved your script (e.g., index.js). Then type:

```
1 node index.js
```

All being well you'll see your console message output to the terminal:

```
1 Server running at http://127.0.0.1:1234/'
```

11.1.5 Open your NodeJS site in a browser

Go to your browser and type the URL 'http://127.0.0.1:1234' in the navigation bar. You should see the output:

```
1 Hello World
2 How are you?
```

You've got your web-server running in NodeJS!

Next you'll look at how to add extra details, such as, sending different information based on the URL path details.

11.2 Paths

Building upon the previous example, you'll add the extra module `url` which enables to you extract and act based on the path name. Here is the full code:

```
1  /*
2   'paths', so you can have index.html, hello.html, ...
3  */
4  var http = require("http");
5  var url = require('url');
6
7  var server = http.createServer(function(request, response){
8    var path = url.parse(request.url).pathname;
9
10   switch(path){
11     case '/':
12       response.writeHead(200, {'Content-Type': 'text/html'});
13       response.write('hello world');
14       response.end();
15       break;
16     case '/hello.html':
17       response.writeHead(200, {'Content-Type': 'text/html'});
18       response.write('you want to say hello? eh!');
```

```

19     response.end();
20     break;
21     default: // anything else, bb.html, cat.html
22     response.writeHead(404);
23     response.write("opps this does not exist - 404");
24     response.end();
25     break;
26 }
27 });
28
29 server.listen(1234);
30 console.log('Server running at http://127.0.0.1:1234/'); // - default '/'
31 console.log('Server running at http://127.0.0.1:1234/hello.html'); // - 'you want to
    ↪ say hello?...'
32 console.log('Server running at http://127.0.0.1:1234/abc.html'); // opps this doesn't
    ↪ exist - 4

```

11.2.1 Acting based on the browser path

The extra line of code that is doing all the work is:

```

1 var path = url.parse(request.url).pathname;

```

When you open your web browser and navigate to different locations, different responses will be provided depending upon the path, such as:

- A http://127.0.0.1:1234/hello.html
- B http://127.0.0.1:1234/index.html
- C http://127.0.0.1:1234/abc.html

You get the path name and response appropriately. For example, if you open the above example and navigate to 'hello.html', you'll receive the reply in your browser:

```

1 you want to say hello? eh!

```

While if you navigate to a 'abc.html', you'll receive the reply:

```

1 opps this does not exist - 404

```

11.2.2 Files

In the previous examples, you've been creating strings and sending them directly. However, as your projects grow in complexity, you'll want to start splitting up your client and server side scripts. As shown in this section, instead of creating the string, you retrieve the requested file and send it to the client.

For this example, you'll use the `fs` (or file server) module for accessing and reading files. You also need to create a `html` file called `'test.html'`. The file contents are sent to the client when they request the details.

```
1  /*
2   Files - Read external files and send them to the client
3  */
4  var http = require('http');
5  var fs = require('fs');
6  http.createServer(function (req, res) {
7    fs.readFile('test.html', function(err, data) {
8      res.writeHead(200, {'Content-Type': 'text/html'});
9      res.write(data);
10     return res.end();
11   });
12 }).listen(1234);
```

Open the web browser and navigate to:

```
1 http://127.0.0.1:1234/test.html
```

You should see the contents of your `'test.html'` file on the screen.

11.3 Summary

Wow! I'm sure you're amazed at how easy it is to setup a web server using the NodeJS framework. After this chapter, it means you can develop a web server using the `http` module. Your application can be made to listen to particular ports and send responses to clients whenever a request is made to your application. The `'url'` module was used to get extra information, enabling you to create either one web page or a complete web site (many pages, paths and directories).

11.4 Questions

Question: Can you access DOM in NodeJS? No, you cannot access DOM in node.

Question: Using the event loop what are the tasks that should be done asynchronously?

- I/O operations
- Heavy computation
- Anything requiring blocking

Question: What is an event loop in NodeJS? To process and handle external events and to convert them into callback invocations an event loop is used. So, at I/O calls, NodeJS can switch from one request to another.

Question: What are the pros and cons of NodeJS?

- *Pros: If your application does not have any CPU intensive computation, you can build it in Javascript top to bottom, even down to the database level if you use JSON storage object DB like MySQL. Crawlers receive a full-rendered HTML response, which is far more SEO friendly rather than a single page application or a websockets app run on top of NodeJS.
- *Cons: Any intensive CPU computation will block NodeJS responsiveness, so a threaded platform is a better approach. Also using relational databases with NodeJS is considered less favourable.



12. Multi-Client Drawing

12.1 Client-server communication

In the previous Chapter, your web-server was one-way. You connected to your NodeJS server, and the server sent the html/text which was displayed on screen. After that, the connection was closed and no further communication occurred. In this example, you'll continue to expand your server implementation, but you'll also add additional client-server communications.

For the examples in this Chapter, you'll make use of web-sockets and the 'socket.io' module. The socket.io module is an external resource, so you'll need to install (i.e., not part of the built-in modules).

12.2 Web-Sockets

Web-sockets provide a connection that never dies, so you can **have full-duplex communication** available to you, meaning you can push data to the client instead of having to wait for them to ask for data from the server. This allows for data to be communicated back and forth, which is ideal for things like real-time drawing applications or even games!

12.2.1 How do Web-sockets Work?

At its core, a web socket is just a TCP connection that allows for full-duplex communication, meaning either side of the connection can send data to the other, even at the same time.

As you did in the previous chapter when you built a simple web-server - this established a connection with the client and sent a reply then the connection was closed. For web sockets, the protocol actually initiates the handshake as a normal HTTP request but an additional request is made in the HTTP header component to keep the connection open (live).

All the protocol has to do to keep the connection open is send some ping/pong packets, which tells the other side that they're still there. To close the connection, a simple 'close connection' packet is sent.

12.2.2 Web-socket (socket.io)

Of the many different web-socket libraries for NodeJS available, 'socket.io' is one of the most common (i.e., it's simple and easy to use).

When you install the 'socket.io' module, you'll need to include the client socket.io script. However, you should never refer to the scripts directly in the `nodes_module` path. When you specify the external client script should be loaded for socket.io (i.e., 'socket.io.js', you only need to specify the 'socket.io' module name for the client to find the script.

```
1 <script src="/socket.io/socket.io.js"></script>
```

12.2.3 Tick-Tock

For the following example, you'll create a simple server, which receives an update every 2 seconds. Every 2 seconds the NodeJS server will send out a `message`, which your client Javascript program will be listening for.

```
1  /*
2  Client-server comms
3  Send data to the client regularly (i.e., every couple of seconds, send a date update)
   ↳ real bread and butter (use socket.io)
4  */
5
6  var http = require("http");
7  var url = require('url');
8  var io = require('socket.io'); // npm install socket.io
9
10 var server = http.createServer(function(request, response){
11   var path = url.parse(request.url).pathname;
12
13   switch(path){
14     case '/':
15       response.writeHead(200, {'Content-Type': 'text/html'});
16       response.write('hello world');
```

```

17     response.end();
18     break;
19     case '/tick.html':
20         var html = `
21             <html><head>
22                 <script src="/socket.io/socket.io.js"></script>
23             </head><body>
24                 <script>
25                     var socket = io.connect();
26                     socket.on('date', function(data){
27                         document.writeln( data.date + " " );
28                     });
29                 </script>
30                 <div>This is our socket.html file (check console.log)</div>
31             </body>
32             </html>`;
33
34     response.writeHead(200, {"Content-Type": "text/html"});
35     response.write( html, "utf8");
36     response.end();
37     break;
38     default:
39         response.writeHead(404);
40         response.write("oops this does not exist - 404");
41         response.end();
42         break;
43     }
44 });
45 server.listen(1234);
46
47 var listener = io.listen(server); // * important part
48 listener.sockets.on('connection', function(socket){
49
50     socket.emit('message', {'message': 'hello world'});
51
52     //send data to client
53     setInterval(function(){
54         socket.emit('date', {'date': new Date() });
55     }, 2000); // send message every 2 seconds
56
57 });

```

12.2.3.1 Establishing the Connection

In order for a connection to be established between the client and server, the server must:

- Hook in to the HTTP server to handle web-socket connections
- Serve up the 'socket.io.js' client library as a static resource

In the code above, you can see item first step being done in . The second step is done for you (by default) by the socket.io library in  and is served on the path '/socket.io/socket.io.js'. Just to note, by default, all web-socket connections and resources are served within the '/socket.io' path.

12.2.3.2 Sending Data from Server to Client

Okay, now on to some of the more interesting parts. In this example you'll be sending data from the server to the client. In this case, you'll be sending a 'tick' every couple of seconds, which you'll subscribed to and received and display in the client. For instance, your client application might be listening for lots of different types of messages which have a specific meaning. As you'll see in later examples, once you're comfortable with the underlying principle of sending and receiving data - it's very easy to expand and develop power inter-connected solutions.

On the server this is done by waiting for the new connection to be established (i.e., ) , then by calling the 'socket.emit()' method to send a message to all connected clients ().

12.2.3.3 Sending Data from Client to Server

Just to be complete, you might be wondering what you do when you want to send data the other way, from client to server? It is very similar to how you send data from the server to the client. Both the client and the server use the 'socket.emit()' and 'socket.on()' methods, as shown below:

```
1 <script src="/socket.io/socket.io.js"></script>
2 <script>
3   var socket = io.connect('/');
4   socket.emit('event', { message: 'hello server!' });
5 </script>
```

12.2.3.4 Testing your Program

Open the web browser and navigate to:

```
1 http://127.0.0.1:1234/tick.html
```

Through the 'socket.io' module you're able to perform real-time, bidirectional event-based communication between the browser and the server.

Server

1. Install 'socket.io' module (e.g., 'npm install socket.io')
2. Add 'var io = require('socket.io');' to your server script
3. Connect and listen for incoming data, 'var listener = io.listen(server);'
4. Send data back to clients using 'socket.emit(..)'

Client

5. Add header '<script src="/socket.io/socket.io.js"></script>'
6. Connect to server receiving/sending 'var socket = io.connect();'

12.2.3.5 Important Parts

The important functions that you should take notice of with the ‘socket.io’ module for sending data are:

- **Server-Side**

```
1 // sending to the client
2 socket.emit('hello', 'can you hear me?', 1, 2, 'abc');
3
4 // sending to all clients except sender
5 socket.broadcast.emit('broadcast', 'hello friends!');
```

- **Client-Side**

```
1 // sending to the server
2 socket.emit(..)
```

When you send a message (either to the client or the server) you catch and manage the received messages through **events** (i.e., call back event handlers).

You structure your code to handle different scenarios depending how you send your data, for example, if you want to send data to a specific individual (socket) or everyone (all sockets).

Scenario 1: use of ‘io.sockets.emit(..)’ means every socket gets the Message including Initiator.

```
1 // io>socket>emit
2 io.sockets.emit('MyChannelBroadcast',
3 {
4   owner:"test",
5   message:"Welcome All"
6 })
7 );
```

Scenario 2: use of ‘socket.broadcast.emit(..)’ means every socket gets the message except one i.e Initiator.

```
1 // socket>broadcast>emit
2 socket.broadcast.emit('BroadCastExceptMe',
3 { data:"Hello"}
4 );
```

12.3 Sending Mouse Information

The tick demo example was very simple and demonstrated how to setup a simple ‘socket.io’ client-server communication implementation. In the next example you will extend the previous implementation, but you will send mouse information as well (i.e., as you move the mouse around the client window the mouse coordinates are sent to the server and relayed to all the clients).

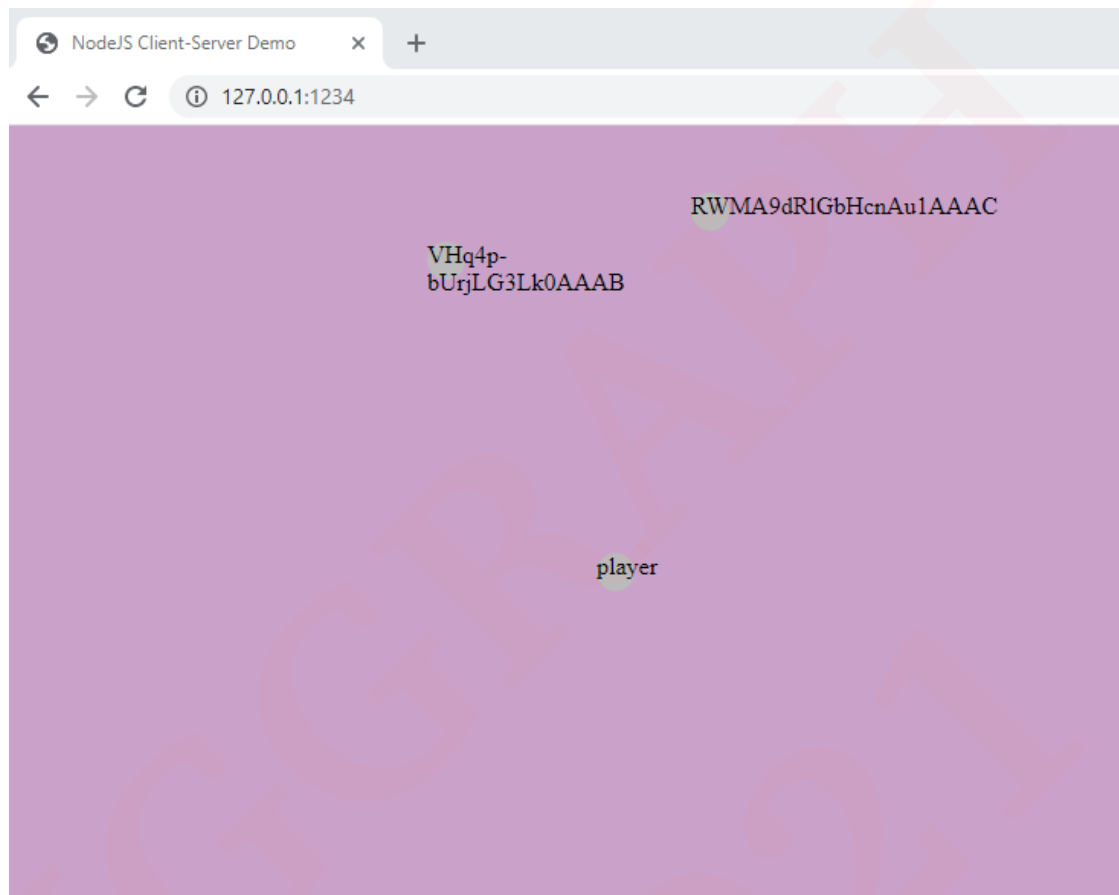


Figure 12.1: *Mouse Coordinates* - Screenshot of the mouse program. Displays the unique socket ID and draws a gray dot at the location of the received clients mouse position.

Listing 12.1: Complete implementation for the showing and sharing mouse positions between multiple clients. Screenshot of the running implementation is shown in Figure 12.1.

```

1  /*
2   Send/receive mouse positions real-time (see each others mouse)
3  */
4
5  let htmlcontent = `
6  <!DOCTYPE html>
7  <html>
8  <head>
9    <meta charset="utf-8" />
10   <title>NodeJS Client-Server Demo</title>
11
12   <style type="text/css">
13     body {
14       background: #caa2c9;
15       overflow: hidden;
16     }
17
18     .player {
19       position: absolute;
20       height: 25px;
21       width: 25px;

```

```

22     background-color: #bbb;
23     border-radius: 50%;
24     display: inline-block;
25 }
26
27 </style>
28 <script src="/socket.io/socket.io.js"></script>
29 </head>
30
31 <body>
32 <script>
33     window.onload = function() {
34         const url = window.location.origin || "http://localhost:1234";
35         let socket = io.connect(url);
36
37         let clients = {};
38         let divs = {};
39
40         let div = document.createElement('div');
41         div.id = 'player';
42         div.className = 'player';
43         div.innerHTML = 'player';
44         document.body.appendChild( div );
45         divs['player'] = div;
46
47         function now() {
48             return new Date().getTime();
49         }
50
51         let lastEmit = now();
52
53         window.onmousemove = function(e) {
54             // limit how often we send mouse movements
55             if (now() - lastEmit > 10) {
56                 socket.emit("mousemove", {
57                     x: e.pageX,
58                     y: e.pageY
59                 });
60                 lastEmit = now();
61
62                 divs[ 'player' ].style.left = e.pageX + 'px';
63                 divs[ 'player' ].style.top = e.pageY + 'px';
64             }
65         };
66
67         socket.on("moving", function(data) {
68             if (!clients.hasOwnProperty(data.id)) {
69                 let div = document.createElement('div');
70                 div.id = data.id;
71                 div.className = 'player';
72                 document.body.appendChild( div );
73                 divs[data.id] = div;
74             }
75
76             divs[data.id].style.left = data.x + "px";
77             divs[data.id].style.top = data.y + "px";
78             divs[data.id].innerHTML = data.id;
79
80             clients[data.id] = data;
81             clients[data.id].updated = now();
82         });
83
84         socket.on("clientdisconnect", function(id) {
85             clients[id].div.parentNode.removeChild( clients[id].div );
86             delete clients[id];
87         });
88     };
89

```

```

90 </script>
91 </body>
92 </html>
93 `;
94
95 var http = require("http");
96
97 var app = http.createServer(function (req, res) {
98   res.write(htmlcontent); //write a response to the client
99   res.end(); //end the response
100 });
101 console.log('server running ...');
102
103 const io = require("socket.io").listen(app, { log: false });
104 app.listen(1234);
105
106 io.sockets.on("connection", (socket) => {
107   let id = socket.id;
108
109   socket.on("mousemove", (data) => {
110     data.id = id;
111     socket.broadcast.emit("moving", data);
112   });
113
114   socket.on("disconnect", () => {
115     socket.broadcast.emit("clientdisconnect", id);
116   });
117 });

```

12.4 CANVAS (Drawing)

As you can see from the previous example, you can send and receive mouse information (including button presses). You take this further, by adding a **CANVAS** object to your client HTML. Then when the mouse button is down, you relay this to the other clients, so they all **draw** information to the canvas as the same time. Every person can draw on the canvas and also see what other people are drawing.

Listing 12.2: Complete implementation for the multiple-client drawing program - after connecting to the server - as you draw on the client window using the mouse cursor - the mouse information is relayed to all the other clients - so all clients see the same drawing (real-time). A screenshot is shown in Figure 12.2.

```

1  /*
2   Send drawing information to other clients
3   (press mouse down an draw, others can see the drawing)
4  */
5
6  let htmlcontent = `
7  <!DOCTYPE html>
8  <html>
9  <head>
10   <meta charset="utf-8" />
11   <title>NodeJS Client-Server Drawing Demo</title>
12
13   <style type="text/css">
14     body {
15       background: #eeeeee;
16       overflow: hidden;
17     }
18
19   </style>

```

```

20 <script src="/socket.io/socket.io.js"></script>
21 </head>
22 <body>
23
24 <canvas id="mycanvas" width="1000" height="1000"></canvas>
25
26 <script>
27 window.onload = function() {
28   const url = window.location.origin || "http://localhost:1234";
29   let socket = io.connect(url);
30
31   let mouseDown = false;
32   let points = [];
33
34   window.onmousedown = function (e){
35     mouseDown = true;
36   }
37
38   window.onmouseup = function(e){
39     // sent here, only when the user has finished
40     // drawing and releases the mouse button
41     socket.emit("drawpoints",
42       {'points':points} );
43     mouseDown = false;
44     points = [];
45   }
46
47   window.onmousemove = function(e) {
48     if ( mouseDown )
49     {
50       points.push( {x:e.pageX, y:e.pageY} );
51
52       // send the data iteratively so you see the
53       // drawing in real-time
54       if ( points.length % 10 ){
55         socket.emit("drawpoints",
56           {'points':points} );
57       }
58     }
59   }
60
61   socket.on("drawpoints", function(data) {
62     if ( data.points.length <= 0 ) return;
63
64     var canvas = document.getElementById('mycanvas');
65     var context = canvas.getContext('2d');
66     // Reset the current path
67     context.beginPath();
68     context.moveTo(data.points[0].x,data.points[0].y);
69     for (let i=1; i<data.points.length; i++)
70     {
71       context.lineTo(data.points[i].x,data.points[i].y);
72     }
73     // Make the line visible
74     context.stroke();
75   });
76
77   socket.on("clientdisconnect", function(id) {
78     console.log('lost someone');
79   });
80 };
81
82 </script>
83 </body>
84 </html>
85 `;
86
87 var http = require("http");

```

```

88
89 var app = http.createServer(function (req, res) {
90   res.write(htmlcontent); //write a response to the client
91   res.end(); //end the response
92 });
93 console.log('server running..');
94
95 const io = require("socket.io").listen(app, { log: false });
96 app.listen(1234);
97
98 io.sockets.on("connection", (socket) => {
99   let id = socket.id;
100
101   socket.on("drawpoints", (data) => {
102     data.id = id;
103     socket.emit("drawpoints", data);
104     socket.broadcast.emit("drawpoints", data);
105   });
106
107   socket.on("disconnect", () => {
108     socket.broadcast.emit("clientdisconnect", id);
109   });
110 });

```

The canvas drawing implementation simply collects the mouse positions in an array called `points` when the user presses the mouse button down. Once the mouse button is released, the positions are sent to all the other connected clients. To make the demo more responsive (i.e., see the drawing as the mouse is moved), an extra couple of lines are added, so that every 10 points the drawing data is sent to all clients.

All drawing currently goes through the server (even the client itself sends the information which is relayed back to the receiving socket call function which does all the drawing).

12.5 Further Ideas

You should continue to develop the simple example, adding more features to help you master inter-client communication. Some ideas include:

- All more functionality and options (login/logout)
- Have the different clients select a draw features (e.g., the pen-color/pen-size)
- Option to clear the draw canvas - Also a clear background to a specific color (don't forget this information should be transmitted to other clients)
- Display statistics (number of points/lines, number of users, delay/ping times)

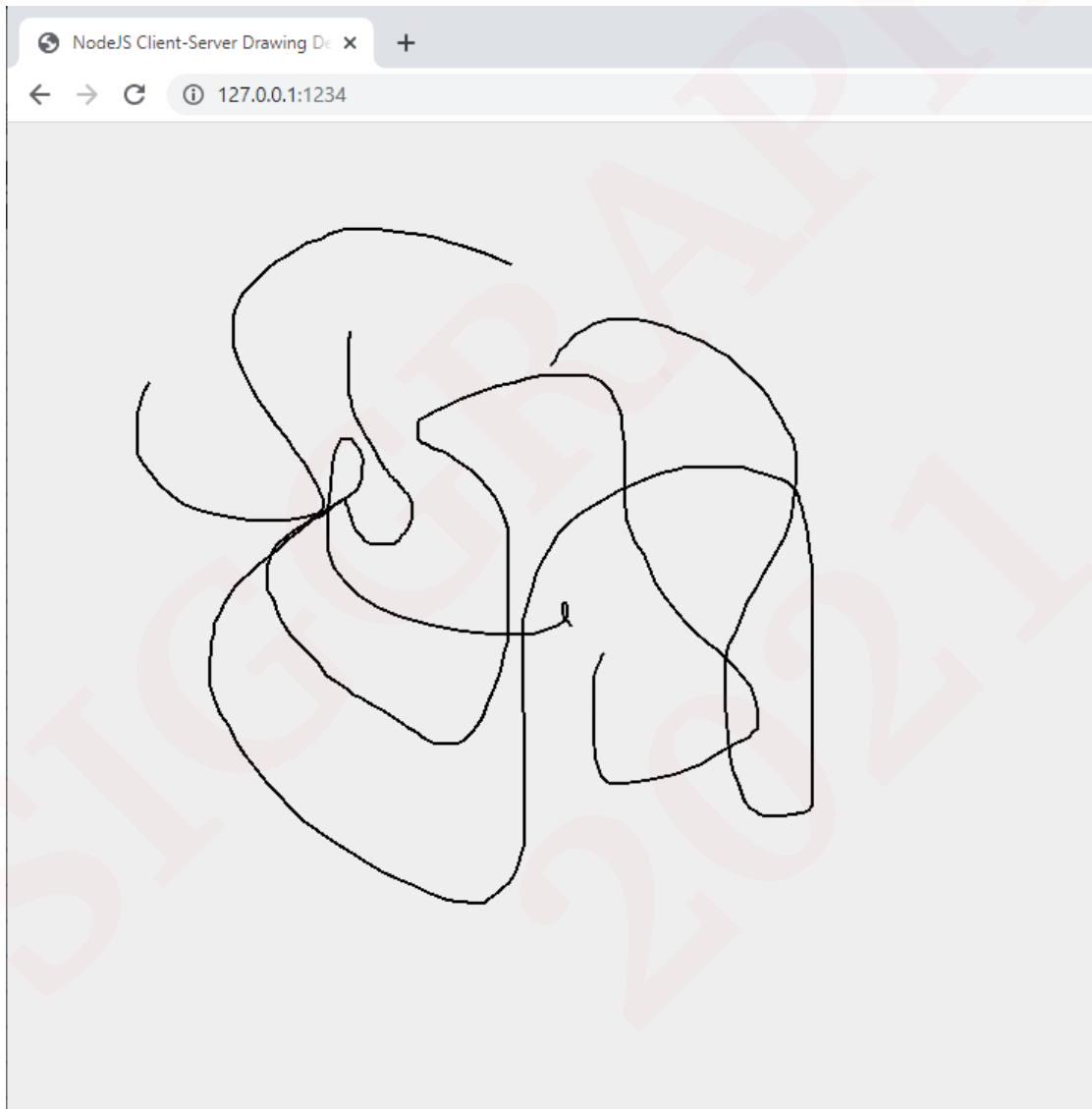


Figure 12.2: Multiple Client Drawing Program - Screenshot of the browser window showing the result of multiple clients drawing. As each client connects to the server, as they draw on the screen using the mouse, the drawing information is relayed to all the other clients.

13. Chat-Client

13.1 Sending Key-Presses

To warm up to the idea of a chat program, a minimal example that sends key-press information to the server and relays this to other connected clients is shown below.

You'll need to install 'socket.io' module for this chapter (as the examples build upon previous chapters and continue to use the 'socket.io' module for inter-client communication).

Listing 13.1: Key Presses Relay - This self contained example, shows you how to relay key presses to all the other clients and forms the foundation of the chat-client system.

```
1  /*
2   Sending Data From the Client To the Server:
3   */
4  var http = require("http");
5  var url = require('url');
6  var io = require('socket.io'); // npm install socket.io
7
8  var server = http.createServer(function(request, response){
9    var path = url.parse(request.url).pathname;
10
11    switch(path){
12    case '/':
13      response.writeHead(200, {'Content-Type': 'text/html'});
14      response.write('hello world');
15      response.end();
16      break;
17    case '/socket.html':
18      var html = `
```

```

19 <html>
20 <head>
21   <script src="/socket.io/socket.io.js"></script>
22 </head>
23 <body>
24   <script>
25     var socket = io.connect();
26     document.onkeypress = function(e){
27       socket.emit("client_data", { "letter": String.fromCharCode(e.charCode) } );
28     }
29   </script>
30   <div>This is our socket.html file (check console.log)</div>
31 </body>
32 </html> `;
33
34 response.writeHead(200, {"Content-Type": "text/html"});
35 response.write( html, "utf8");
36 response.end();
37 break;
38 default:
39   response.writeHead(404);
40   response.write("oops this doesn't exist - 404");
41   response.end();
42   break;
43 }
44 });
45 server.listen(1234);
46
47 var listener = io.listen(server);
48 listener.sockets.on('connection', function(socket){
49   //receive client data
50   socket.on('client_data', function(data){
51     process.stdout.write('received:>' + data.letter + '<' );
52   }
53   // Send the key press details to other clients
54   socket.broadcast.emit("key", data);
55 });
56 });

```

-  - on the client side in Javascript you setup the client to relay events using the 'emit' to the server
-  - each time the client receives a key press signal event (i.e., 'client_data' it prints it to the console window for debugging
-  - one extra line to relay the key press data to other clients

The example is very similar to the canvas drawing example in the previous chapter, however, this time you're sending key-presses instead of mouse positions. To make the example a little more different, you'll also incorporate a simple 'login screen' before you enter the chat session.

13.2 Chat Demo

This example, uses the `express` module (which you need to install) as well as 'socket.io'. Again, you'll continue to expands on the previous key relay example, but instead of single key-presses, you'll send complete sentences. Also you'll add login screen, so you can't begin to chat/communicate until you've logged in. For the login screen, the only check is that the user must enter a

unique name which the server checks and then allows you to connect to the chat room.

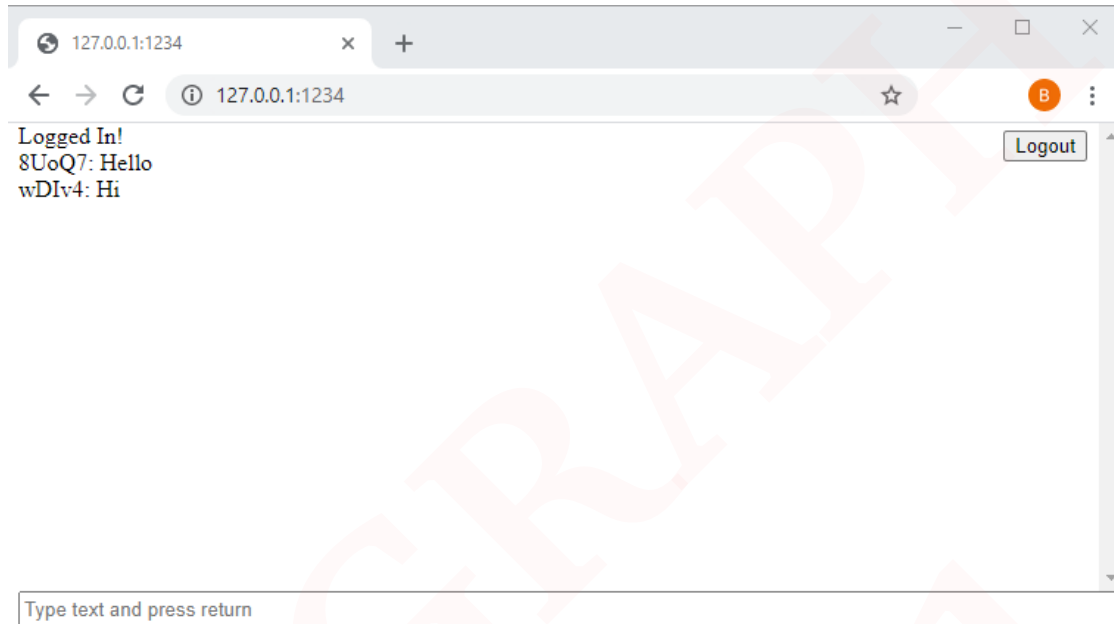


Figure 13.1: **Chat Login Screen** - Screenshot of the chat login screen (have to enter a unique name to enter the chat room).

Listing 13.2: Complete server-client implementation. Screenshot of the program running in Figure 13.1 and Figure 13.2.

```

1  /*
2   Simple login and chat
3   Need to provide a username before you can send/see chat messages
4   Stores username locally on server
5  */
6
7  // html that is sent to the server - put here as a single string
8  let htmlContent = `
9  <div id="signDiv">
10   Username: <input id="username" type="text"></input><br>
11   <button id="signIn">Sign In</button>
12 </div>
13
14 <div id="gameDiv" style="display:none;">
15   <div id="chat-text"
16     ↳ style="position:absolute;top:0px;bottom:25px;width:99%;overflow-y:scroll;">
17     <div>Logged In!</div>
18   </div>
19   <form id="chat-form">
20     <input id="chat-input" type="text"
21       ↳ style="position:absolute;bottom:0px;height:25px;width:99%;"></input>
22     </form>
23     <button style='position:absolute;right:25px;top:5px;'
24       ↳ onClick="window.location.reload();">Logout</button>
25   </div>
26 <script src="/socket.io/socket.io.js"></script>
27 <script>
28   var socket = io();

```

```

29
30 //sign
31 var signDivUsername = document.getElementById('username');
32 var signDivSignIn = document.getElementById('signIn');
33
34 signDivSignIn.onclick = function(){
35     socket.emit('signIn',{username:signDivUsername.value});
36 }
37 socket.on('signInResponse',function(data){
38     if(data.success){
39         signDiv.style.display = 'none';
40         gameDiv.style.display = 'inline-block';
41     } else
42         alert("Sign in unsuccessful/username taken?");
43 });
44
45 //chat - only be used after signing in
46 var chatText = document.getElementById('chat-text');
47 var chatInput = document.getElementById('chat-input');
48 var chatForm = document.getElementById('chat-form');
49 chatInput.placeholder = 'Type text and press return';
50
51
52 socket.on('addToChat',function(data){
53     chatText.innerHTML += '<div>' + data + '</div>';
54     chatText.scrollTop = chatText.scrollHeight;
55 });
56
57 chatForm.onsubmit = function(e){
58     e.preventDefault();
59     socket.emit('sendMsgToServer',chatInput.value);
60     chatInput.value = '';
61 }
62 </script>
63 `;
64
65 // Server
66 var express = require("express");
67 var app = express();
68 var serv = require("http").Server(app);
69
70 app.get("/", function (req, res) {
71     res.send(htmlContent);
72 });
73
74 serv.listen(1234);
75 console.log("Server started.");
76
77 var usernames = [];
78 var SOCKET_LIST = {};
79
80 var isUsernameTaken = function (data, cb) {
81     if ( data.username in usernames )
82         { cb(false); }
83     else
84         { cb(true); }
85 }
86
87 var io = require("socket.io")(serv, {});
88 io.sockets.on("connection", function (socket) {
89     SOCKET_LIST[socket.id] = socket;
90
91     socket.on("signIn", function (data) {
92         isUsernameTaken(data, function (res) {
93             if (res) {
94                 usernames.push( data.username );
95                 socket.emit("signInResponse", { success: true });
96             } else {

```

```

97     socket.emit("signInResponse", { success: false });
98   }
99   });
100  });
101
102  socket.on("sendMsgToServer", function (data) {
103    var playerName = "" + socket.id.slice(2, 7);
104    for (var i in SOCKET_LIST) {
105      SOCKET_LIST[i].emit("addToChat", playerName + ": " + data);
106    }
107  });
108
109  socket.on("disconnect", function () {
110    delete SOCKET_LIST[socket.id];
111  });
112
113  });

```

-  Login message checks if the username is already taken (checks list of usernames stored)
-  Relays chat messages to all the clients

After you've setup your project (including installing the relevant modules you're ready to run). Type the above script into a file (e.g., `server.js`). Open the command prompt and navigate to the folder and launch the script (e.g., '`>node server.js`'). Then open a browser and type:

```
1 http://127.0.0.1:1234/
```

Open multiple browser windows using the local url and test inter-client communication via the server.

The name displayed in the chat window uses the socket `id` to provide a unique identifier, however, it should be easier enough for you to modify this to use the entered username (possibly add other information and visual aesthetics, such as date, time and different colors).

13.3 Summary

In this chapter you developed a server-client interaction solution using the 'socket.io' and 'express' modules. While you could do without the `express` framework, it has gained a lot of popularity as it's built on top of the NodeJS framework and helps with fast-track development of server-based applications, such as, routing different parts of the web applications based on the request made. As you have seen in the examples, the response for each route can be varied depending on what needs to be shown to the user.

13.4 Further Ideas

Keep enhancing and improving the chat system, some examples of things you might want to add:

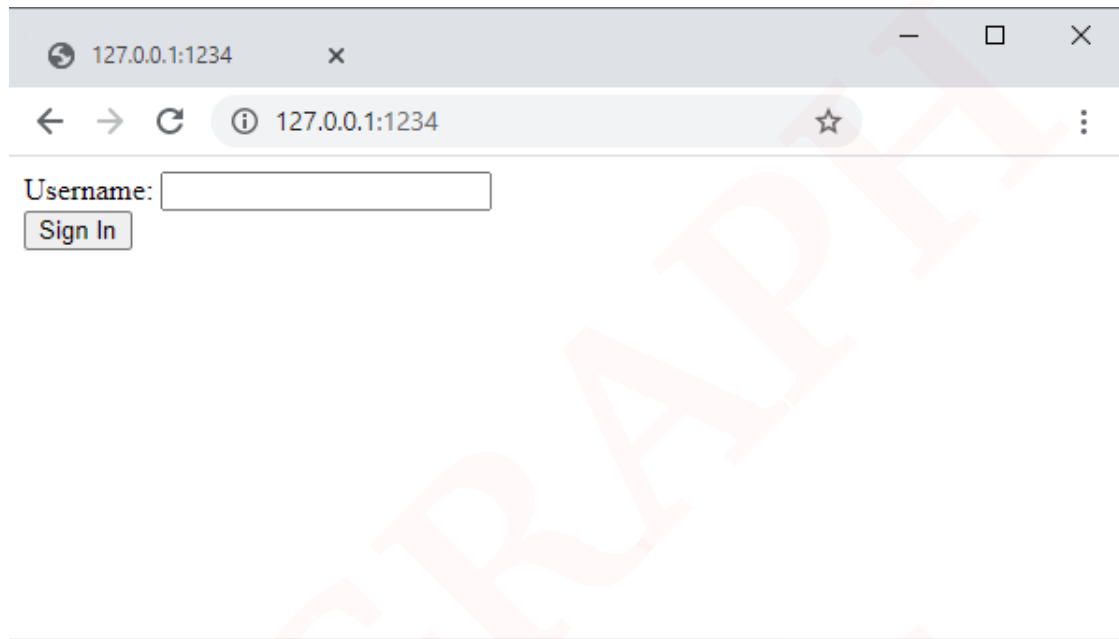


Figure 13.2: **Client Chat** - You can see messages relayed between the multiple clients. The chat also displays the unique socket ID next to the client that typed the message.

- Improve the interface - Custom backgrounds - Fonts - Time/date information to the chat details
- Add a log to the server (stores previous chats)
- Create channels so you can only chat with people in the same channel (also only receive messages)
- Allow people to register their username/password
- Add 'emojis' - certain words/phrases create graphics in the chat client
- Check for time-out clients (add occasional 'ping' to see if the client is still alive and there)
- Show statistics, such as, how many people are online (or been online in the past 24 hours), how many words have been typed
- Block certain words (add censoring)
- Create a 'bot', so the NodeJS server can chat with people autonomously
- Allow admin (block some users/IP addresses)
- Allow user to change/set their username

14. Tic-Tac-Toe

Tic-Tac-Toe (or Noughts and Crosses)

You should be getting comfortable with sending and receiving information to and from the server and client. In this Chapter, you'll add a simple game called **0s and Xs**. A 3x3 grid is created and each person takes a turn adding a '0' or a 'X'. The first player to get three in a row (horizontally, vertically or diagonally wins). For this example, you'll need to add more logic to the server side system to keep track of who's turn it is and if one of the players has won.

As this program is a bit long, it's split into two files. The client side script/html (which is in a file called client.html and the server side Javascript called server.js).

Join Game:

*Figure 14.1: **Join Game** - Join game message when you initially connect to the server in the browser. You need to type the name of the game you wan to join.*

Client side script 'client.html'

Listing 14.1: Tic-Tac-Toe client-script (client.html).

```
1  /*  
2  Tic Tac Toe - XOX
```

```

3  (demo uses red and green instead of the X and O symbol)
4  Minimal working node game demo - to show basic working
5  principles and concepts
6
7  The implementation on is very flexible, while the 'default'
8  follows the basic 2 player with a 3x3 grid. It's easy to
9  change the grid size to any size and the code will
10 automatically check for all the rows set to the same type.
11 Also, it's easy to expand to more than 2 players.
12 Currently uses red-green but you could add more colors
13 so 3, 4 or more players can join.
14 */
15
16 <html>
17 <head>
18 <title>Tic Tac Toe (XOX)</title>
19
20 <script src="/socket.io/socket.io.js"></script>
21
22 <script>
23   var socket = io();
24 </script>
25
26 <style>
27 body{
28   margin:0;
29   padding:0;
30 }
31
32 /* game will be within this area */
33 #gameArea {
34   position : relative;
35   width : 300;
36   height : 300;
37   background-color: #dddddd;
38   margin : 20px;
39 }
40
41 /* place the game area inside container so it's centered */
42 #gameContainer {
43   position : absolute;
44   top : 50%;
45   left : 50%;
46   transform : translate(-50%, -50%);
47   text-align : center;
48   display : none;
49 }
50 </style>
51
52 </head>
53 <body>
54
55
56 <!--/*
57 Simple join game menu
58 Need to provide a game name, before you can play
59 tic-tac-toe. Only 2 people can play a single game.
60 */-->
61 <div id="signDiv">
62   Joint Game: <input id="gamename" type="text"></input><br>
63   <button id="signIn">Sign In</button>
64 </div>
65
66 <!--/*
67 Game screen
68 */-->
69 <div id="gameContainer">
70   Click Square <span id="playersturn"></span>

```

```

71 <div id="gameArea"></div>
72 Make a full horizontal, vertical or diagonal row the same to 'win'.
73 <button style='position:absolute;right:5px;top:5px;'
    ↪ onClick="window.location.reload();">Logout</button>
74 </div>
75
76
77
78 <script type="text/javascript">
79
80 let fieldSize = 3; // number of tiles per row/column
81 let tileSize = 200; // tile size, in pixels
82 let tileTypes = 2; // different kind of tiles allowed
83 let tileArray = []; // array with all game tiles
84
85 let tileColors = ['white', 'red', 'green' ];
86
87 let gameArea = document.getElementById( 'gameArea' );
88 gameArea.style.width = fieldSize*tileSize+fieldSize+1;
89 gameArea.style.height = fieldSize*tileSize+fieldSize+1;
90
91 let playerTurn = 0;
92
93 // create a tile (specify the position in the grid where
94 // the tile will be put)
95 function createTile(i, j)
96 {
97     //let randomTile = Math.floor(Math.random()*tileTypes)
98     let theTile = document.createElement('div');
99     theTile.style.position = "absolute";
100     theTile.style.width = tileSize;
101     theTile.style.height = tileSize;
102     theTile.style.left = (i*tileSize + 1*(i+1)) + "px";
103     theTile.style.top = (j*tileSize + 1*(j+1)) + "px";
104     theTile.style.padding = 0;
105     theTile.style.margin = 0;
106     theTile.style.backgroundColor = tileColors[ 0 ];
107     // set an event for the mouse here - if any tile is picked
108     // with the mouse, the pickTile(..) function will be called
109     theTile.onclick = function()
110     {
111         socket.emit('sendMsgToServer', { 'cmd':'pickTile',
112             'i':i, 'j':j,
113             'playerId':socket.playerId } );
114     }
115     theTile.frame = -1;
116     gameArea.appendChild( theTile );
117     return theTile;
118 }
119
120 // when the game is created, generate the tiles
121 function startGame(){
122     // create the `square` grid ( fieldSize x fieldSize )
123     // tiles are created and stored in an array 'tileArray'
124     for(let i=0;i<fieldSize;i++){
125         tileArray[i]=[];
126         for(let j=0;j<fieldSize;j++){
127             tileArray[i][j]=createTile(i, j);
128         }
129     }
130
131     updateStatusText();
132 }
133
134 function resetGame() {
135     for(let i=0;i<fieldSize;i++){
136         for(let j=0;j<fieldSize;j++){
137             tileArray[i][j].style.backgroundColor = tileColors[0];

```

```

138     tileArray[i][j].frame = -1;
139   }
140 }
141 }
142
143 socket.on('pickTile',function(data){
144   console.log('recv pickTile', data);
145   if ( data.playerId == playerTurn )
146   {
147     pickTile( data.i, data.j );
148   }
149   else
150   {
151     // only show the message to the player
152     if ( data.playerId == socket.playerId )
153     {
154       alert("It is not your turn yet");
155     }
156   }
157 });
158
159 // tile has been picked
160 function pickTile(i, j){
161   //console.log('row:'+i+',col: '+ j);
162   if ( tileArray[i][j].frame === -1 )
163   {
164     let c = ( playerTurn + 1 );
165     tileArray[i][j].frame = playerTurn;
166     tileArray[i][j].style.backgroundColor = tileColors[ c ];
167
168     playerTurn = (playerTurn + 1) % 2;
169   }
170   updateStatusText();
171
172   let won = -1;
173   if ( checkWinner(0) ) won = 0;
174   if ( checkWinner(1) ) won = 1;
175
176   if ( won != -1 )
177   {
178     alert('Congratulations
179           Player ' + won + ' has won.
180           Play again?');
181     resetGame();
182   }
183 }
184
185 function updateStatusText()
186 {
187   let elm = document.getElementById('playersturn');
188   elm.innerHTML = "<b>(Player " + playerTurn + " Turn)</b>";
189   elm.style.color = tileColors[ playerTurn+1 ];
190 }
191
192 function checkWinner(playerId)
193 {
194   /*
195   [0,0] [1,0] [2,0] [3,0]
196   [0,1] [1,1] [2,1] [3,1]
197   [0,2] [1,2] [2,2] [3,2]
198   [0,3] [1,3] [2,3] [3,3]
199   */
200
201   // horizontal
202   for(let j=0;j<fieldSize;j++)
203   {
204     let m = 0;
205     for(let i=0;i<fieldSize;i++)

```

```

206     {
207         let v = tileArray[i][j].frame;
208         if ( v == playerId ) m++
209         if ( m == fieldSize) return true;
210     }
211 }
212
213 // vertical
214 for(let i=0;i<fieldSize;i++)
215 {
216     let m = 0;
217     for(let j=0;j<fieldSize;j++)
218     {
219         let v = tileArray[i][j].frame;
220         if ( v == playerId ) m++
221         if ( m == fieldSize) return true;
222     }
223 }
224
225 // diagonals
226 let d = 0;
227 let s = 0;
228 for(let i=0;i<fieldSize;i++)
229 for(let j=0;j<fieldSize;j++)
230 {
231     let v = tileArray[i][j].frame;
232     if ( i==j )
233     {
234         if ( v == playerId ) d++
235     }
236     if ( d == fieldSize) return true;
237
238     if ( i == fieldSize-1-j )
239     {
240         if ( v == playerId ) s++
241     }
242     if ( s == fieldSize) return true;
243 }
244 }
245 </script>
246
247 <script>
248
249 // Sign-in/Join a game
250 var signDivGamename = document.getElementById('gamename');
251 var signDivSignIn = document.getElementById('signIn');
252
253 signDivSignIn.onclick = function(){
254     socket.emit('signIn',{'gamename':signDivGamename.value});
255 }
256 socket.on('signInResponse',function(data){
257     if(data.success){
258         //console.log('signInResponse:', data);
259         signDiv.style.display = 'none';
260         gameContainer.style.display = 'inline-block';
261         socket.playerId = data.playerId;
262         // Launch the game
263         startGame();
264     } else
265         alert("Sign in unsuccessful/max players");
266 });
267
268 socket.on('joinMessage',function(data){
269     alert( data.msg );
270     // if you don't have enough players, you can't continue
271     // so go back to the login screen
272     if ( data.playerleft )
273         window.location.reload();

```

```

274 });
275
276 </script>
277
278 </body>
279 </html>

```

Server side script ‘server.js’

Listing 14.2: Tic-Tac-Toe server-script (server.js). (Screenshot of the game running in the client is shown in Figure 14.2.)

```

1  /*
2  Tic-tac-toe multi-player example using NodeJS
3  (player over the internet)
4
5  http://127.0.0.1:1234/client.html
6
7  Features
8  1. only 2 people can join and play the game
9  2. only 1 person can move at a time
10 3. if one person leaves the game ends
11 4. after game ends (winner/lose) restarts
12 */
13
14 // Server
15 var express = require("express");
16 var app = express();
17 var serv = require("http").Server(app);
18
19 app.get("/", function (req, res) {
20   res.sendFile(__dirname + '/client.html');
21 });
22
23 serv.listen(1234);
24 console.log("Server started.");
25
26 var gamelist = {};
27 var SOCKET_LIST = {};
28
29 var isGameAvailable = function (data, cb) {
30
31   if ( ! (data.gamename in gamelist) )
32   {
33     return cb(true);
34   }
35   if ( gamelist[ data.gamename ].player0 == null ||
36       gamelist[ data.gamename ].player1 == null )
37   {
38     return cb(true);
39   }
40   return cb(false);
41 }
42
43 var io = require("socket.io")(serv, {});
44 io.sockets.on("connection", function (socket) {
45   SOCKET_LIST[socket.id] = socket;
46   //
47   socket.on("signIn", function (data) {
48     isGameAvailable(data, function (res) {
49       if (res) {
50         if ( ! (data.gamename in gamelist) )
51         {
52           gamelist[ data.gamename ] = {};
53           gamelist[ data.gamename ].player0 = null;
54           gamelist[ data.gamename ].player1 = null;

```

```

55     }
56
57     let playerId = 0;
58     if ( gamelist[ data.gamename ].player0 == null )
59     {
60         gamelist[ data.gamename ].player0 = socket.id;
61     }
62     else
63     {
64         playerId = 1;
65         gamelist[ data.gamename ].player1 = socket.id;
66     }
67
68     SOCKET_LIST[socket.id].gamename = data.gamename;
69     socket.emit("signInResponse", { success: true, 'playerId':playerId });
70     } else {
71     socket.emit("signInResponse", { success: false });
72     }
73 });
74 });
75
76 socket.on("sendMsgToServer", function (data) {
77     data.id = socket.id;
78     console.log('sending pickTile');
79
80     let gamename = SOCKET_LIST[ socket.id ].gamename;
81     if ( gamename == undefined ) return;
82
83     // require 2 people to have joined this game before
84     // you can start playing
85     if ( gamelist[ gamename ].player0 != null &&
86         gamelist[ gamename ].player1 != null )
87     {
88         // only send messages to the people in the game
89         let id0 = gamelist[ gamename ].player0;
90         let id1 = gamelist[ gamename ].player1;
91
92         SOCKET_LIST[id0].emit("pickTile", data);
93         SOCKET_LIST[id1].emit("pickTile", data);
94     }
95     else
96     {
97         // try and start playing without sufficient players
98         // send a message
99         socket.emit('joinMessage', { 'msg': 'Not enough players joined yet' } );
100     }
101 });
102
103 socket.on("disconnect", function () {
104
105     let gamename = SOCKET_LIST[ socket.id ].gamename;
106     console.log('gamename:', gamename );
107
108     if ( gamename == undefined ) return;
109
110     if( gamelist[ gamename ].player0 == socket.id )
111         gamelist[ gamename ].player0 = null;
112
113     if( gamelist[ gamename ].player1 == socket.id )
114         gamelist[ gamename ].player1 = null;
115
116     if ( gamelist[ gamename ].player0 == null &&
117         gamelist[ gamename ].player1 == null )
118     {
119         delete gamelist[ gamename ];
120     }
121     else
122     {

```

```

123   let ids = [ gamelist[ gamename ].player0,
124               gamelist[ gamename ].player1 ];
125   for (let i=0; i<ids.length; i++)
126   {
127       let id = ids[i];
128       if ( id != null )
129       {
130           SOCKET_LIST[id].emit('joinMessage',
131                               { 'msg': 'Player left game',
132                                 'playerleft': true } );
132       }
133   }
134 }
135 }
136
137 delete SOCKET_LIST[socket.id];
138 });
139 });

```

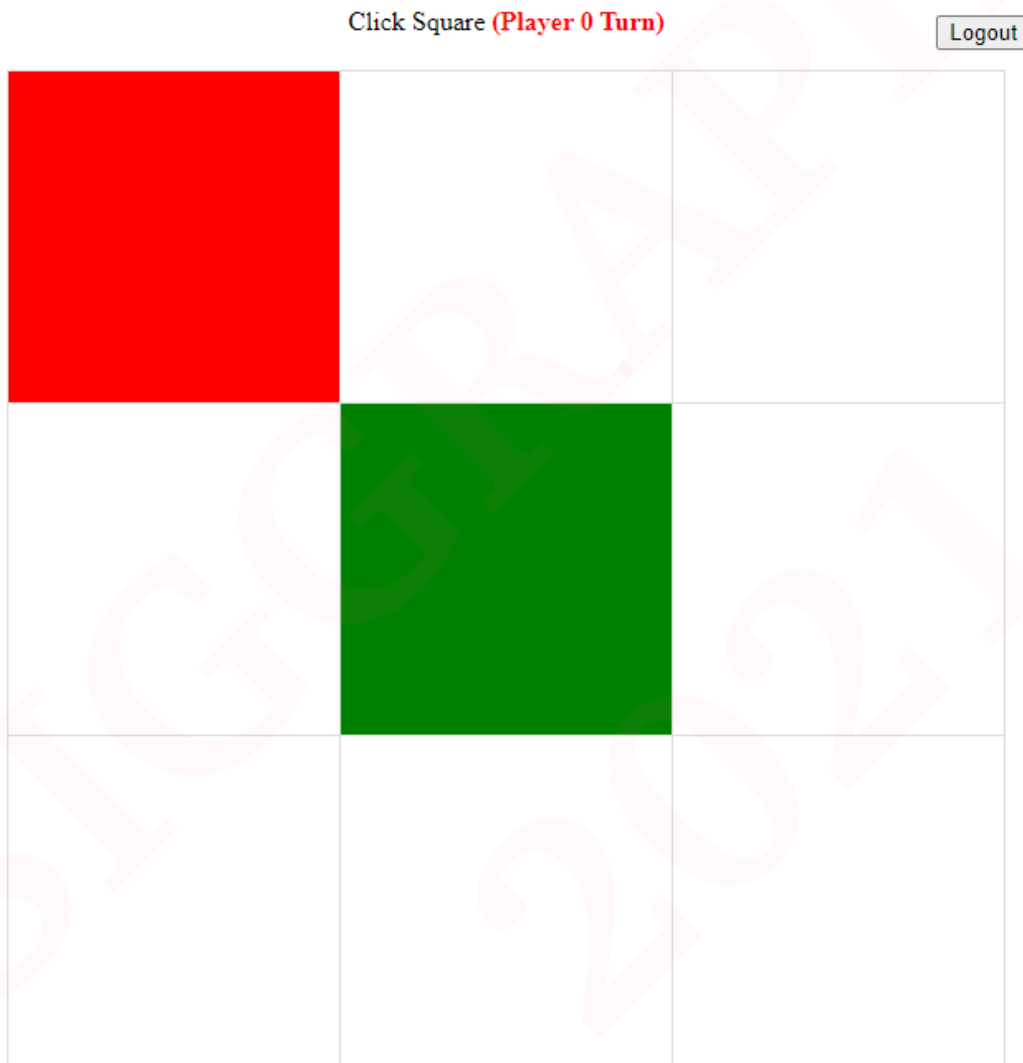
14.1 Further Ideas

The tic-tac-toe implementation is basic, but it's very easy to keep on going and adding more features. For example, change the grid size from 3x3 to 4x4 or 5x5. You can add more `join` options, such as a password key so you can only join games you've been invited.

Combine the game with other ideas from previous Chapters, such as, adding in a `chat` dialog so you can also type messages.

Other ideas:

- Add a leader board (best scores/times) - Add online database (record a players wins/loses)
- Multiple players (more than 2 players, e.g., 3 players and 3 colors)
- Add a password to join a certain game - Host or join a game
- Extra security to check for `cheating`
- Stop and save the current game
- Other board game types (instead of tic-tac-toe how about connect4 or chess)
- Share statistics such as how many games are currently running
- Record/playback games - Show details on how many moves it took to win/lose
- Add visual candy (animations and sprites)



Make a full horizontal, vertical or diagonal row the same to 'win'.

Figure 14.2: **XOX Game** - XOX game running, but instead of writing 'O' and 'X' it uses two colors (red and green).



15. Tips, Hints and Warnings

Over time, as you use NodeJS more, you'll learn useful facts to make your life a lot easier - especially in the beginning stages. Here are a few tips that you might want to remember as you progress on your NodeJS journey.

15.1 When shouldn't you use NodeJS?

NodeJS might be amazing, but you shouldn't use it for everything! NodeJS can be used for a lot of applications with various purposes. The only scenario where it should not be used is where there are long processing times, which is required by the application.

Node is structured to be single-threaded. If an application is required to carry out some long-running calculations in the background, it won't be able to process any other requests. As discussed above, NodeJS is used best where processing needs less dedicated CPU time.

15.2 Never forget about security!

NodeJS makes it very easy to setup web-based servers. However, by default NodeJS has a number of security flaws, for example it is easy for a NodeJS script to access the file system or the system environment. Hence, you need to be extra careful when you receive or process any external

information (e.g., request/send files).

15.3 Using Strict Mode In Coding

using `strict` mode in NodeJS code, can help you figure out poorly written Javascript syntax and enforces you to write better code syntax. To use this, you just add the following line in every Javascript file you create/write code on:

```
1 'use strict';  
2 //rest of your NodeJS code goes here.
```

For example, in a Javascript file without strict mode, you can initialize a new variable even without the `var` keyword.

```
1 abcd = "some string value";
```

This line of code won't result in any error. But if you're using `use strict`, it will throw a error like below:

```
1 iterate = "some string value"  
2 ~  
3 ReferenceError: abcd is not defined
```

Thus enforcing you to write little bit more structured code. Which is very useful in generating quality code with a dynamic language like Javascript.

15.4 Node shell

If you are trying to test a NodeJS function how it works, NodeJS shell works lot better for that. Just run `node` and it will bring you up the node shell. write code just like in a text editor and it will work just fine. However, when you're done, to exit the node shell, it doesn't have an `exit/quit` button or a similar command just like other command line tools. You have to either use the shortcut keys or type the valid NodeJS statement to exit the process. Here is the command you will need for that:

```
1 $process.exit(0)
```

The shortcut keys:

```
1 exit node shell: CTRL+C, twice
```

As when you start to use NodeJS/Shell using, one of the first things you'll probably ask is, how do you get out of the NodeJS/Shell!

15.5 Global variable make a good Cache

When you start with NodeJS, you'll be creating lots of small app where it can be an overhead to integrate a third-party caching mechanism. Instead, you can handle caching yourself. As in NodeJS when you declaring a global variable it persists and is accessible to all requests (has benefits for caching, but also be careful as well).

```
1  'use strict';
2
3  // This is your cache for this js file
4  var cache = {};
5
6  exports.someMethod = function(data){
7
8      /*Your logic*/
9
10     //set cache
11     cache[key] = val;
12
13     //retrieve cache
14     cachedVal = cache[key];
15
16     //delete cache
17     delete cache[key];
18
19     /*Your logic*/
20 }
```

15.6 Asynchronous looping

You get some very useful support for asynchronous control flow/looping with use of underscoreJS/AsyncJS. However, it's not that hard to do it natively as well, as long as you understand the callback concepts. So, in case you don't want to use such libraries for a simple task, you can use the following mechanism

```
1  var iterate = function(items, i){
2
3      console.log("handling: "+i);
4
5      //async methods etc
6      someAsyncMethod(params, function callback(){
7          if(items.length <= i)return;
8          return iterate(items, ++i);
9      })
10 };
11
12 var items = [1,2,3,4];
13 iterate(items, 0);
```

15.7 Quick and simple logging

You usually use the ‘console.log’ method for printing temporary info while development and some good logging libraries for logging during the deployment stages. If you are lazy enough and trying to avoid third-party logging libraries, there’s good news for you. You can still get a descent logging system with your NodeJS app using the built in native support.

The `console` object in NodeJS has several different levels for logging(error/info/warn), which you can check more here on the console object’s official documentation page. Use them appropriately as needed. Then, while running your NodeJS app, refer to a log file where you want everything written as below:

```
1 $node app.js > mylog.log
```

You are all set. All information will now be logged in your ‘mylog.log’ file, which you can check later for useful information.

One more thing about console object, you usually use the following format to print something to terminal window:

```
1 console.log("Trying to see the value of abc"+abc);
```

It works for single valued item. For JSON/Objects, you can use ‘console.log(abc)’ but won’t work with ‘+’ operator. Here you can use comma ‘,’ separator to print different values:

```
1 console.log("Trying to see the value of abc", abc);
```

15.8 Avoid using asterix * for dependencies

In your ‘package.json’ file, you can specify a version to use or keep ‘*’ to have a latest version always. It’s a good way to have all of your packages updated to their latest version which might have several improvement, bug fixes, new features included. Nevertheless, this makes good sense as long as you are working only in development mode and your product isn’t in production (release) yet. You can fix compatibility issues as they occur in development phases (not so easy when you’ve deployed your software).

```
1 //instead of this:
2 "dependencies": {
3   "clustered-node": "*"
4 }
5
6 //use this
7 "dependencies": {
8   "clustered-node": "~0.0.10"
9 }
```

Very risky to use the asterix notation for dependencies. You never know which package is gonna break when and could cause crashes or other runtime issues! So, only use the asterix in the development states, but switch to specific version based dependencies as soon as your software is to be released (final version).



Index

.js, 74
127.0.0.1, 91
applications, 70
architecture, 72
bidirectional, 98
canvas, 102
chat client, 111
client-server, 95
communication, 95
concurrency, 72
connection, 97
cross-platform, 69
custom modules, 87
downloading, 72
drawing, 95, 102
ebay, 71
event handlers, 99
event loop, 72
event-driven, 72
events, 72
express module, 108
express, 87
files, 92
fs, 83, 87
full-duplex, 95
Google, 69
hello nodejs, 74
http, 83, 87
http header, 96
install, 86
installing, 72
key press events, 107
linkedin, 71
localhost, 91
login screen, 108, 113
module, 89
modules, 83
mouse data, 99
networking, 70
nodejs, 69
package.json, 84
packages, 83
paths, 91
paypal, 71
perl, 77
php, 77
portfolio, 70
scalable, 70
tcp, 96
text editor, 74
tic-tac-toe, 113
tick, 98
tick-tock, 96
tick.html, 98
url, 83
url.parse(..), 92
V8 Javascript engine, 69
version, 72
web stack, 69
web-socket, 96
web-sockets, 95, 96
website, 89
writeHead(), 90

